

PREDICTING

THE UNKNOWN

ML WORKSHOP

PRELIMINARIES

Getting the workshop notebooks

Option A: Download as a ZIP (easiest, no account needed)

1. Go to [**https://github.com/NUS-SDS-Workshops-AY2627/Sem-1**](https://github.com/NUS-SDS-Workshops-AY2627/Sem-1)
2. Click the **green < > Code** button.
3. Click "Download ZIP" and unzip it.
4. Open the week-8-ml/ folder — it has the notebooks (**hands_on_ensemble.ipynb**, **part_1_time_series.ipynb**, **part_2_ensemble.ipynb**, **part_3_unsupervised.ipynb**) and a data/ subfolder with the CSV files already inside. Keep this folder structure as-is. Don't move the notebooks out of week-8-ml/ or separate them from data/.
5. Follow the instructions in week-8-ml/README.md to setup the notebooks.

Option B: Clone with Git (if you already have Git installed)

1. In your terminal, run: **git clone https://github.com/NUS-SDS-Workshops-AY2627/Sem-1.git**
2. In your terminal, run: **cd Sem-1/week-8-ml**
3. Follow the instructions in week-8-ml/README.md to setup the notebooks.



CLARENCE
Y3 DSE

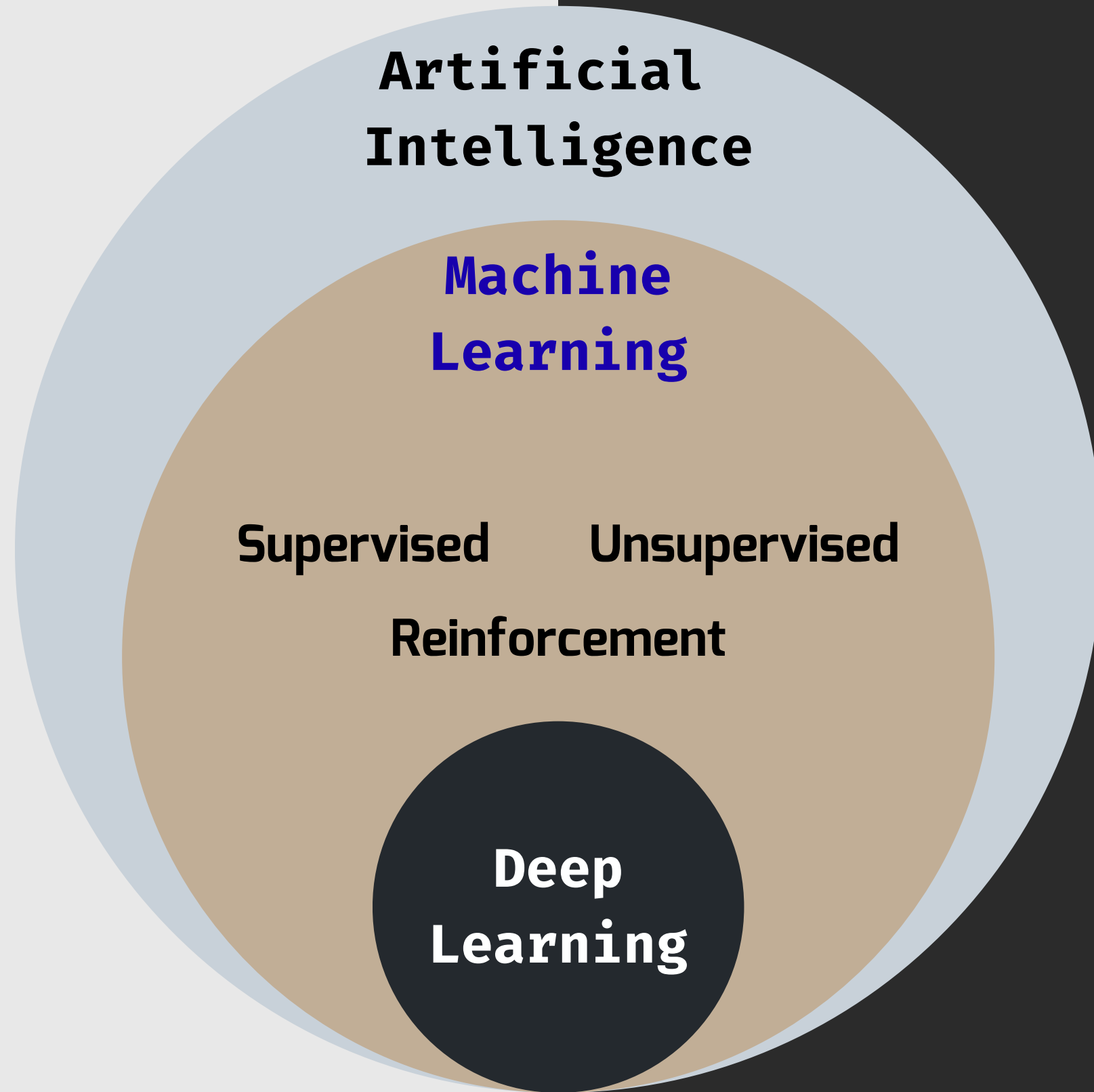


YI CHENG
Y3 DSA



BEN
Y2 DSE

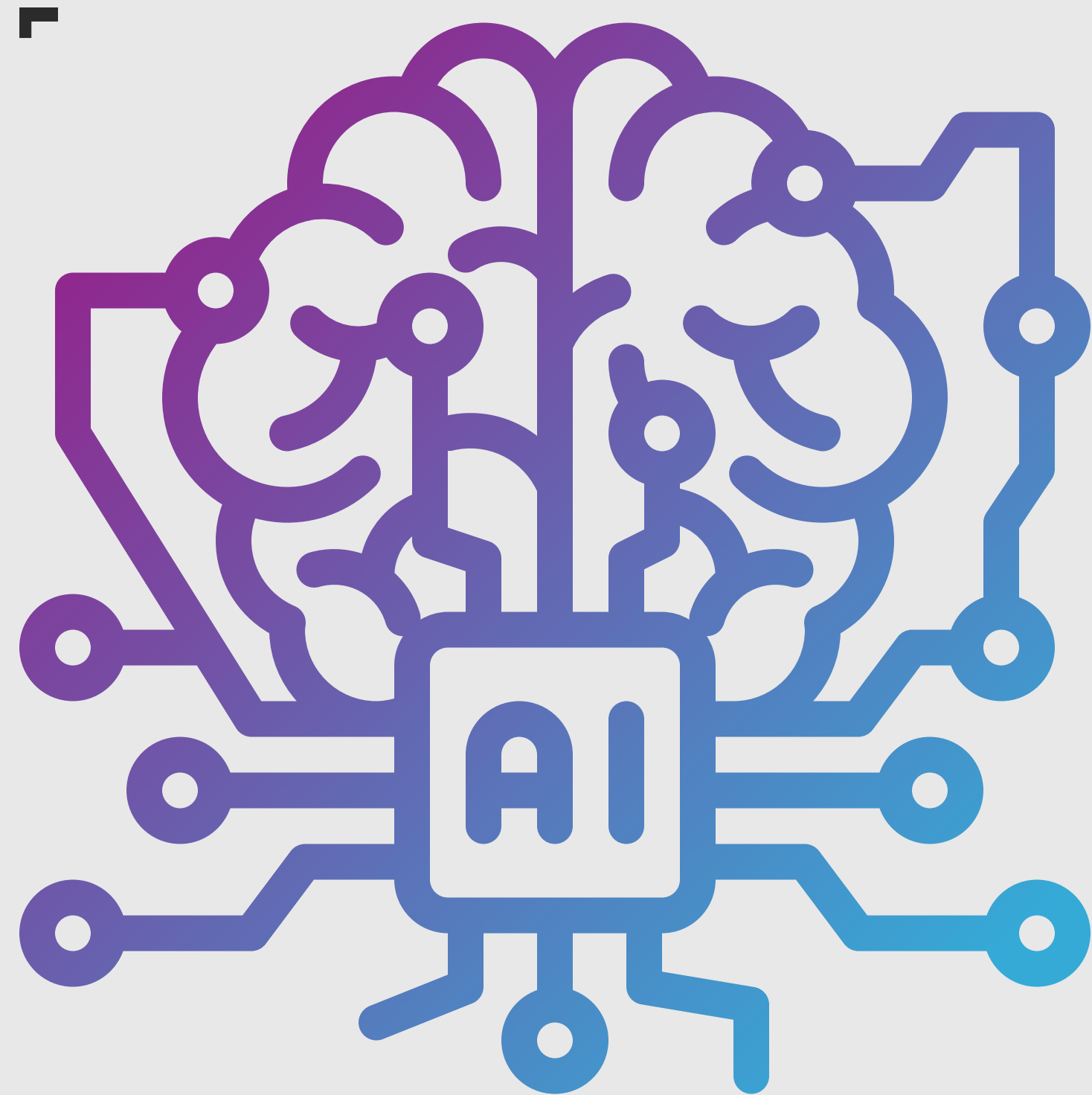




WHAT IS MACHINE LEARNING

To get computers to learn from data patterns and make decisions or predictions when given new data

...



TYPES OF MACHINE LEARNING

SUPERVISED LEARNING

UNSUPERVISED LEARNING

REINFORCEMENT LEARNING

...

SUPERVISED LEARNING



A machine learning approach that learns from labelled examples to predict outcomes for new data.



CLASSIFICATION

Predicts a discrete category or class.

EXAMPLES:

- Naive Bayes
- Logistic Regression
- Support Vector Machine



REGRESSION

Predicts a continuous numerical value.

EXAMPLES:

- Linear Regression
- Decision Trees
- Time Series Models

LABELLED DATA SET AND TEST SET



Supervised learning uses examples with known labels to learn a mapping from features to outcomes, then evaluates predictions on unseen data.



LABELLED TRAINING DATA

Used to learn the mapping.

Email features	Label
Contains 'free' = yes, Links = 3	Spam
Contains 'free' = no, Links = 0	Not spam
Contains 'free' = yes, Links = 1	Spam
Contains 'free' = no, Links = 0	Not spam
Contains 'free' = yes, Links = 2	Spam



TEST SET

Unseen examples used to evaluate predictions.

Email features	Label (unknown)
Contains 'free' = no, Links = 2	?
Contains 'free' = yes, Links = 0	?
Contains 'free' = no, Links = 1	?
Contains 'free' = yes, Links = 3	?
Contains 'free' = no, Links = 0	?



LEARN FROM LABELLED DATA

The model learns the mapping.



PREDICT UNSEEN LABELS

The model predicts the test set labels.



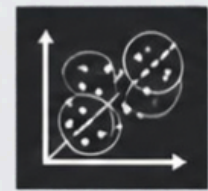
COMPARE WITH HIDDEN TRUE LABELS

Evaluate prediction accuracy.

UNSUPERVISED LEARNING



A machine learning approach that finds patterns and structure in data without labelled outcomes.

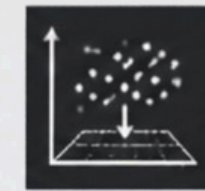


CLUSTERING

Groups similar data points into clusters.

EXAMPLES:

- KMeans
- DBSCAN

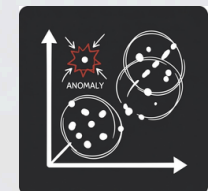


DIMENSIONALITY REDUCTION

Simplifies data by reducing number of features while preserving important patterns

EXAMPLES:

- Principal Component Analysis
- t-SNE/ UMAP



ANOMALY DETECTION

Identifies rare data points, unusual events or observations that deviate significantly

EXAMPLES:

- Isolation Forest
- Local Outlier Factor



ASSOCIATION RULE MINING

Discovers relationships, conditional co-occurrences and dependencies among variables

EXAMPLES:

- Apriori
- Frequent Pattern Growth

UNLABELLED DATA



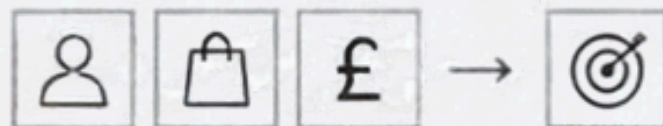
Data that has features or observations but no known target labels attached to each example.



The model must discover patterns or structure on its own.

LABELLED DATA

Features +
known outcome



VS

UNLABELLED DATA

Features only
(no outcome)



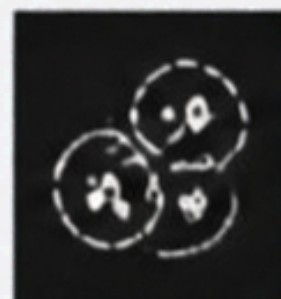
EXAMPLE: CUSTOMER DATA (UNLABELLED)

Features about customers, without any known outcome.

Age	Annual spend (£)	Visits per month
28	1,250	3
34	2,450	5
45	980	2
52	3,150	6
23	620	1
39	1,780	4



USES OF UNLABELLED DATA



CLUSTERING

Group similar
customers.



ANOMALY DETECTION

Identify unusual
behaviour.



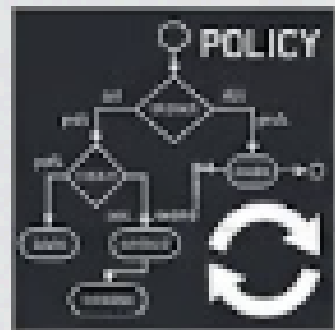
DIMENSIONALITY REDUCTION

Simplify complex data
for visualization.

REINFORCEMENT LEARNING



A machine learning approach that learns an optimal policy through trial and error, maximising a cumulative reward signal.

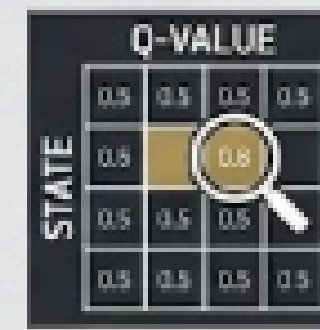


POLICY OPTIMIZATION

Iteratively improves the action-selection policy (mapping states to actions) to increase performance

EXAMPLES:

- Proximal Policy Optimisation (PPO)
- Trust Region Policy Optimisation (TRPO)
- Evolutionary Strategies



Q-LEARNING & VALUE BASED

Learns the value (Q-value or state value) of each action or state, then derives the best policy

EXAMPLES:

- Deep Q-Network (DQN)
- State-Action-Reward-State-Action
- Actor-Critic Methods

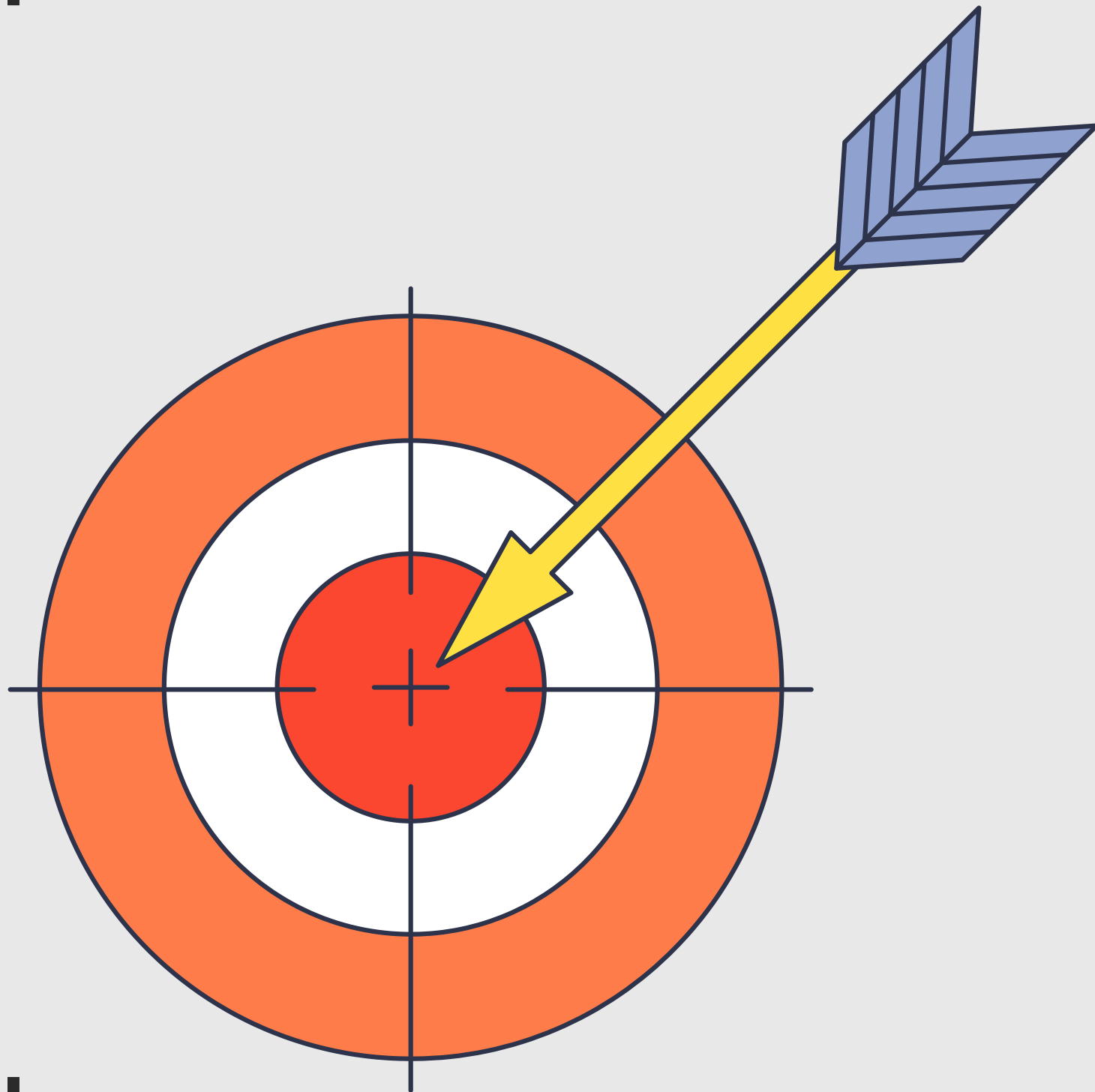
ERROR METRICS

How do we measure
success?

Regression Metrics

Classification Metrics

...



REGRESSION METRICS

MEAN ABSOLUTE ERROR (MAE)

You care about the average size of the errors more than penalising large errors. If the data has many outliers, this is a robust metric to use.

MEAN SQUARED ERROR

You want to penalise large errors. The standard metric to use for further analysis as it is differentiable.

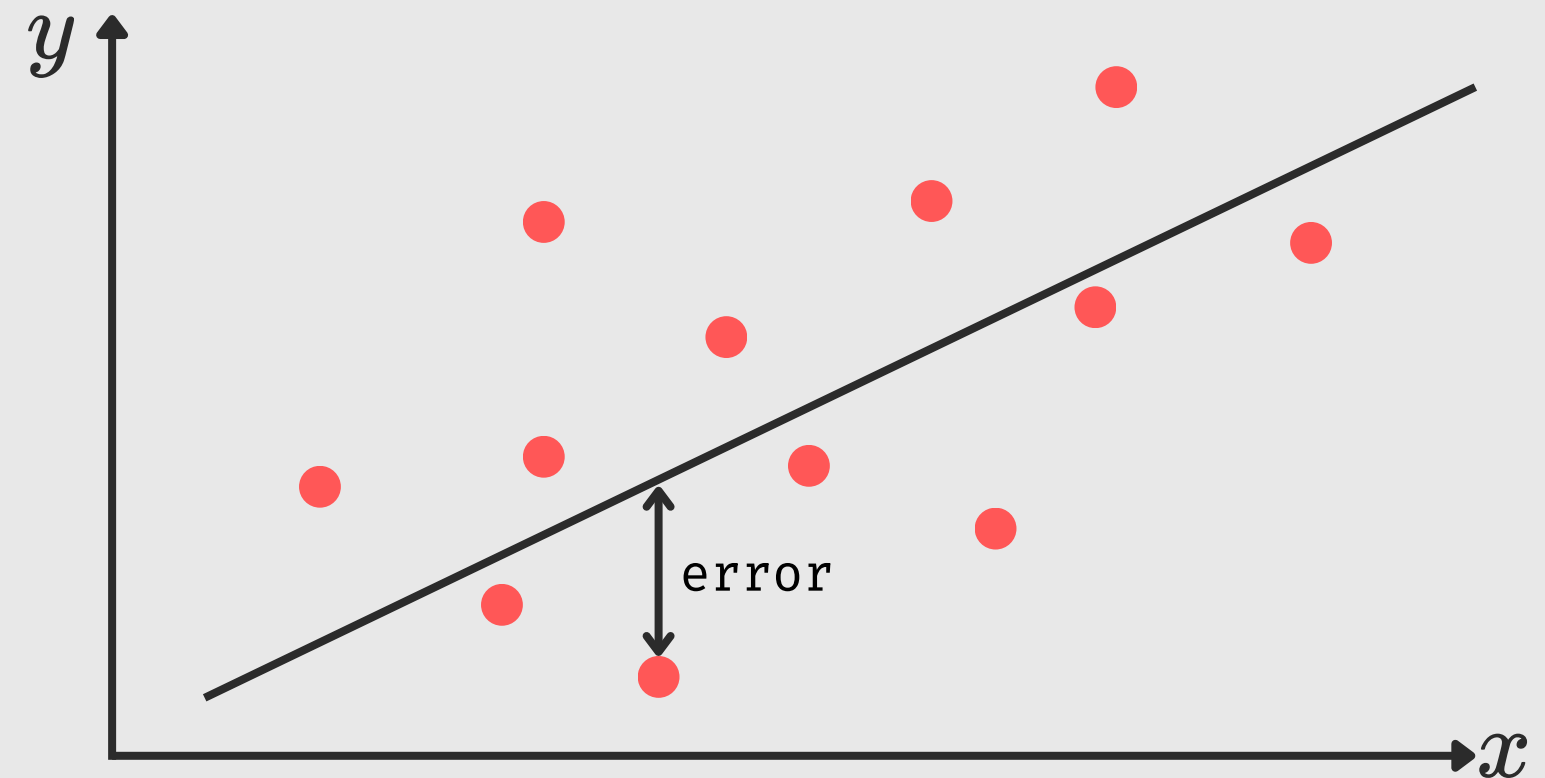
ROOT MEAN SQUARED ERROR

Very similar to MSE but on a more reportable scale. Units are comparable to the response variable.

$$MAE = \frac{1}{N} \sum_{i=1}^N |Y_i - \hat{Y}_i|$$

$$MSE = \frac{1}{N} \sum_{i=1}^N (Y_i - \hat{Y}_i)^2$$

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (Y_i - \hat{Y}_i)^2}$$



CONFUSION MATRIX

...

	<u>Actual Positive</u>	<u>Actual Negative</u>
<u>Predicted Positive</u>	True Positive	False Positive
<u>Predicted Negative</u>	False Negative	True Negative

Actual FP FN

 ● ● ● ● ●

Predicted ● ● ● ● ●

 ✓ ✓ ✗ ✓ ✗

 ● ●

 ● ●

 Positive

 Negative

SENSITIVITY

$$\frac{TP}{TP + FN}$$

Aim to decrease False Negative classification.

Use when missing a positive case is costly.

e.g., Cancer Detection

SPECIFICITY

$$\frac{TN}{TN + FP}$$

Aim to decrease False Positive classification.

Use when incorrectly classifying as positive is costly.

e.g., Doping Test

PRECISION

$$\frac{TP}{TP + FP}$$

Aim to decrease false positive classifications among predicted positives

Use when incorrectly predicting Positive is costly.

e.g., Spam Detection

CONTENTS

1. LINEAR REGRESSION
2. LOGISTIC REGRESSION
3. TIME-SERIES
 - A. MOVING AVERAGE SMOOTHING
 - B. NAIVE METHODS
 - C. SIMPLE EXPONENTIAL SMOOTHING
4. ENSEMBLE LEARNING
 - A. BAGGING
 - B. BOOSTING
 - C. STACKING

SUPERVISED LEARNING

1. KMEANS
2. DBSCAN

UNSUPERVISED LEARNING

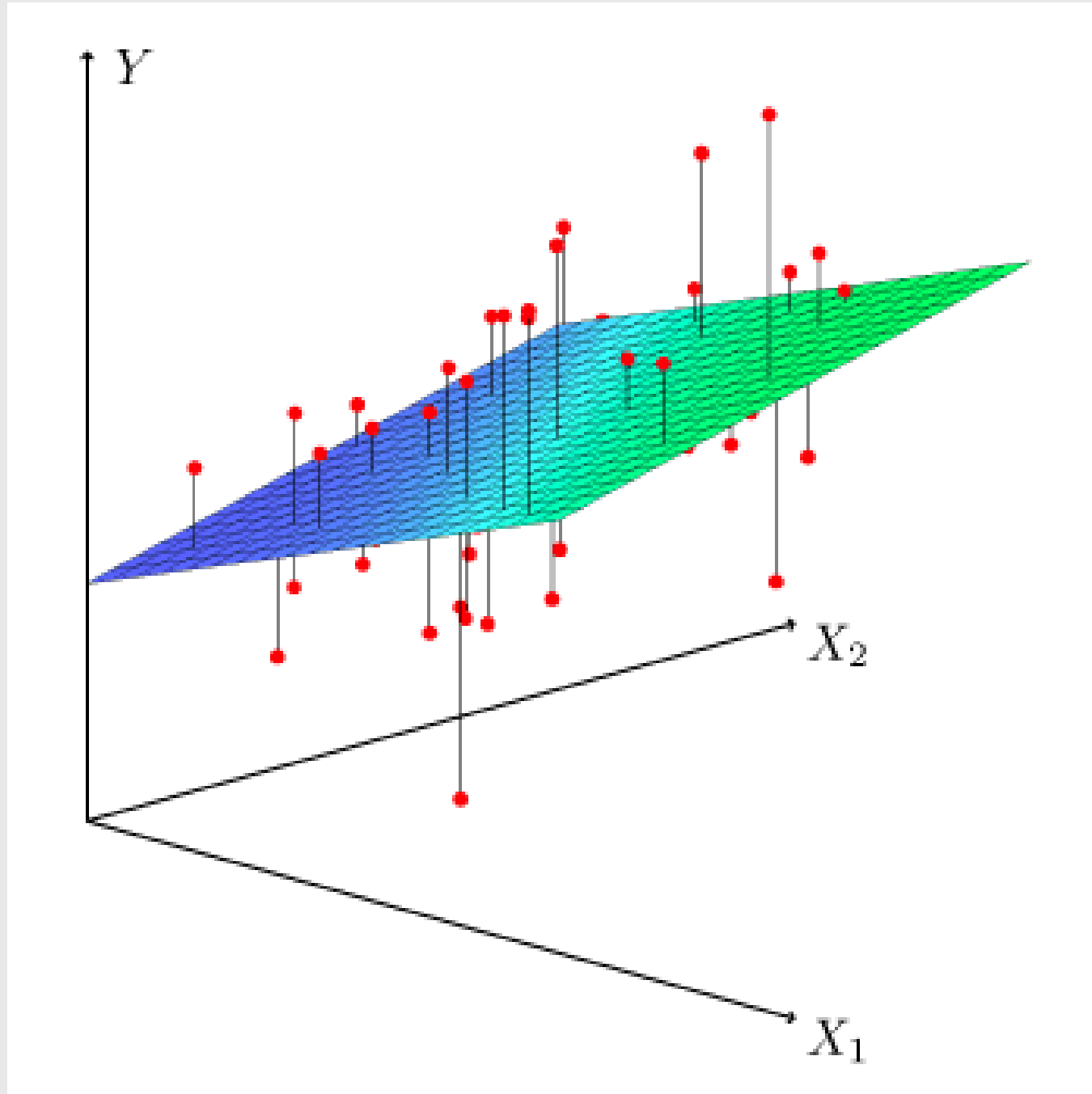
1. ARMA (SUPERVISED - TIME-SERIES)
2. APRIORI (UNSUPERVISED - ASSOCIATION RULE MINING)

IN APPENDIX

...

LINEAR REGRESSION

Baseline Regression
Model



...

MECHANISM

Linear regression draws a straight line that best follows the data, then uses that line to make predictions.

MODEL SPECIFICATION

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

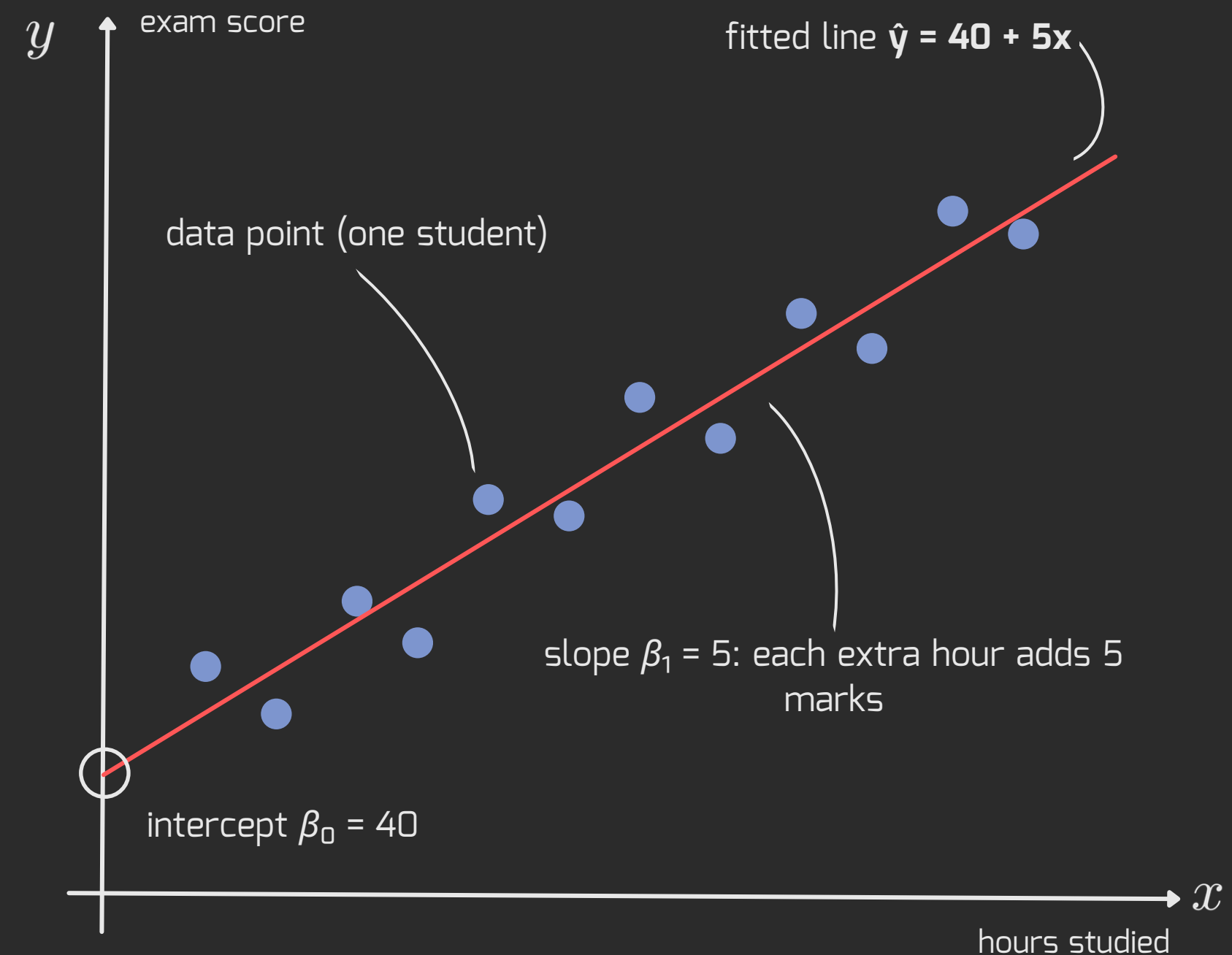
y_i exam score (outcome) β_1 slope / gradient

x_i hours studied (predictor) ε_i error (noise)

β_0 intercept

Example: 12 students, with x = hours studied and y = exam score. The best line through them is $\hat{y} = 40 + 5x$.

VISUALISATION



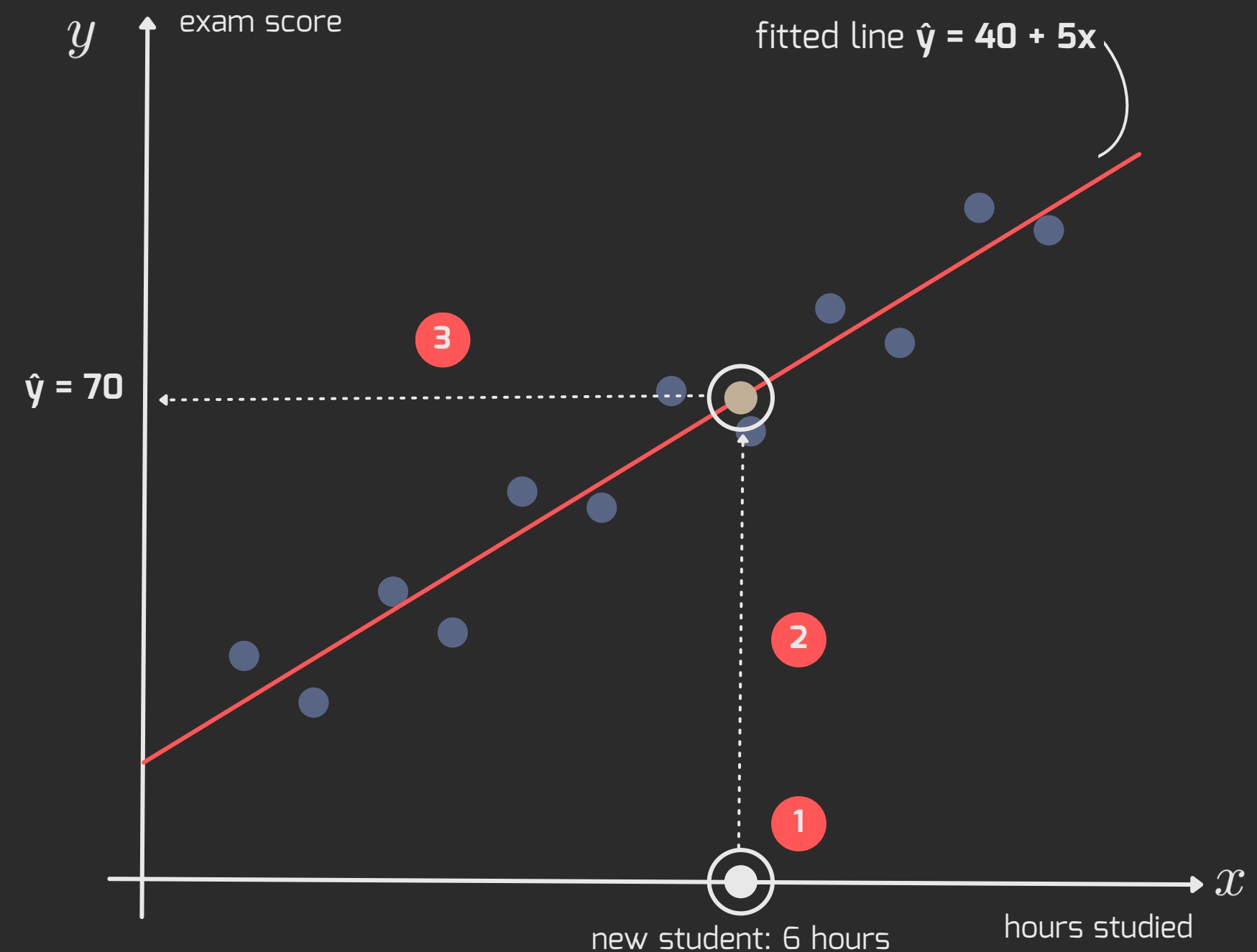
PREDICTION

A new student tells us they studied 6 hours. What exam score should we predict?

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x = 40 + 5 \times 6 = 70$$

Key idea: once trained, the whole model is two numbers, β_0 and β_1 . Predicting is just plugging x into the line.

VISUALISATION



ESTIMATION

How do we figure out how to plot this line?

LEAST SQUARES

$$\min_{\beta_0, \beta_1} \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 x_i))^2$$

Residuals = (actual - predicted)

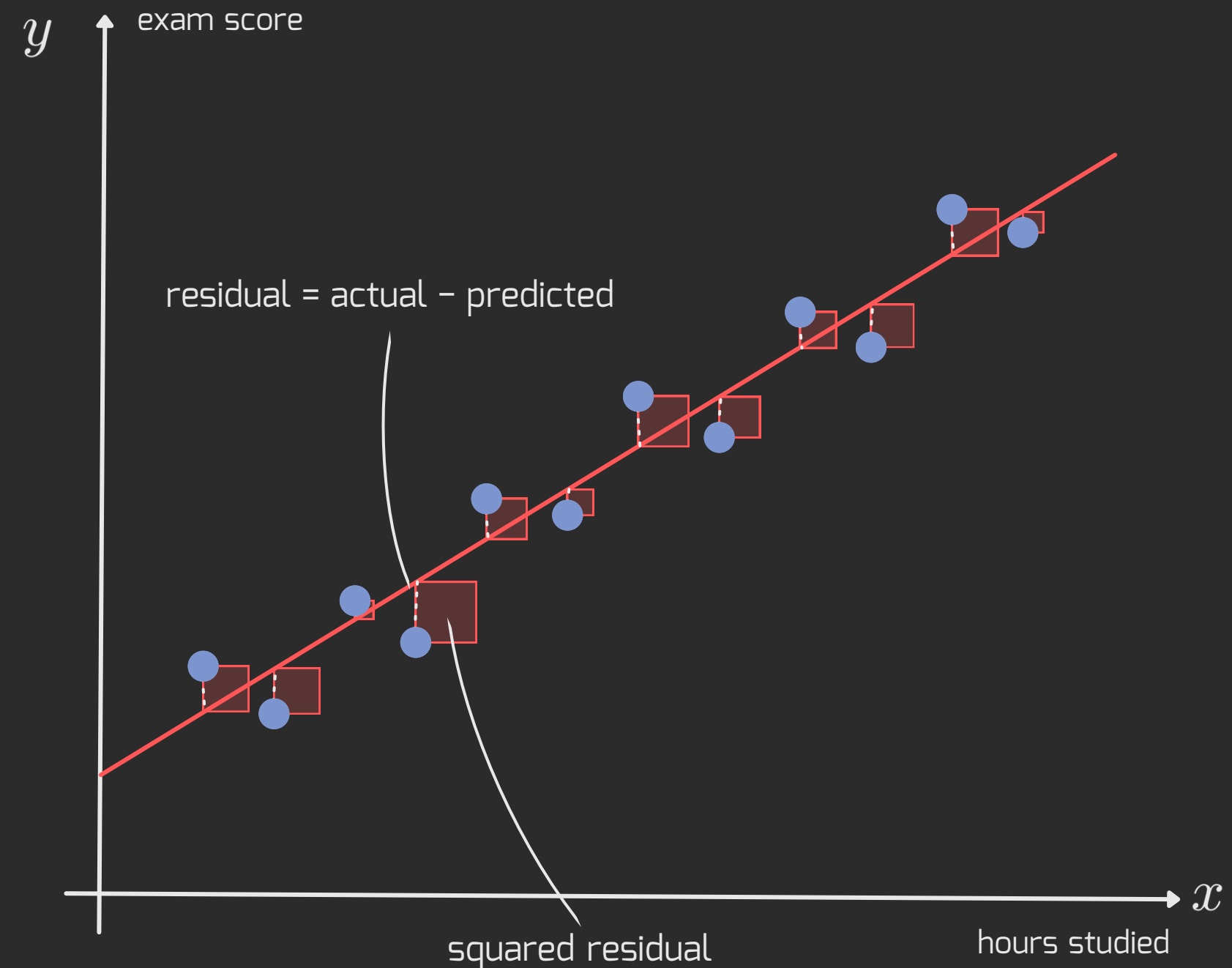
Least squares picks the β_0 and β_1 that make the sum of squared residuals as small as possible.

GOODNESS OF FIT

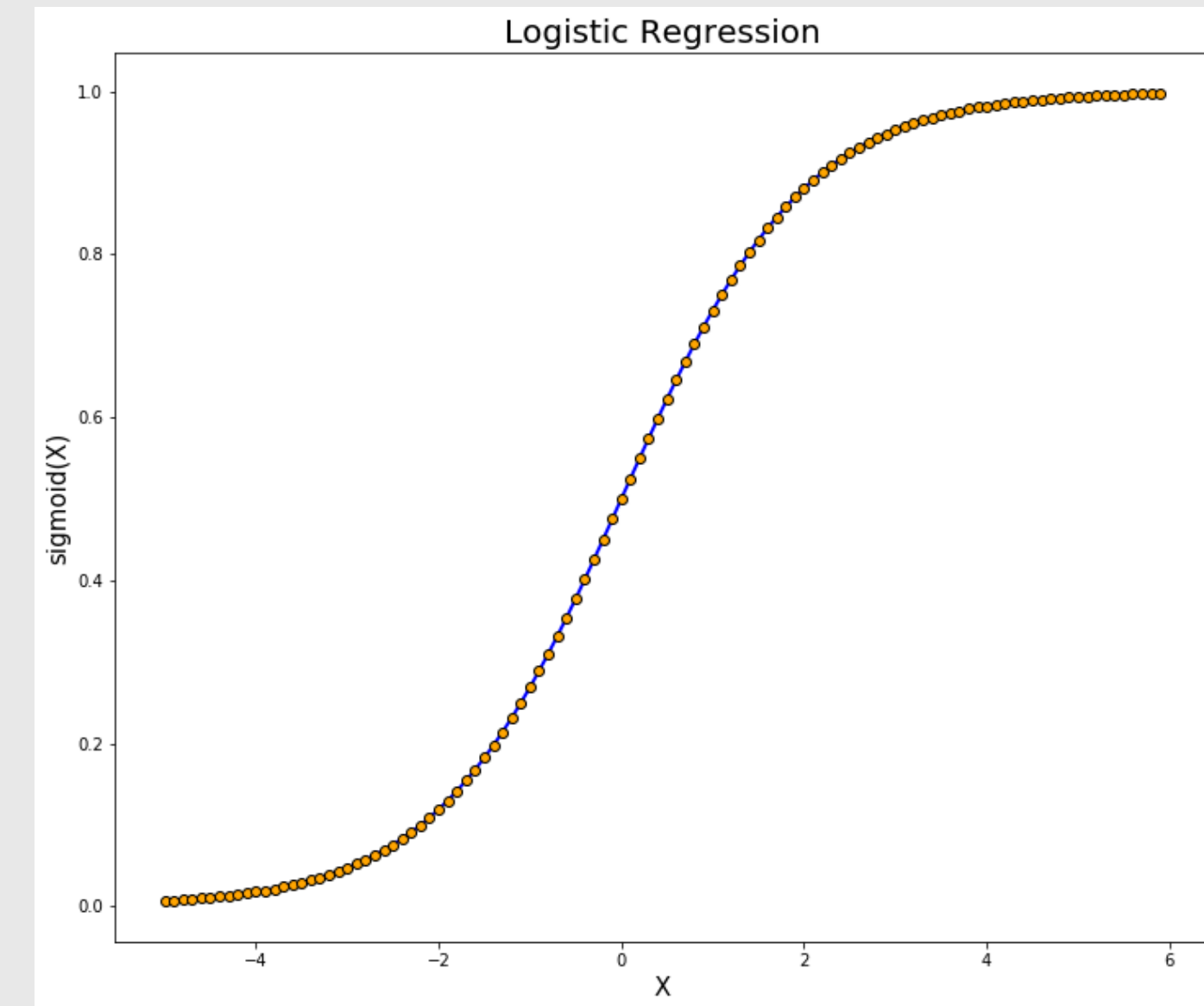
$$R^2 = \frac{\sum (\hat{y}_i - \bar{y})^2}{\sum (y_i - \bar{y})^2}$$

R^2 is the share of the variation in y that the line explains. 1 means every point sits on the line.

VISUALISATION



Least squares makes the total red area as small as possible.



LOGISTIC REGRESSION

Baseline Classification Model



MECHANISM

Say now we want to classify students based on the probability - how likely is this student passing?

Fit a Sigmoid curve to labeled pass/fail data. Assigning **Pass = 1** and **Fail = 0** allows us to transform hours studied into a continuous probability between 0 and 1.

MODEL SPECIFICATION

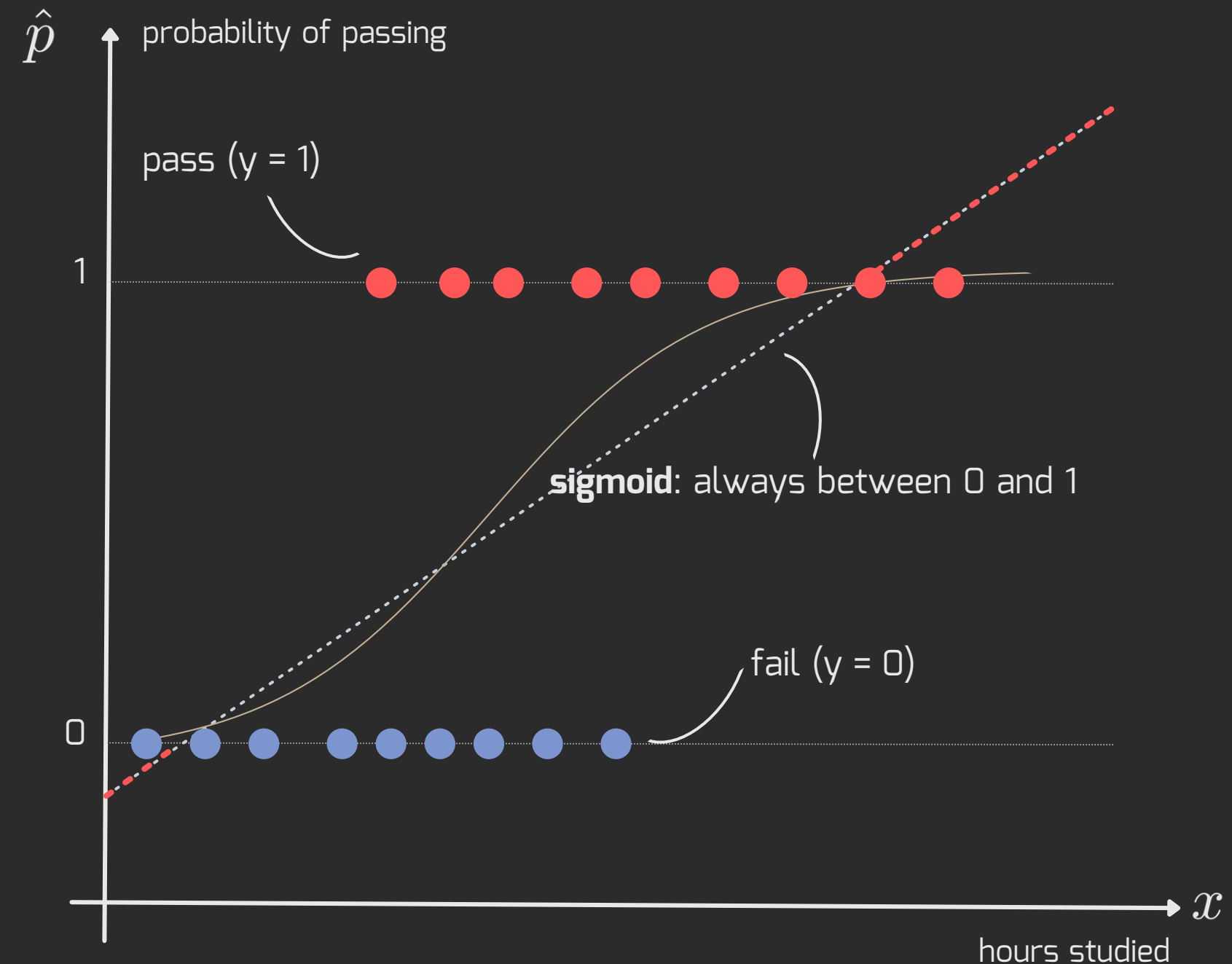
$$z = b_0 + b_1 x$$

the straight line formula for linear regression

$$\hat{p} = \frac{1}{1 + e^{-z}}$$

the sigmoid squeezes any z into a probability between 0 and 1

VISUALISATION



PREDICTING NEW DATA

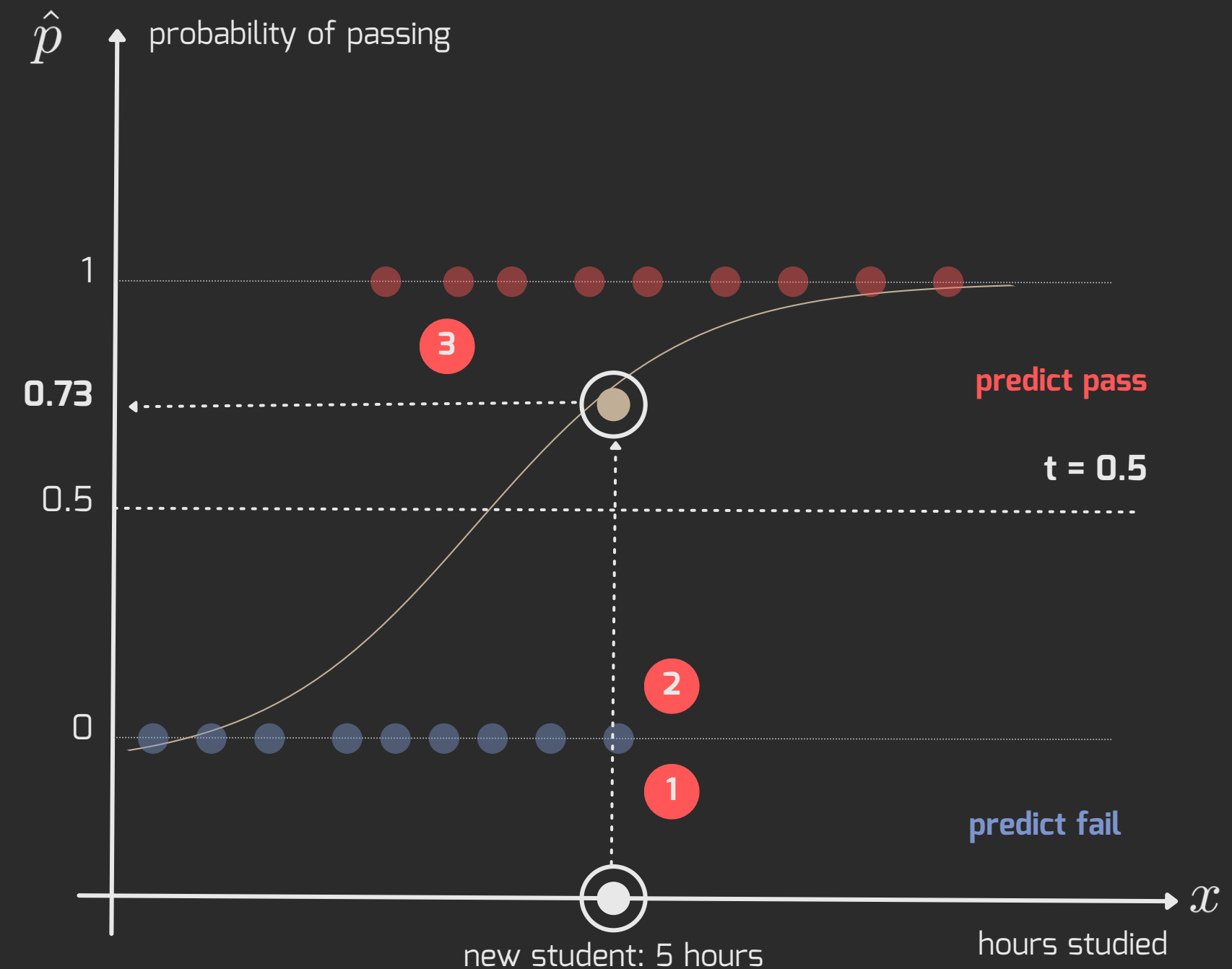
A new student studied 5 hours. Will they pass? Suppose training gave $\mathbf{b}_0 = -4$ and $\mathbf{b}_1 = 1$.

THREE STEPS

- 1 Plug x into the line
 $z = -4 + 1 \times 5 = 1$
- 2 Convert Z into probability using the Sigmoid
 $\hat{p} = \frac{1}{1 + e^{-1}} \approx 0.73$
- 3 Compare with the threshold $t = 0.5$
 $0.73 \geq 0.5 \Rightarrow \hat{y} = 1$ (pass)

Key idea: We estimate a 73% probability of passing for a student who studied 5 hours. Since this exceeds our threshold, we predict the student will pass.

VISUALISATION



DECISION THRESHOLD

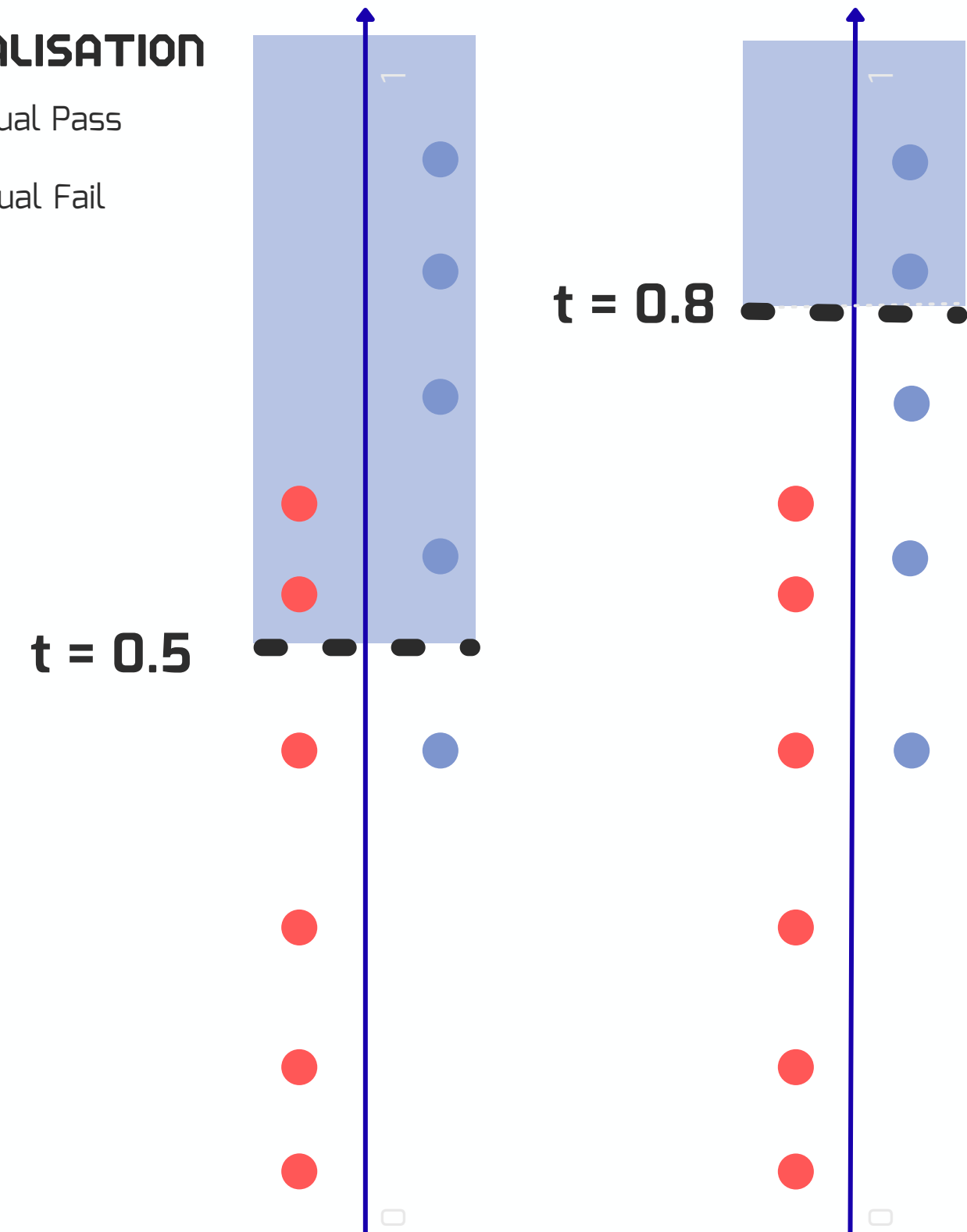
Setting the decision threshold is left to us, as shifting the threshold comes with trade-offs.

- **Lower t (0.5):** Having a lenient threshold catches most of the students who actually pass but misclassifies a few students who actually failed.
- **Higher t (0.8):** Having a stricter threshold misclassifies no students who failed but misses some students who actually passed.

Hence, depending on what we value more in our problem, we adjust the threshold accordingly.

VISUALISATION

- Actual Pass
- Actual Fail

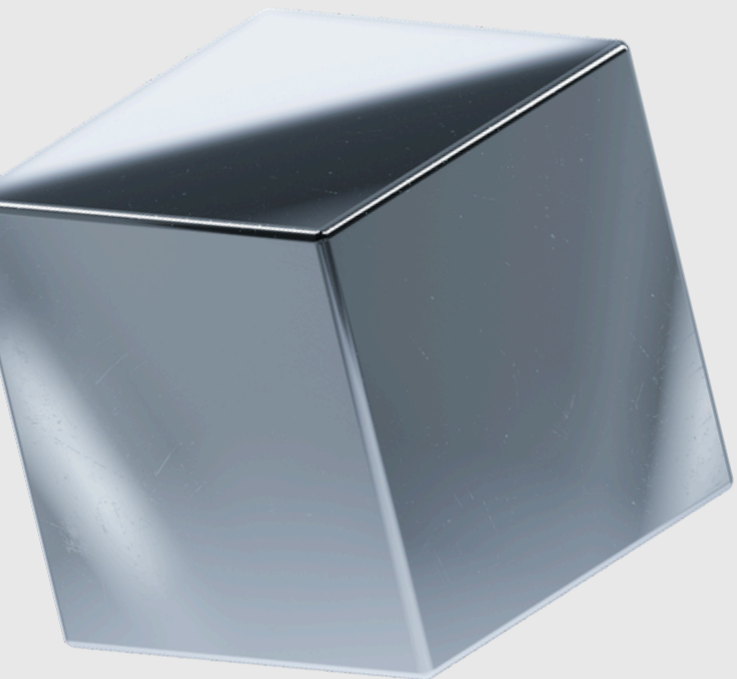


Finds 4/5 students who actually passed and misclassifies two students who actually failed.

Finds 2/5 students who actually passed and misclassifies no one who actually failed



**K – NEAREST
NEIGHBOUR**



KNN

K-NEAREST NEIGHBOURS

Objective: To classify a new data point using the labelled points closest to it.

Idea: Points that are close together tend to share the same label.

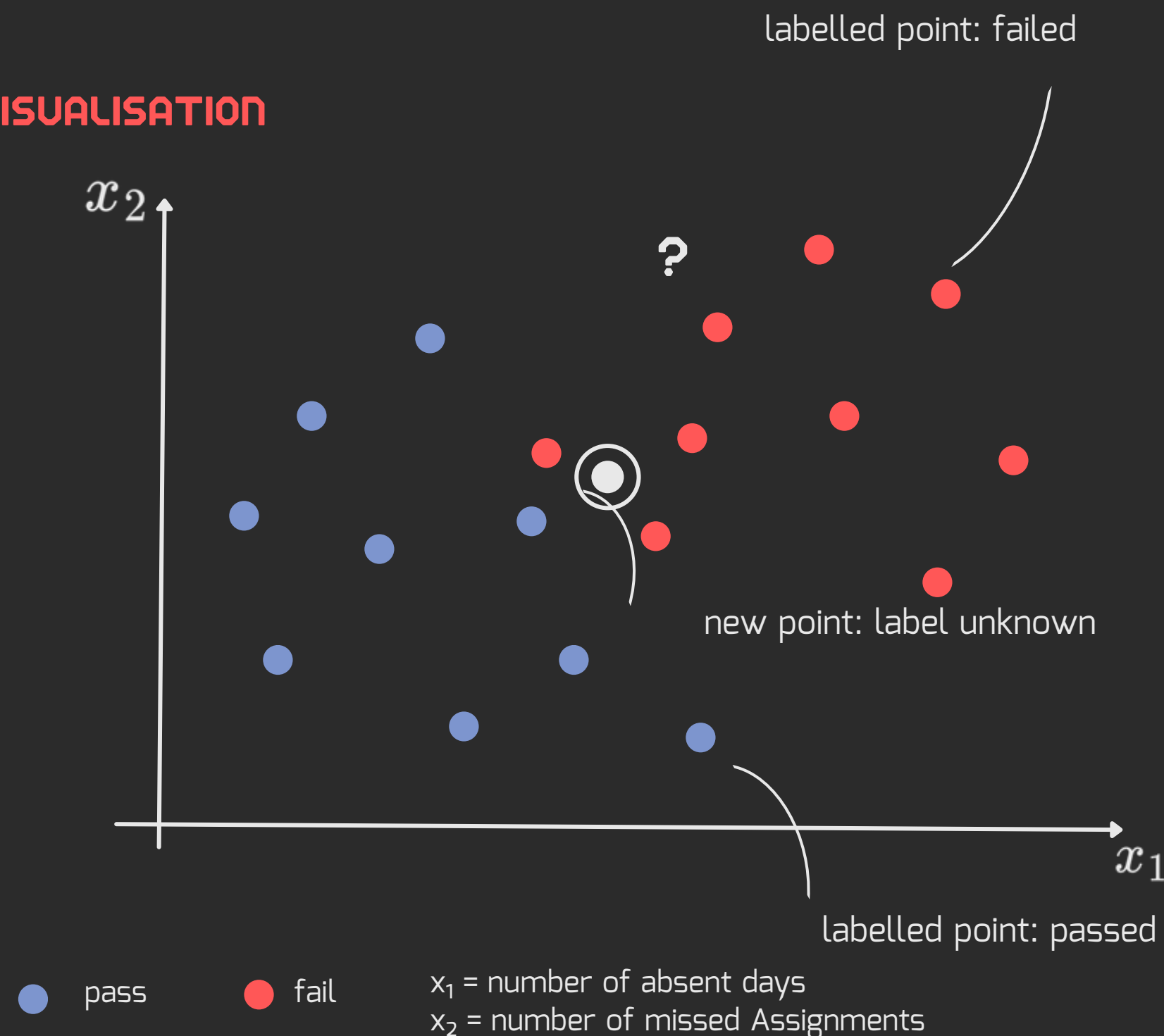
A LAZY LEARNER

- Training is just storing the labelled data. No parameters are learned.
- The work starts only when a new point arrives where KNN measures its distance to every training point.

Majority vote

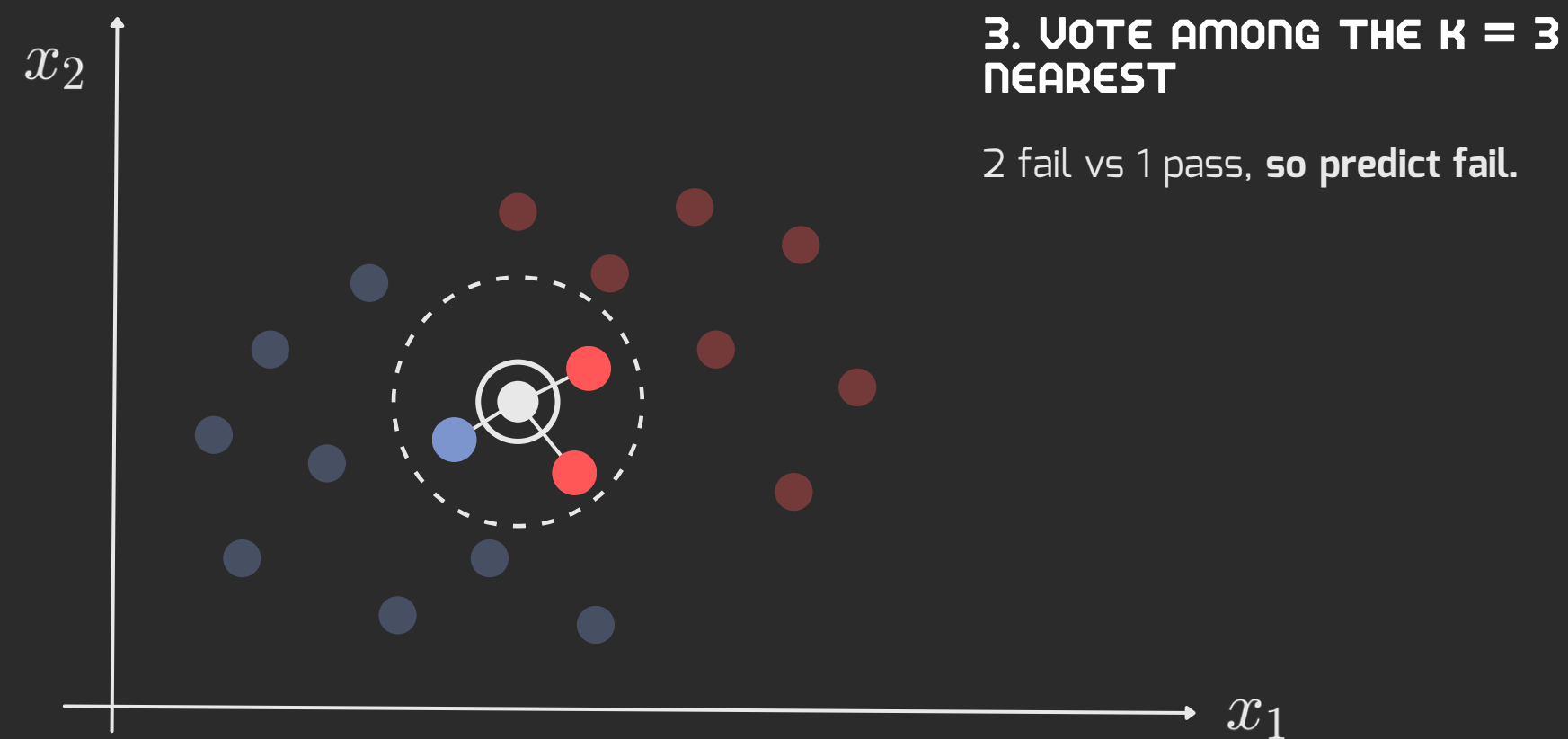
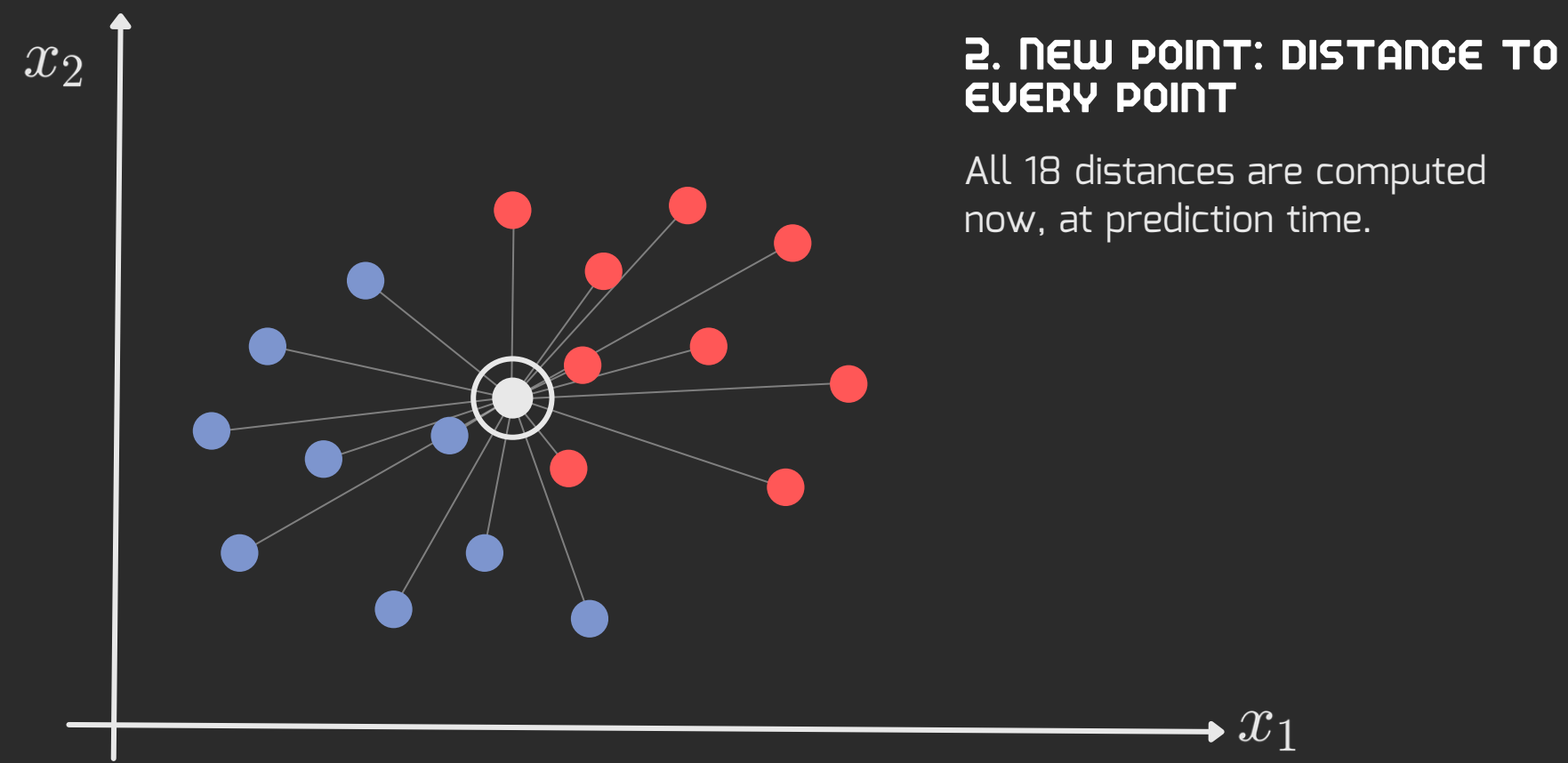
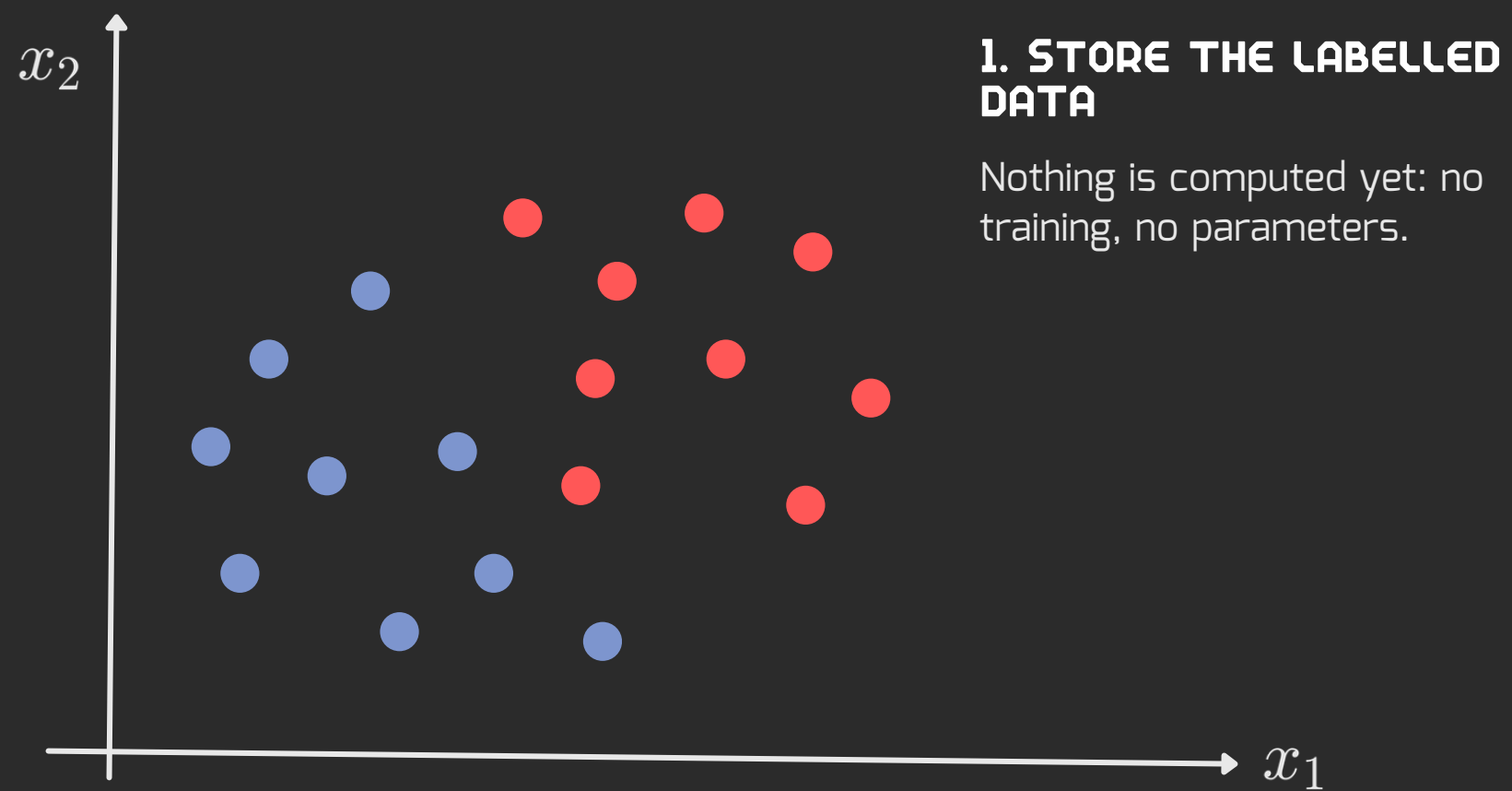
The new point takes the most common label among its k nearest neighbours. With $k = 3$: two fails and one pass, so predict fail.

VISUALISATION

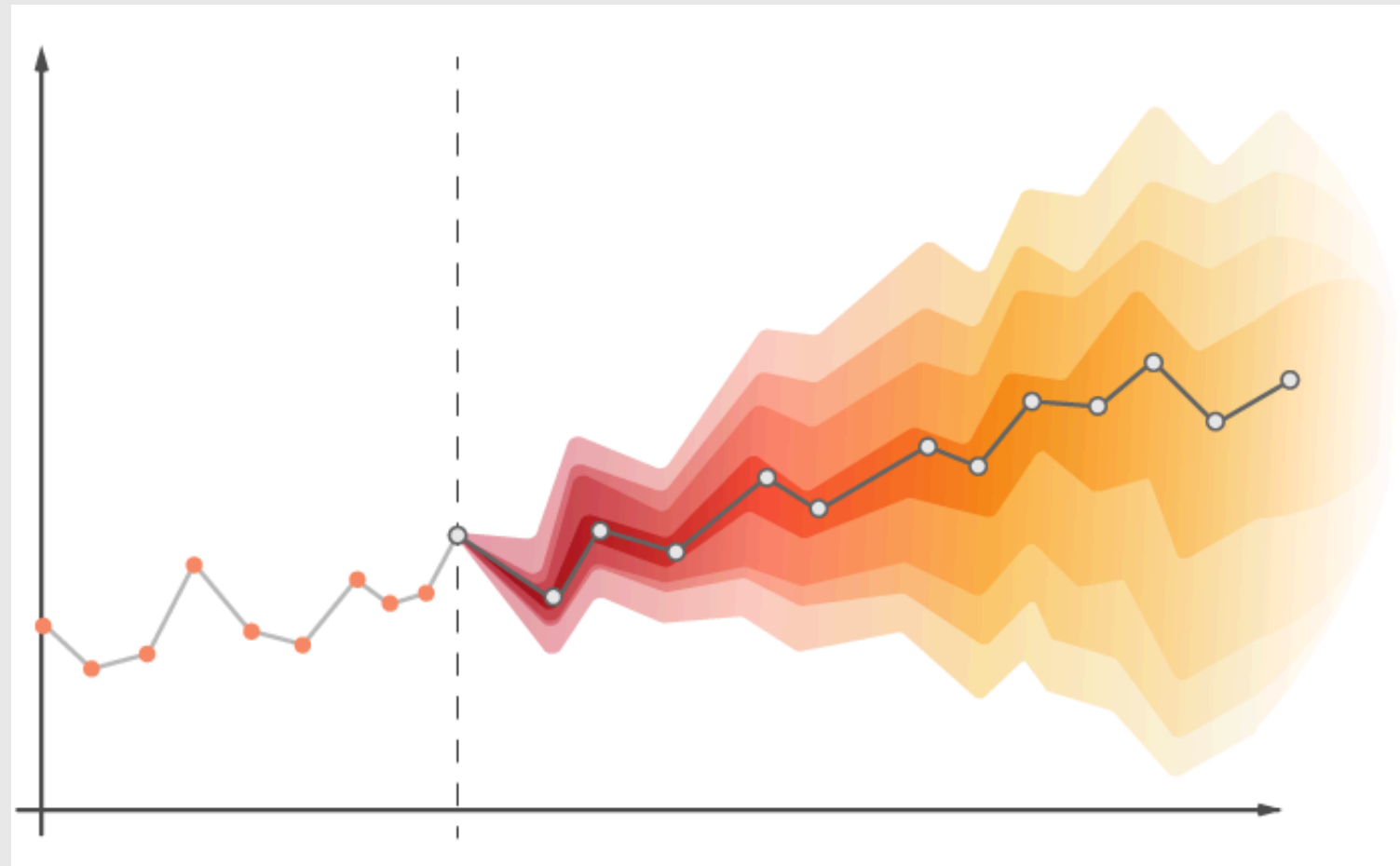


VISUALISING KNN

k is ours to choose, not learned. An odd k avoids ties between two classes.



TIME SERIES

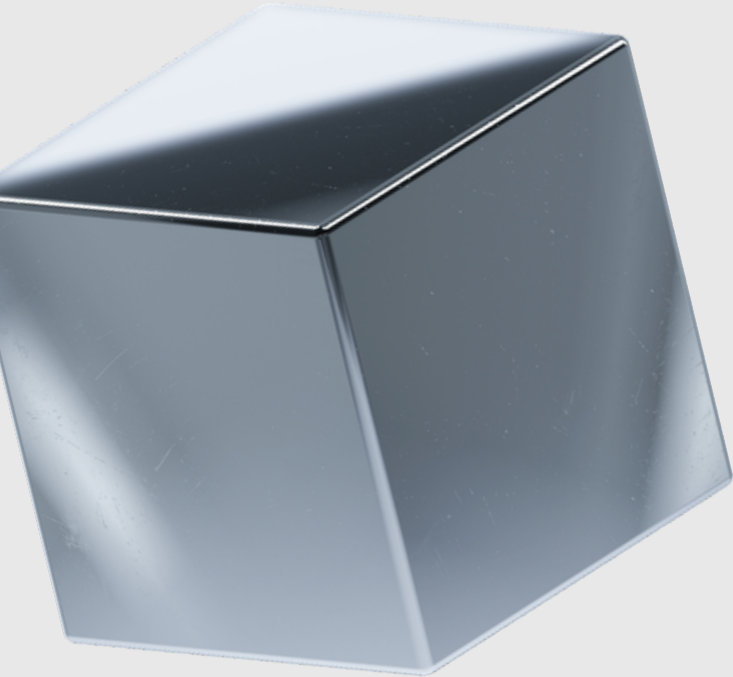


FINDING RELATIONSHIP
BETWEEN DIFFERENT TIME
PERIODS

...



TIME SERIES



WHAT IS TIME SERIES DATA?

A sequence of measurements recorded at ordered points in time.
E.g. Daily NUS bus ridership, gdp growth measured in years

KEY CHARACTERISTICS

- **Timestamp:** Every data point has a specific date or time.
- **Intervals:** Usually regular steps: hours, days, weeks or years.
- **Order:** The order carries useful information. Oftentimes, the past is a good way to figure out what the future looks like, so we should keep the order intact.

How is it used?

- **Spot trends:** Does the data show rising or falling trend in the long run? e.g. growth in stock price of Nvidia since 2024.
- **Find patterns:** Cycles that repeat, like retail hiring increasing in December.
- **Make forecasts:** Use the past to estimate what comes next, e.g. estimating the number of exports of a country next year.

TREND, SEASONALITY, REMAINDER

Most series can be split into three parts:

$$y = \text{trend} + \text{seasonal} + \text{remainder}$$

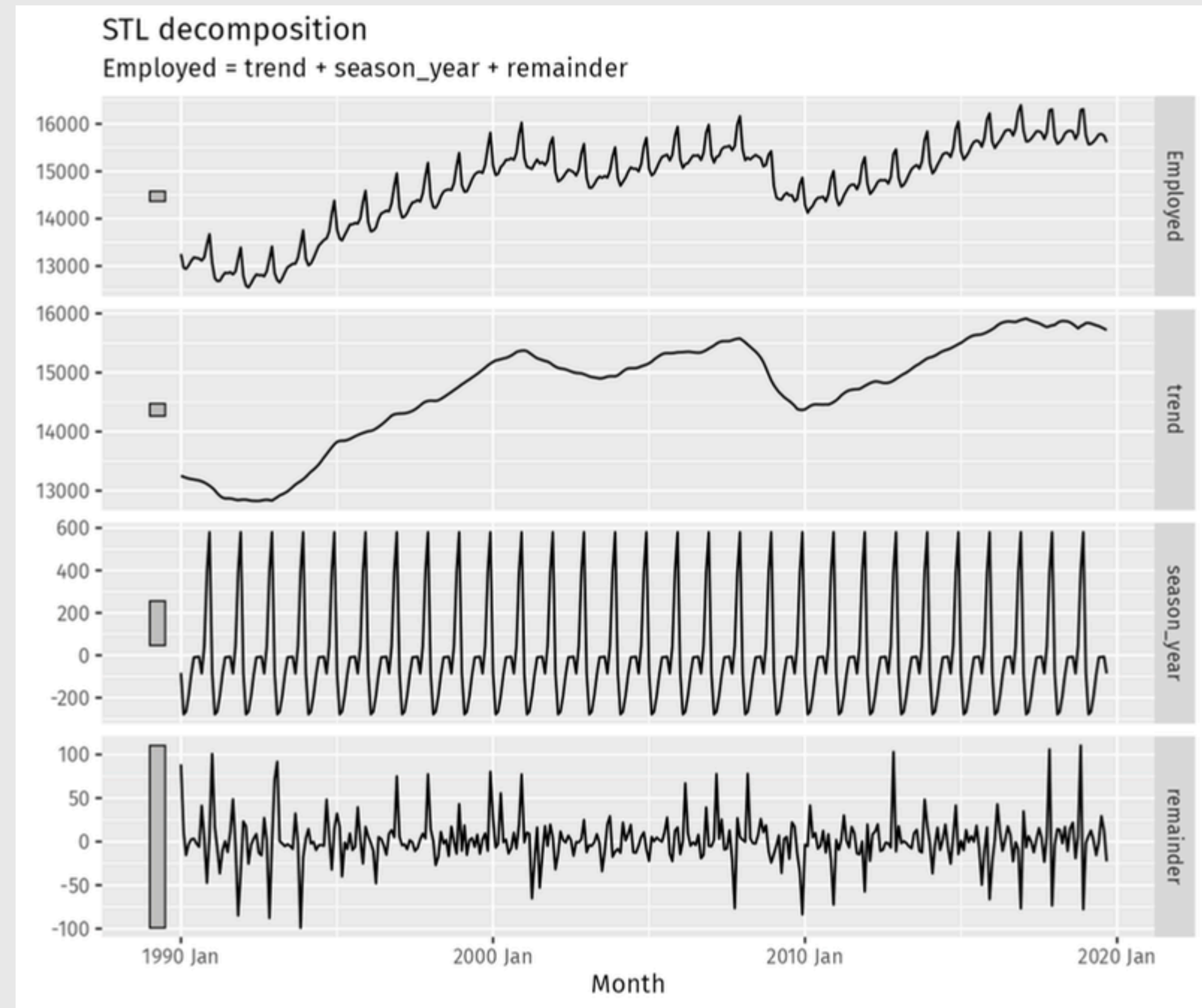
The chart shows this for US retail employment.

TIME SERIES DECOMPOSITION

Trend: the long run direction (up, down, flat),
e.g. Long run growth in employment

Seasonality: a pattern with a fixed, known
period, e.g. retail hiring every December.

Remainder: whatever is left, the irregular
noise.



MOVING AVERAGE SMOOTHING

$$\hat{Y}_t = \frac{Y_{t-\frac{k-1}{2}} + \cdots + Y_{t-1} + Y_t + Y_{t+1} + \cdots + Y_{t+\frac{k-1}{2}}}{k}$$

- Used in time series analysis to smooth out short-term fluctuations, noise, or random spikes in data
- See the underlying trend or pattern more clearly.

How it works:

1. Pick a Window Size (K): For example, a 3-period moving average (K = 3).
2. Calculate the Average: For any point in time, take the average of that data point and the $(K-1)/2 = 1$ data points before and after it.
3. Move one step forward in time and repeat the calculation for the next point.

Quarter	Sales	3-Qtr MA
Q1	10	—
Q2	12	12
Q3	14	13
Q4	13	14.3
Q5	16	15.7
Q6	18	

MOVING AVERAGE SMOOTHING

[Yahoo finance link](#)



Reading the chart

Three moving averages of the daily price:

10 day (orange),
50 day (red) and
200 day (green).

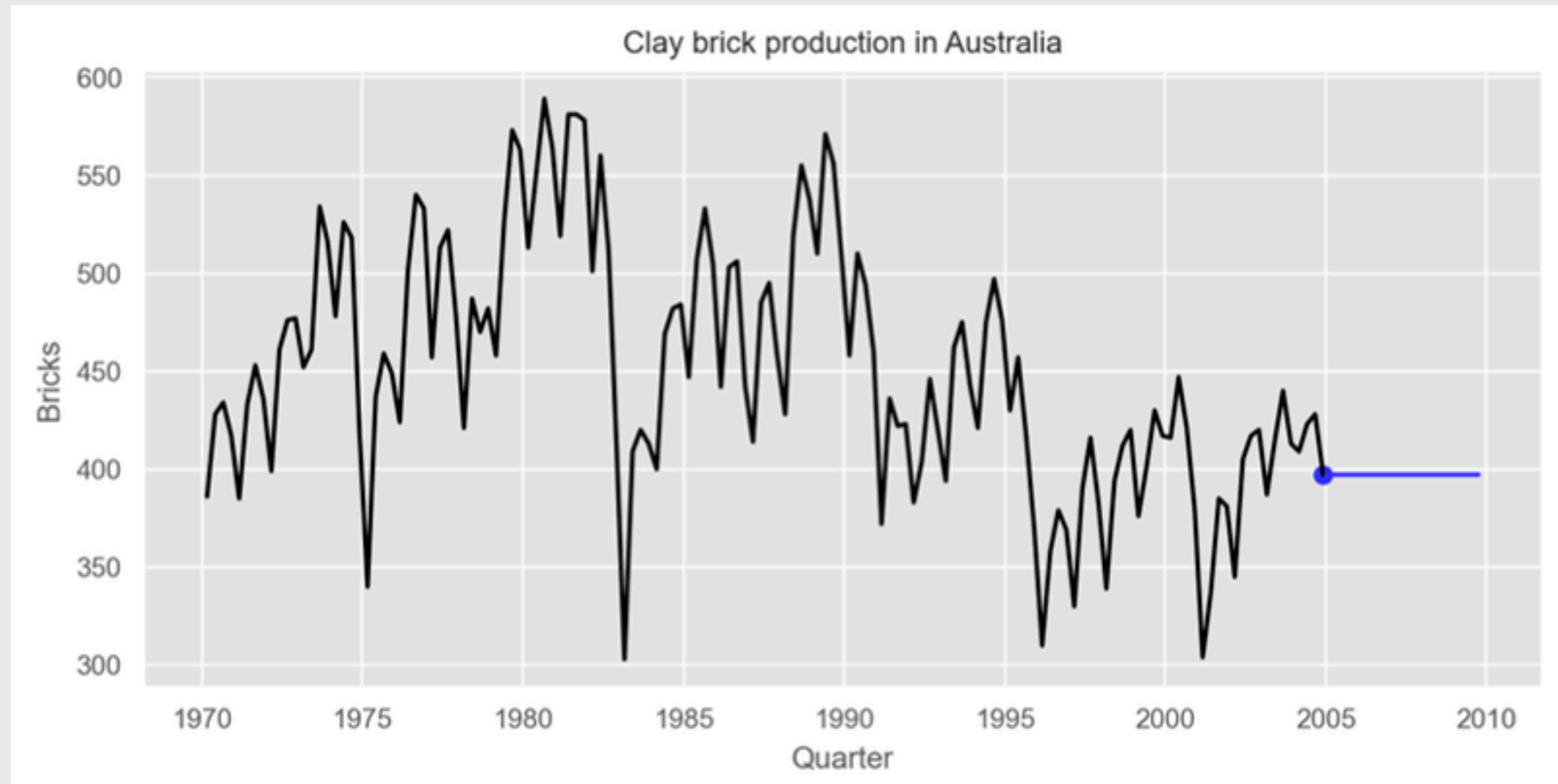
The longer the
window, the
smoother the
line, and the later
it turns.

BASIC FORECASTING TECHNIQUES

NAIVE METHOD

- Assume the next datapoint is the previous value.
- Surprisingly hard to beat for series that appears to fluctuate randomly, e.g. stock prices: tomorrow's best guess is often today's price.

$$\hat{y}_{T+h|T} = y_T,$$

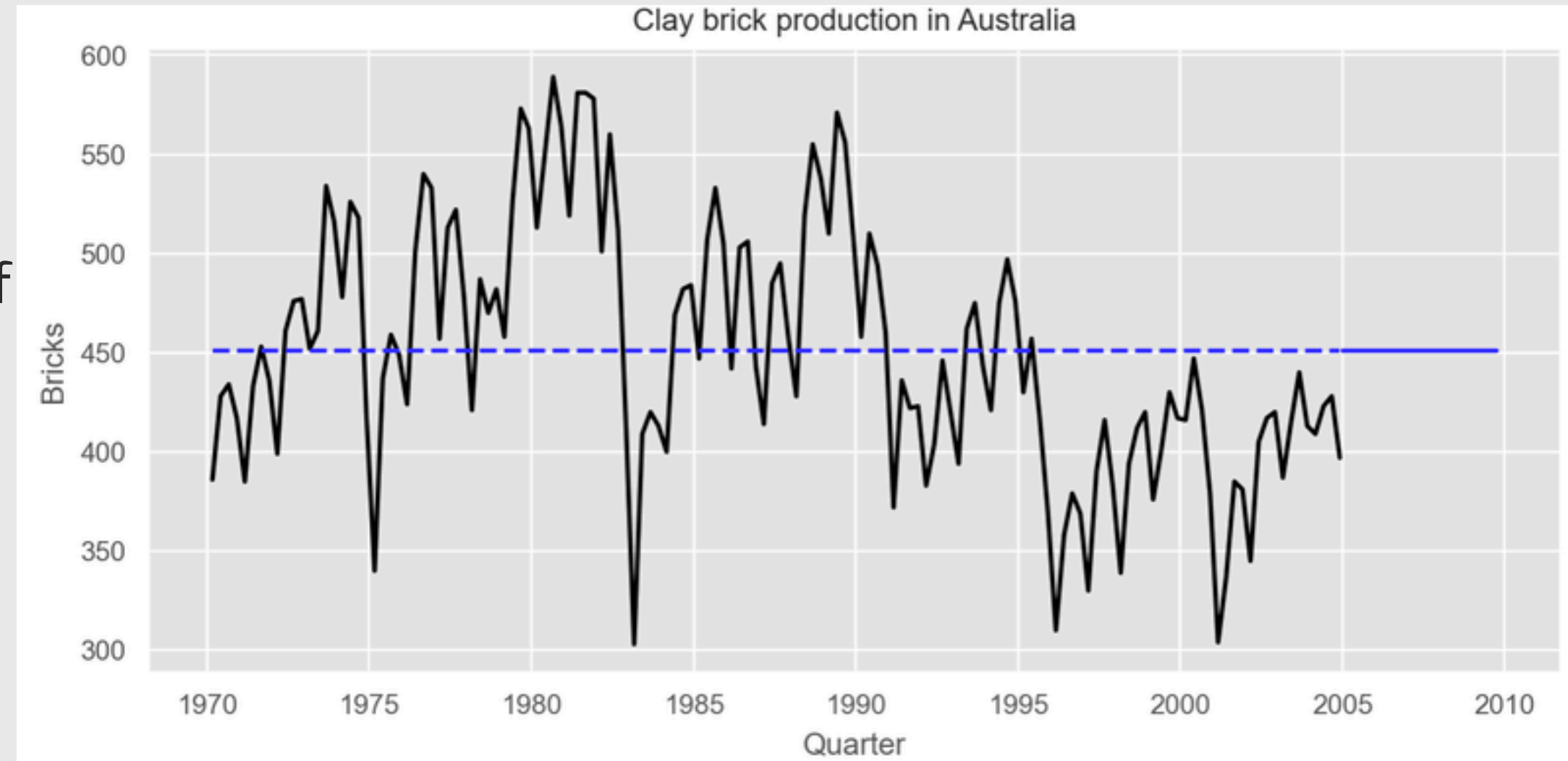


BASIC FORECASTING TECHNIQUES

MEAN METHOD

- Assume the next datapoint is the mean of all the previous values.
- Essentially, we are taking a moving average of all the values as the next datapoint.

$$\hat{y}_{T+h|T} = \frac{1}{T} \sum_{t=1}^T y_t,$$

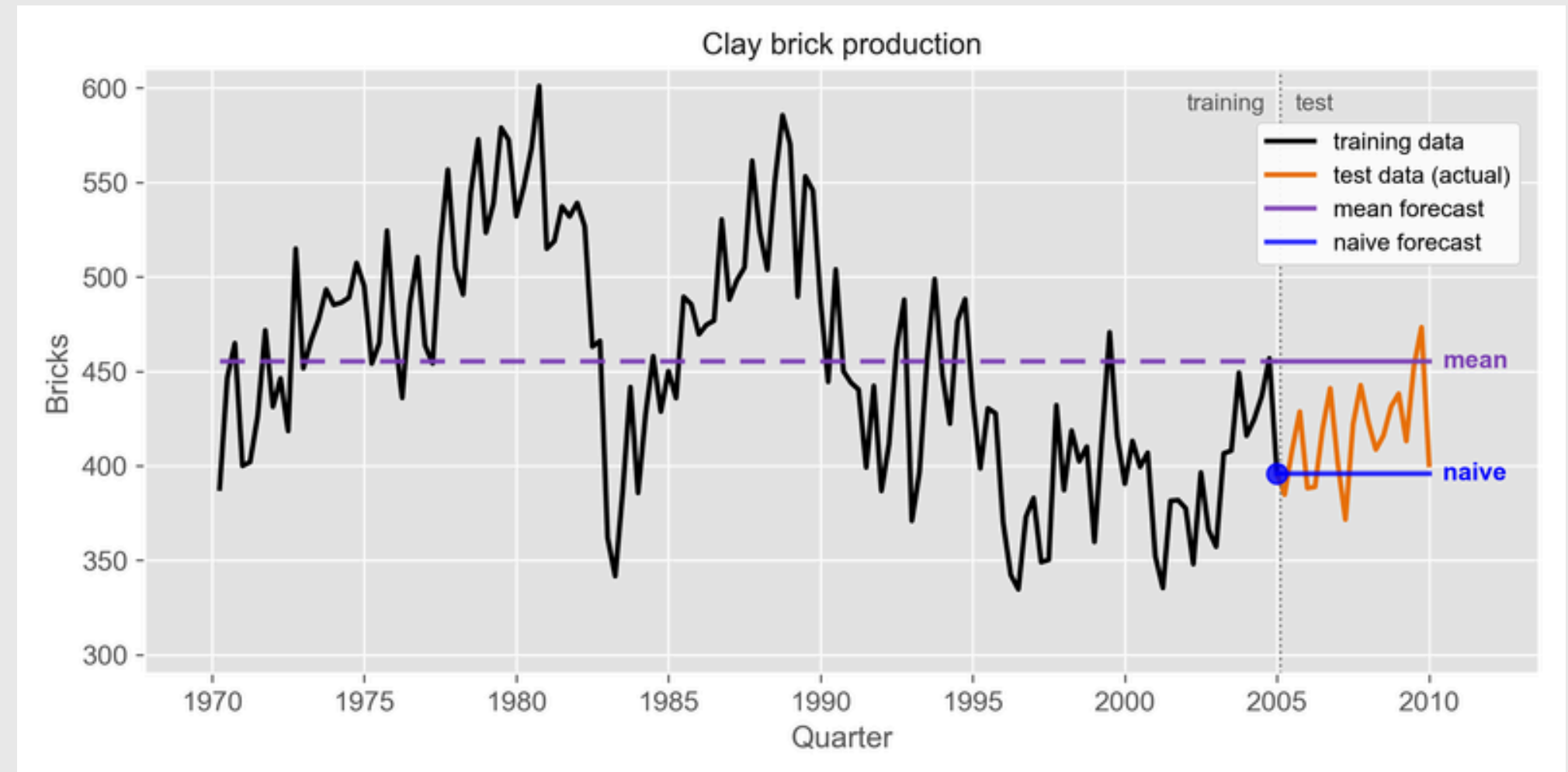


LIMITATIONS OF NAIVE METHOD AS FORECASTING TOOL

- **Disregards historical patterns** — By relying on the latest data point, it misses broader historical context, cycles or patterns.

LIMITATIONS OF MEAN METHOD AS FORECASTING TOOL

- **EQUAL WEIGHTS** — every point in the window counts the same. Is the value 50 days ago really as relevant as yesterday's?
- **Slow to react:** after a shock, the average takes a long time to catch up.



Ideally, we want something that can:

- Assign more importance to the most recent value.
- While considering all the past values.

SIMPLE EXPONENTIAL SMOOTHING

- SES is a weighted average of all past values: every point counts, but older points count less.
- Each step back in time multiplies the weight by $(1 - \alpha)$.
- α (alpha), between 0 and 1, sets how fast the influence fades. With $\alpha = 0.5$ the weights are 0.5, 0.25, 0.125, ...

$$\hat{y}_{T+h|T} = \alpha y_T + \alpha(1 - \alpha) y_{T-1} + \alpha(1 - \alpha)^2 y_{T-2} + \dots$$

When α becomes smaller, the weights become similar $\rightarrow$ Mean Method

$$\hat{y}_{T+h|T} = \frac{1}{T} \sum_{t=1}^T y_t,$$

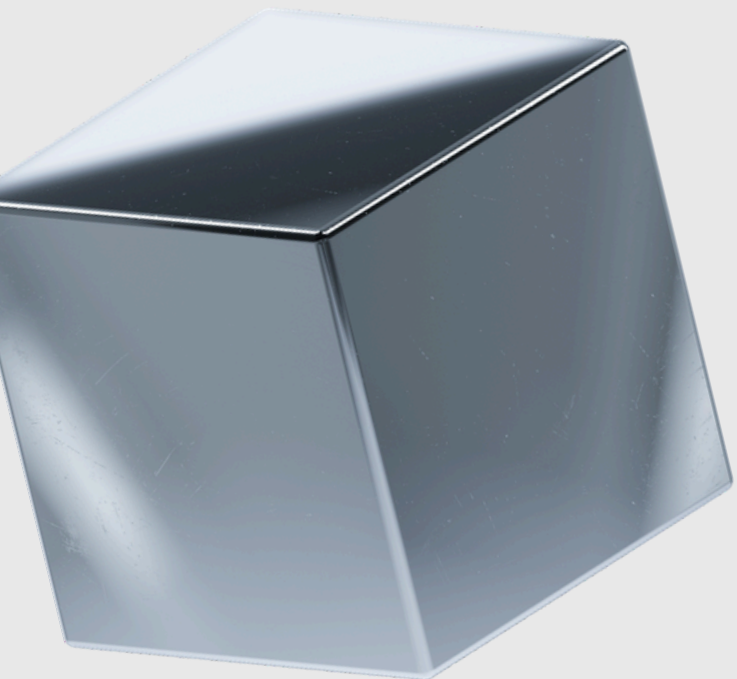
When $\alpha = 1$, the model simplifies into Naive Method

$$\hat{y}_{T+h|T} = y_T,$$



DECISION TREES

...



STUDENT SUCCESS DATASET

Goal: Predict whether a student will pass or not

Target variable

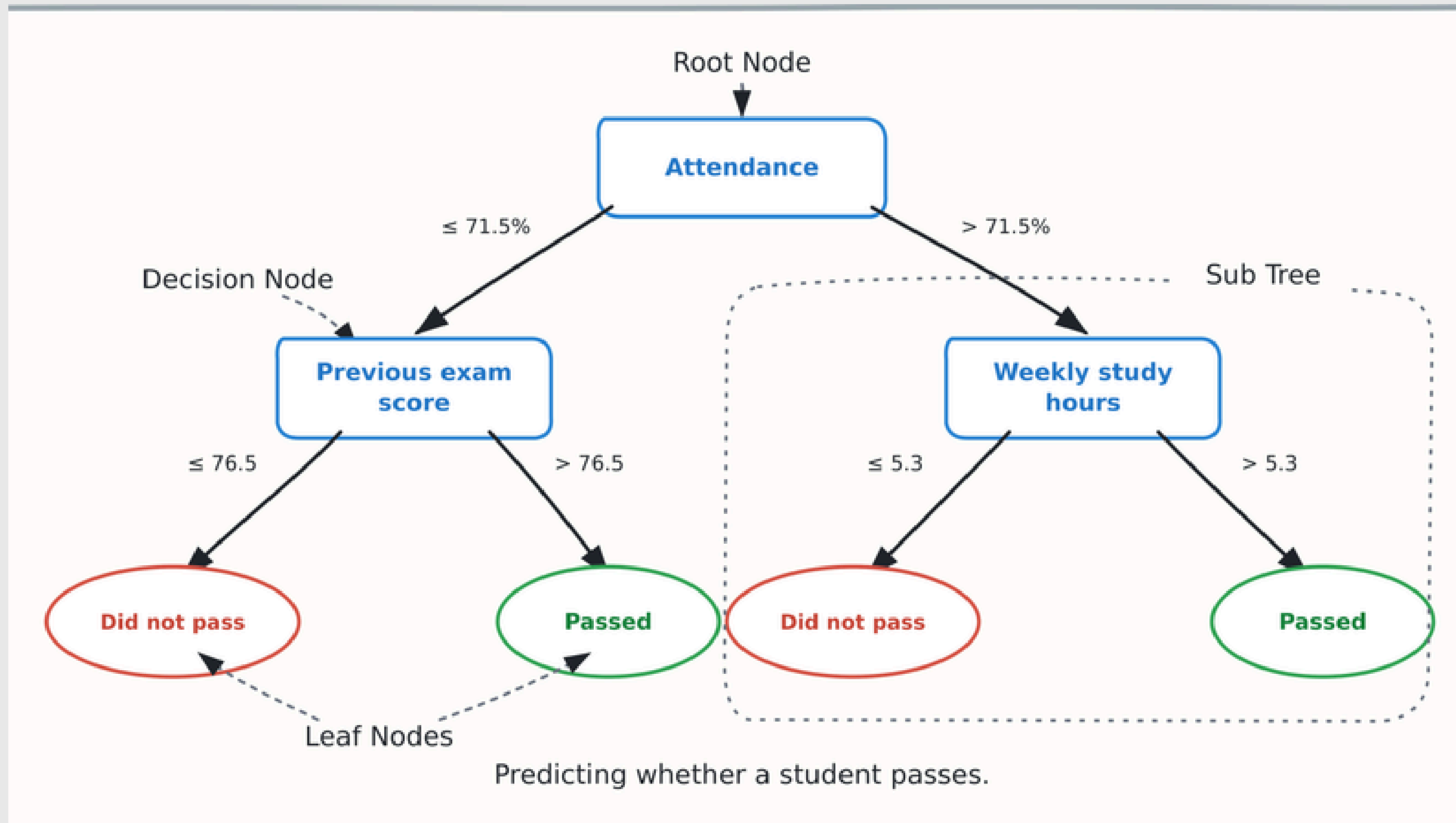
- **passed:** 1 = Passed; 0 = Did not pass

Input variables

- **hours_studied_weekly** – Number of hours spent studying each week
- **attendance_pct** – Percentage of classes attended
- **previous_exam_score** – Score obtained in the previous examination
- **practice_tests_completed** – Number of practice tests completed
- **sleep_hours** – Average number of hours slept each night
- **tutoring_sessions** – Number of tutoring sessions attended
- **assignment_completion_pct** – Percentage of assignments completed
- **class_participation** – Classroom participation rating from 1 to 5

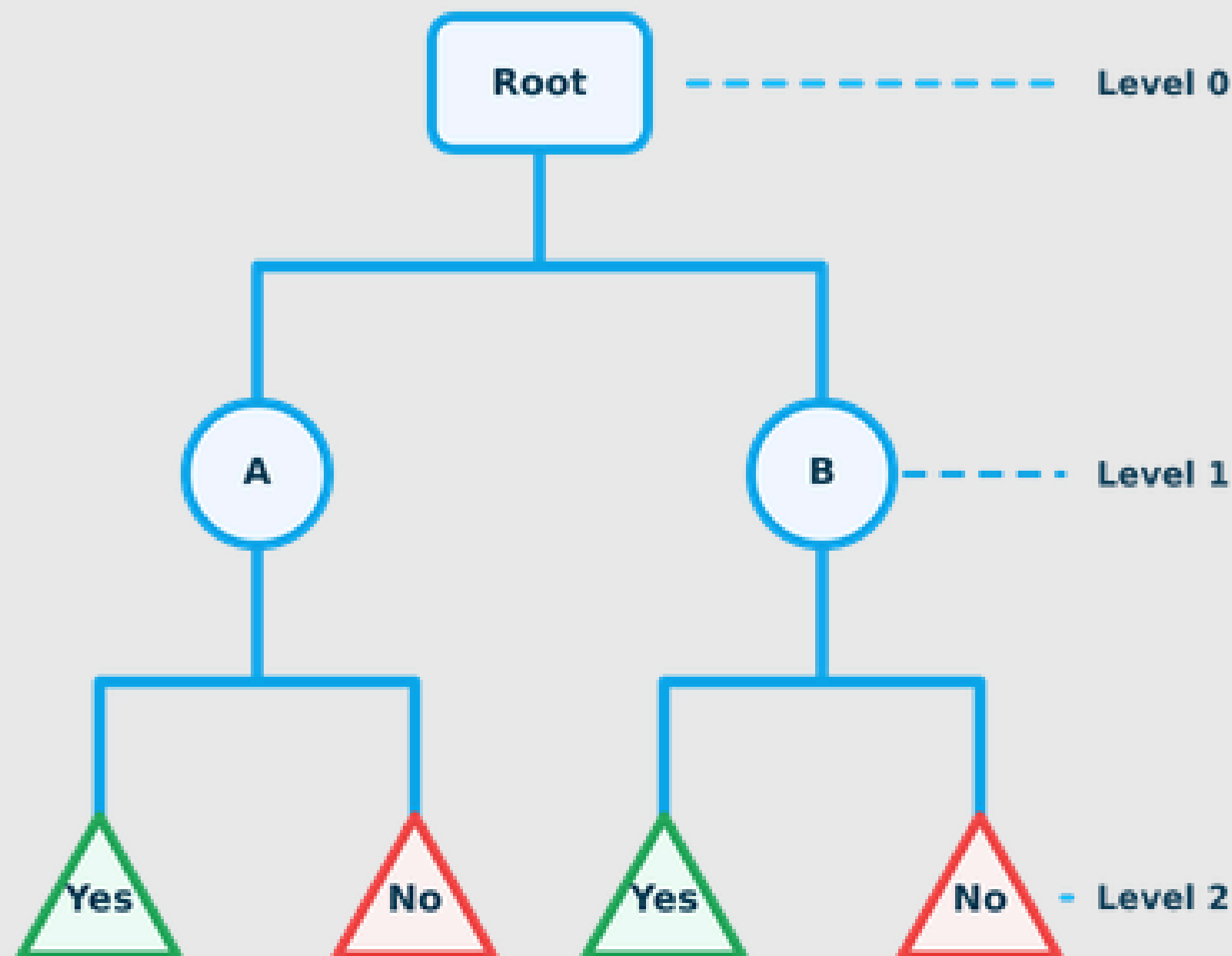
DECISION TREE

Goal: Predict whether a student will pass or not



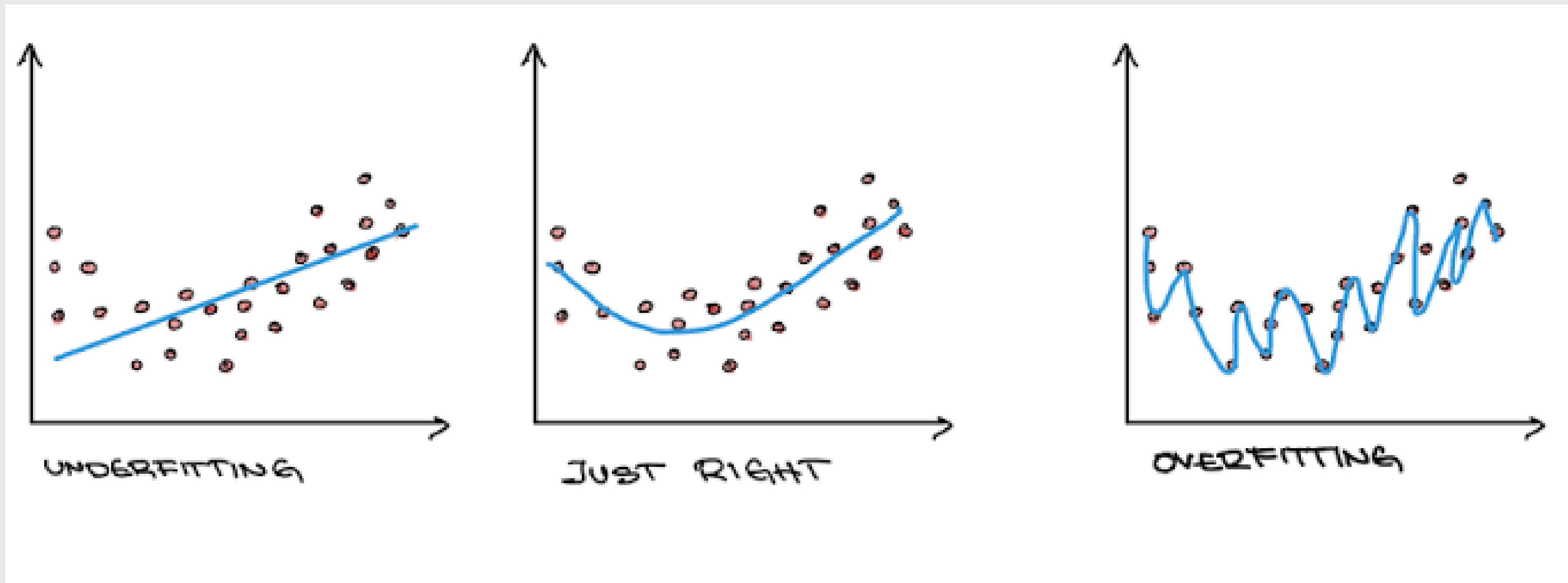
DECISION TREE

Anatomy of a Decision Tree

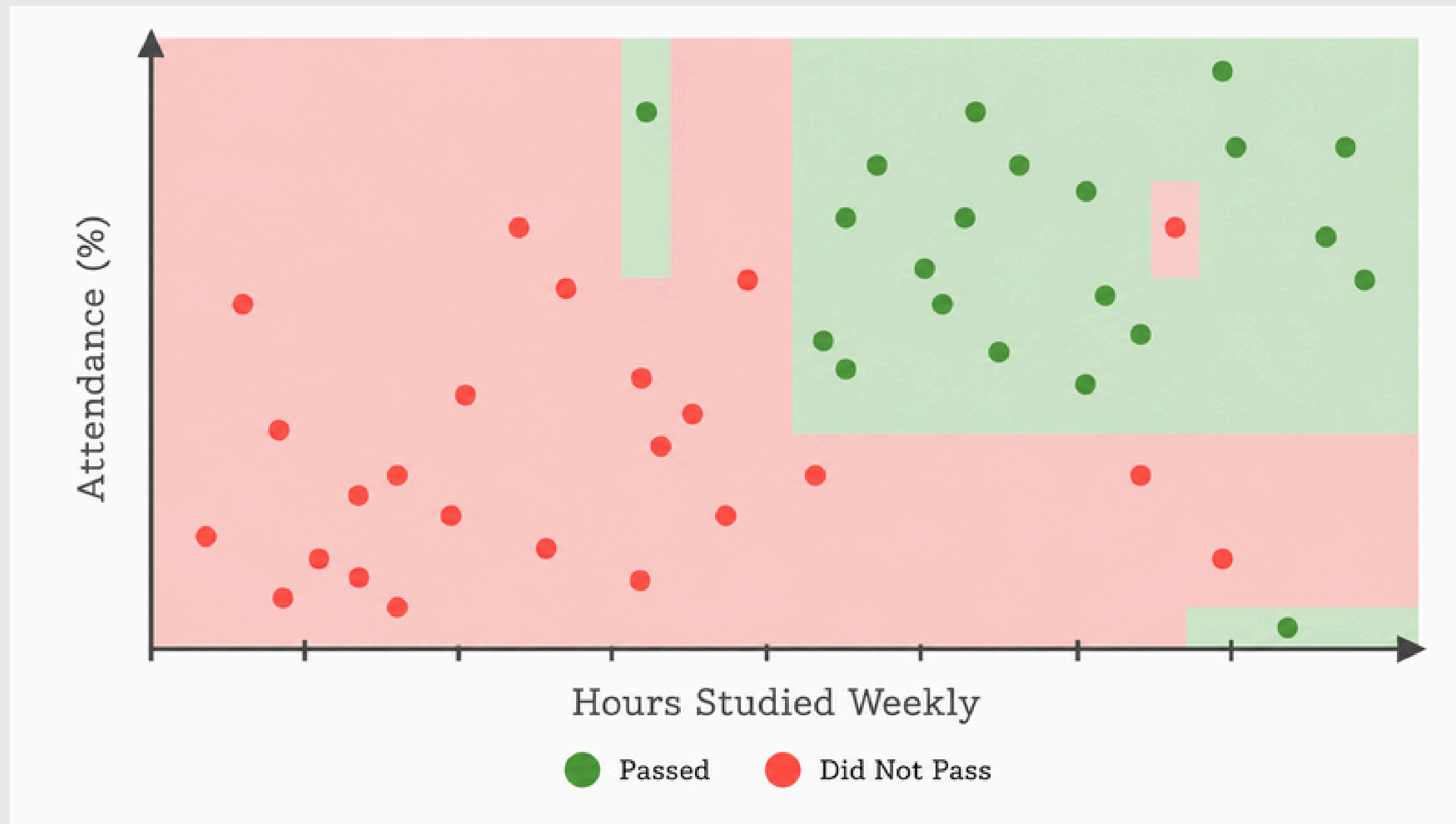


- 1 Root node**
The topmost node and the tree's first decision. It usually splits on the most useful feature.
- 2 Internal nodes**
Further questions that divide the data again. Each split sends observations down a new path.
- 3 Leaf nodes**
The endpoints where splitting stops. Each leaf gives the final prediction, such as Yes or No.
- 4 Branches**
The lines connecting the nodes. Each branch represents the result of a condition.

WHAT IS OVERFITTING?



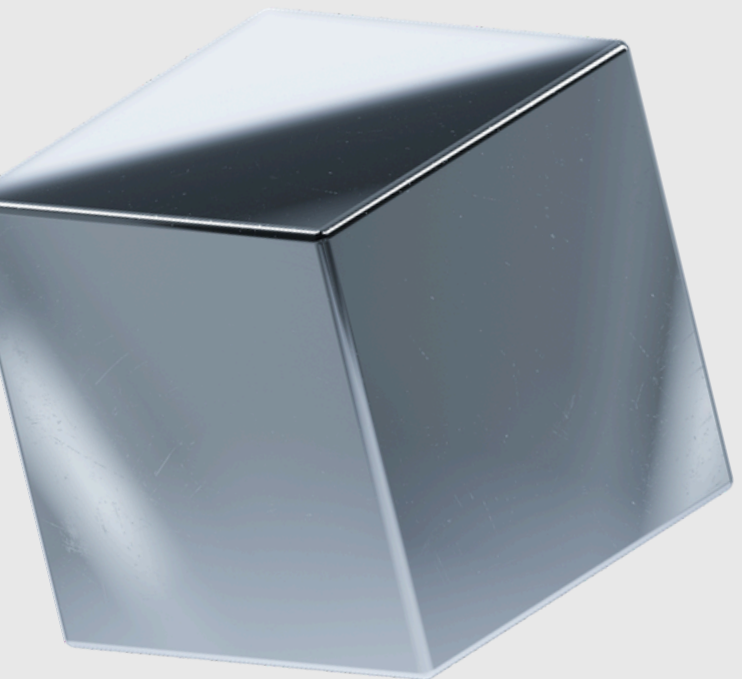
OVERFITTING: DECISION TREE





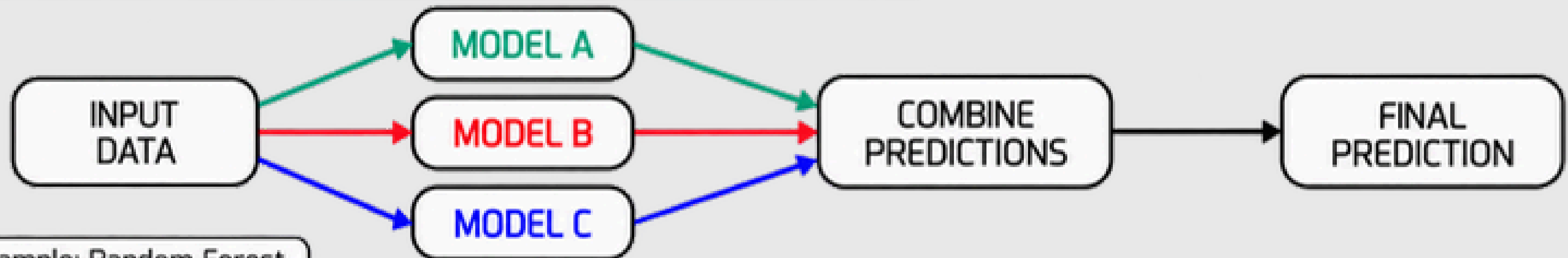
ENSEMBLE

...

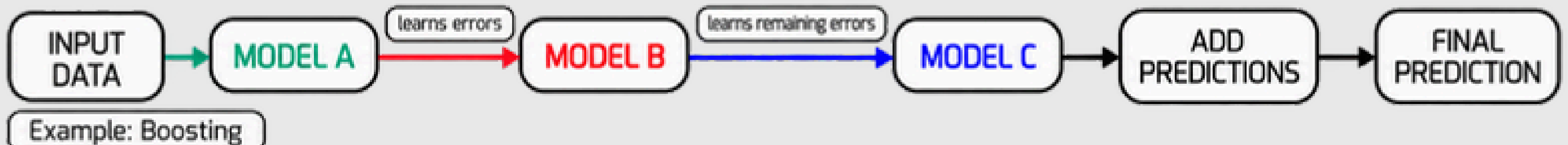


WHAT IS ENSEMBLE?

PARALLEL — MODELS LEARN INDEPENDENTLY



SEQUENTIAL — EACH MODEL CORRECTS EARLIER MISTAKES



WHY ENSEMBLE?

A single model depends on one set of assumptions

MODEL A

0.82

MODEL B

0.61

MODEL C

0.74

Combining helps when model makes different errors

Individual skill

- Each model must contain some predictive signal

Diverse errors

- Models that fail in different places can compensate for one another

THREE ENSEMBLE STRATEGIES

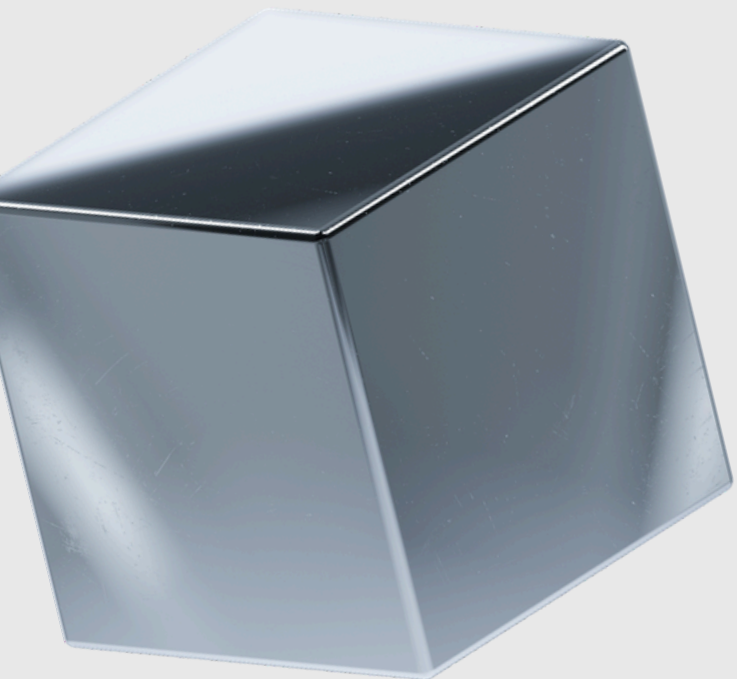
The training relationship determines the ensemble type

STRATEGY	HOW MODELS TRAIN	HOW PREDICTIONS COMBINE	EXAMPLE
Bagging	Independently on different samples	Average or majority vote	Random Forest
Boosting	Sequentially to improve earlier predictions	Weighted sum	XGBoost
Stacking	Different base models train separately	A meta-learner that combines output	StackingClassifier



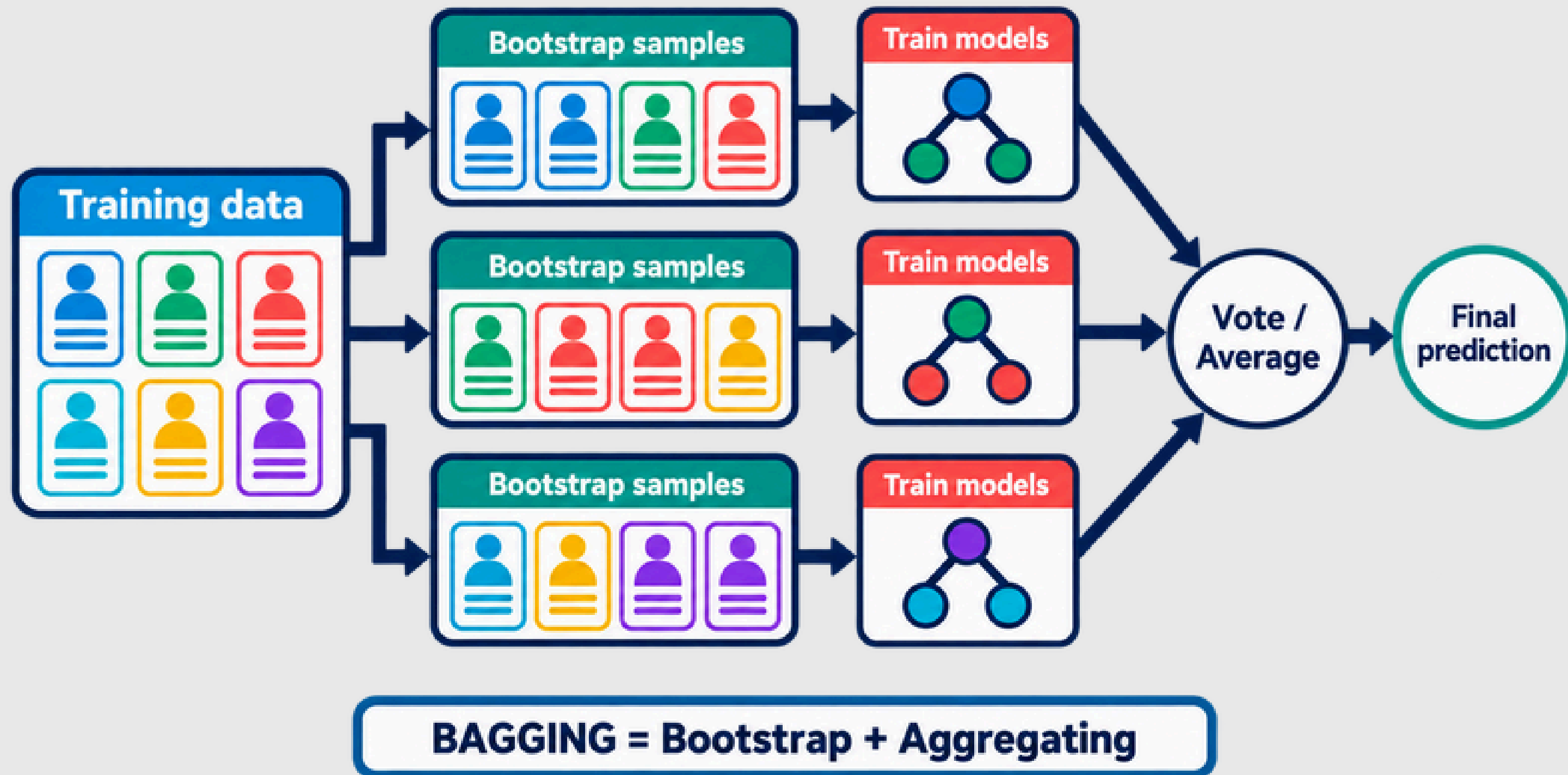
RANDOM FOREST

...

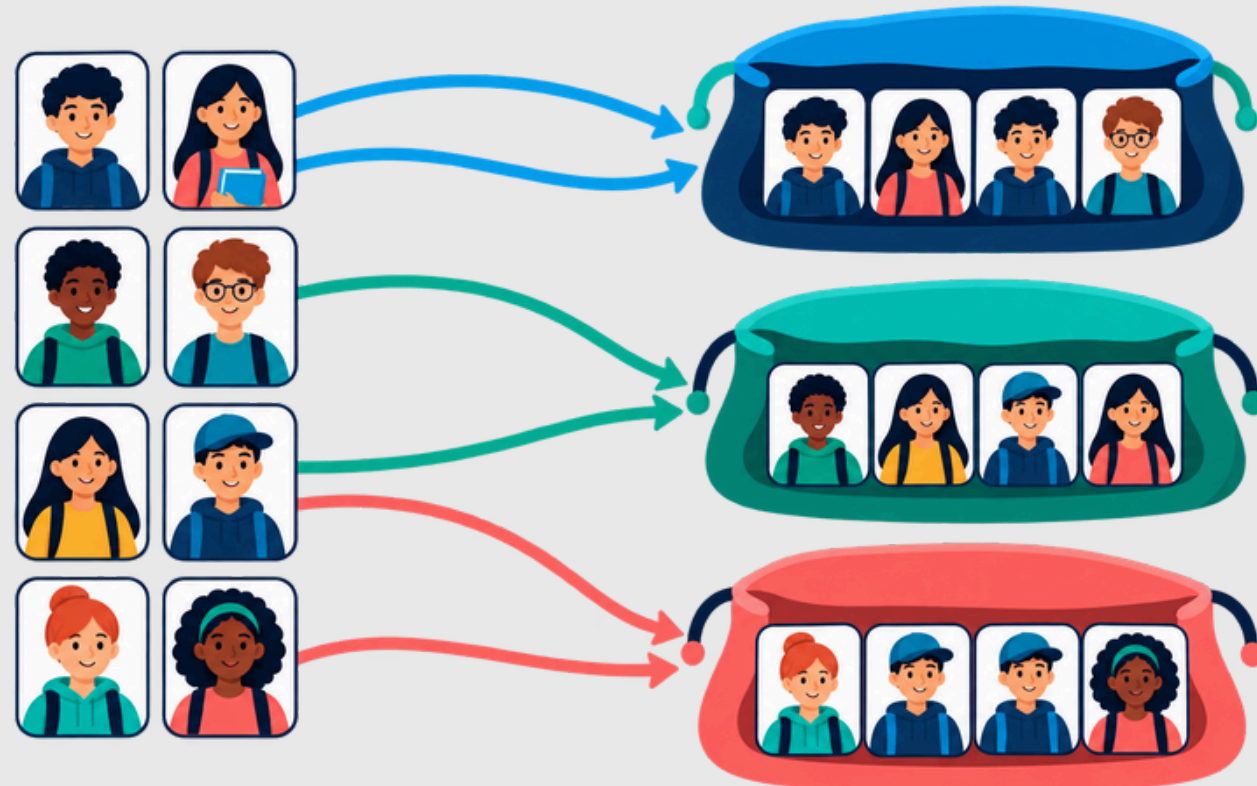


WHAT IS RANDOM FOREST?

Random Forest combines many decision trees

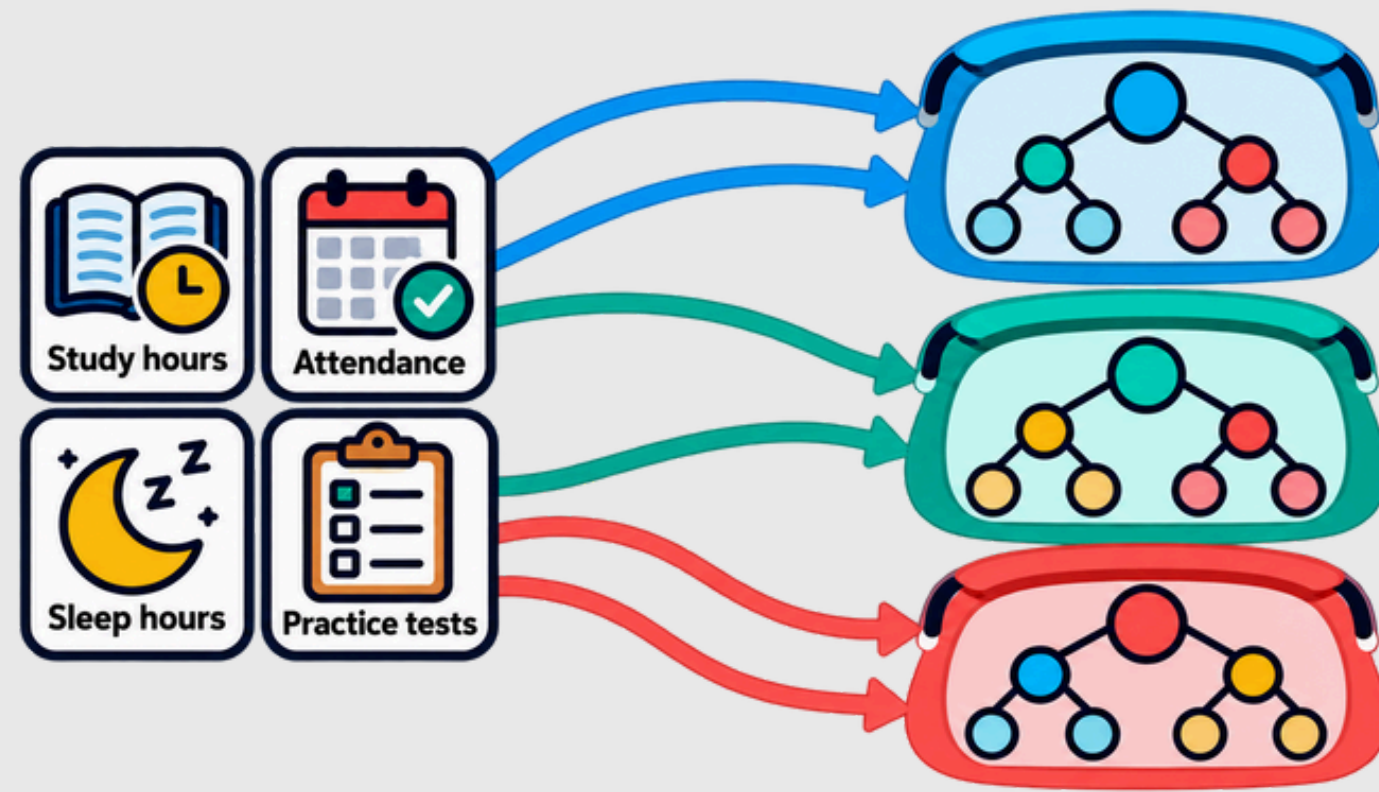


BOOTSTRAPPING



Random rows

Each tree samples students with **replacement**. Some students appear more than once, while others are left out

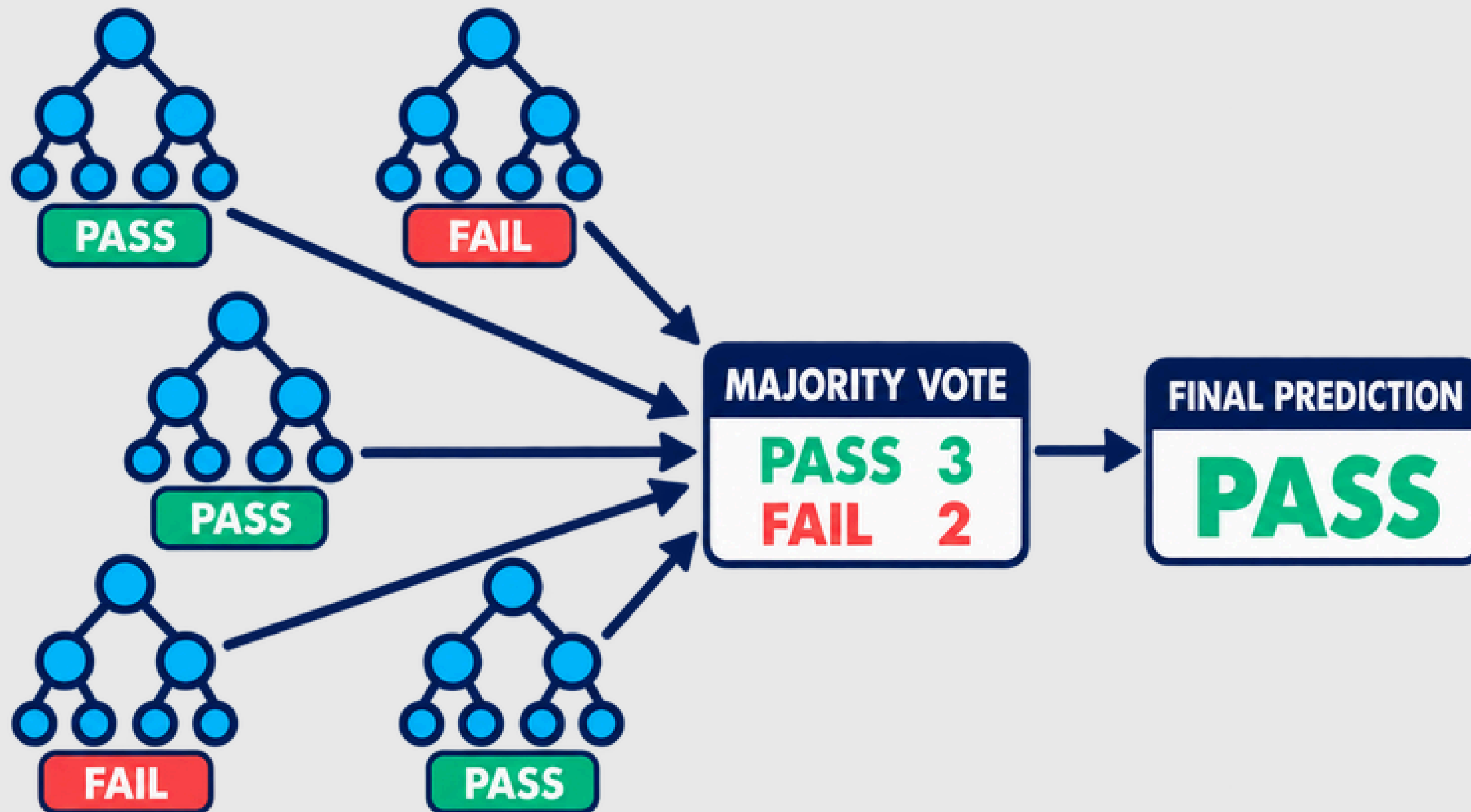


Random features

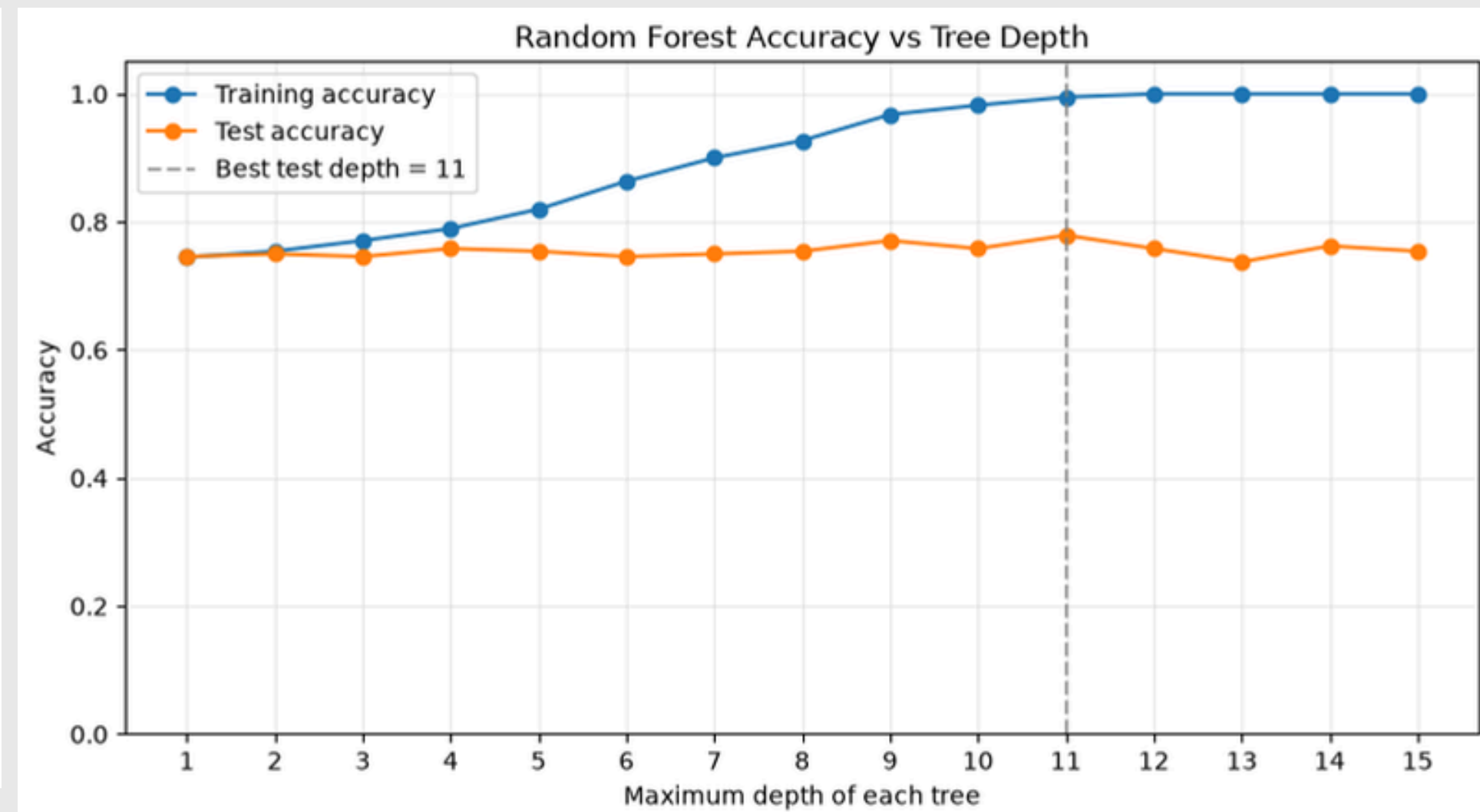
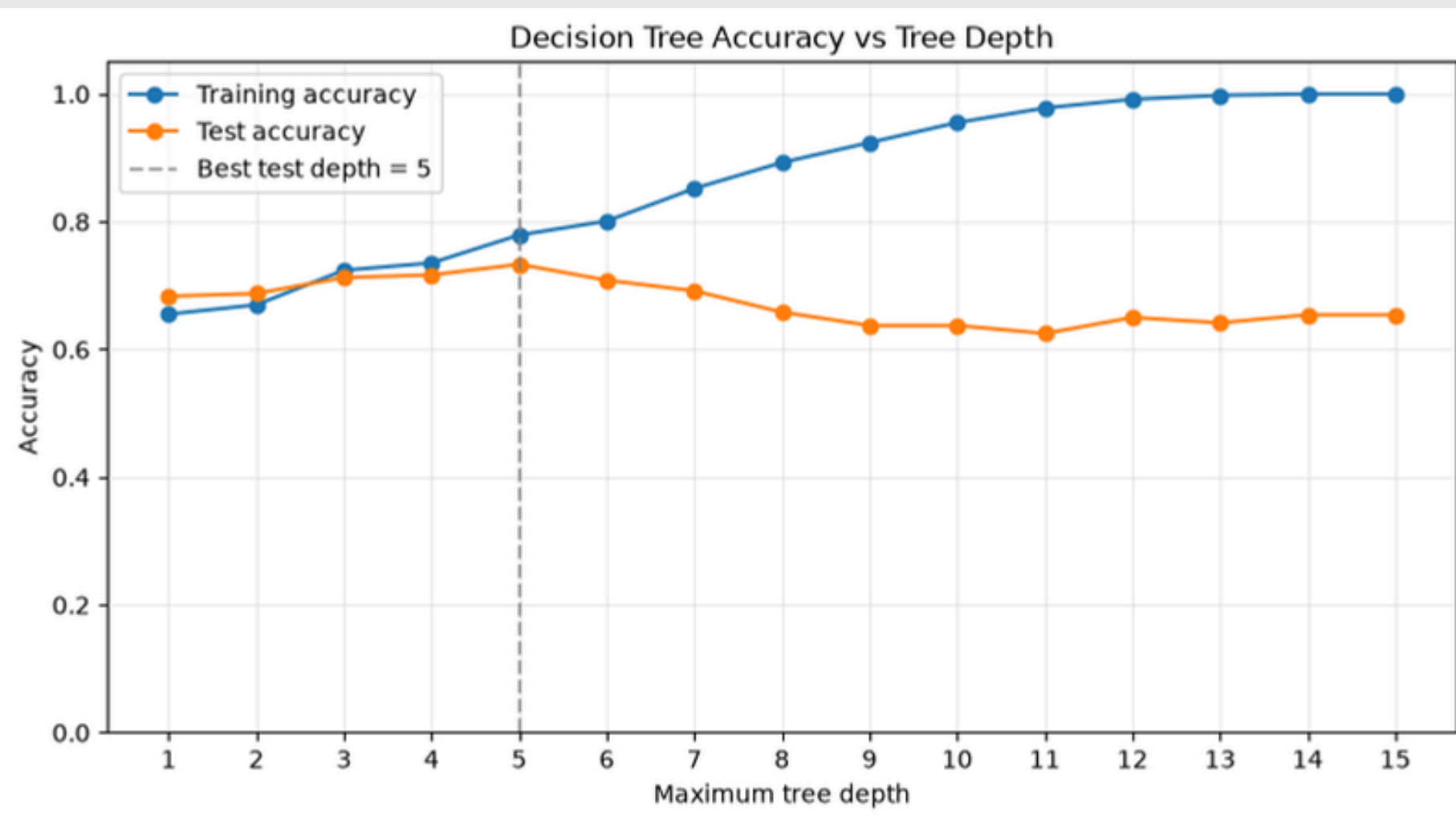
At each split, a tree checks only some features. One tree may use attendance while other may use study hours

AGGREGATING: CLASSIFICATION

How predictions are combined



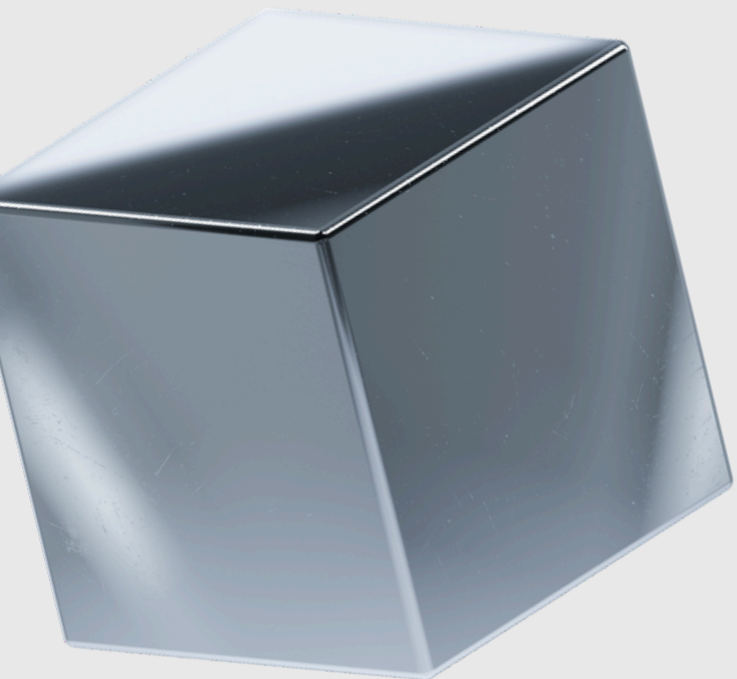
HOW RF REDUCES OVERFITTING





BOOSTING

...



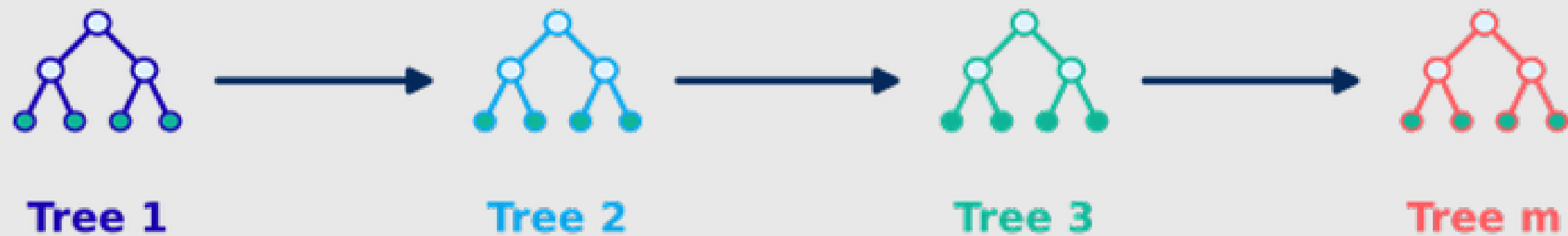
WHY BOOSTING?

Recall: A single tree can be too simple or too complex

Approach	What happens
One shallow tree	May miss useful patterns: underfitting
One very deep tree	May memorise noise: overfitting
Boosting: Many smaller trees	Each new tree corrects the current model so the combined prediction improves

GRADIENT BOOSTING

Each weak tree makes a small correction to the model built so far



Trees are added sequentially — each new tree learns the remaining errors

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

new ensemble = current ensemble + learning rate × new tree

EXAMPLE

Starting prediction = average score = $(20+100)/2 = 60$

Student	Hours	Actual score	Prediction	Error
A	1	20	60	-40
B	5	100	60	+40

Error = Actual - Predicted

The first tree uses hours studied to predict these errors

Hours ≤ 3 : output -40

Hours > 3 : output +40

TWO UPDATES, STEP BY STEP

Follow student B: actual score = 100, learning rate $n = 0.5$



Recall: Error = Actual - Predicted

The second tree learns the NEW remaining errors

Final prediction so far = $60 + 0.5 \times 40 + 0.5 \times 20 = 90$

XGBOOST

We will be using XGBoost, an optimised implementation of gradient boosting!

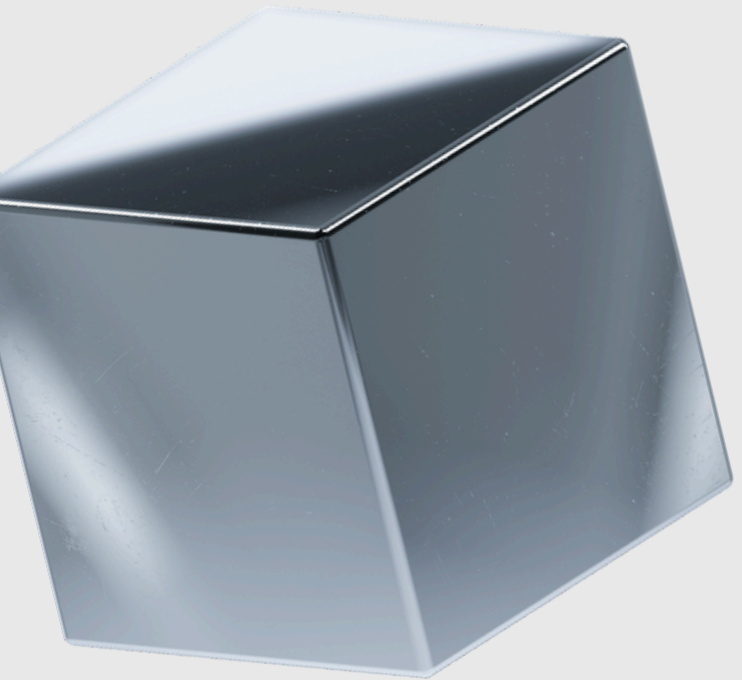
```
xgb_model = XGBClassifier(  
    n_estimators=120,      # Number of boosting trees  
    max_depth=3,          # Maximum depth of each tree  
    learning_rate=0.05,    # Scale each new tree's contribution  
    subsample=0.90,        # Use 90% of training rows per boosting round  
    colsample_bytree=0.90,  # Use 90% of features for each tree  
    objective="binary:logistic", # Binary classification with probability output  
    eval_metric="logloss",  # Log loss metric for evaluation  
    random_state=RANDOM_STATE, # Make random sampling reproducible  
    n_jobs=-1,             # Use all available CPU cores  
)
```

$$L = -\left[y \log(p) + (1 - y) \log(1 - p)\right]$$



STACKING

...



WHAT IS STACKING?

A model learns how to combine other models' predictions

Example: will a student pass an exam?

Original inputs Hours studied, attendance, practice score, etc

Base models XGBoost, Random Forest and KNN

Meta-learner Logistic regression uses their three probabilities

Target: pass=1, fail=0

WHY STACK MODELS?

Stacking can help when models make different mistakes

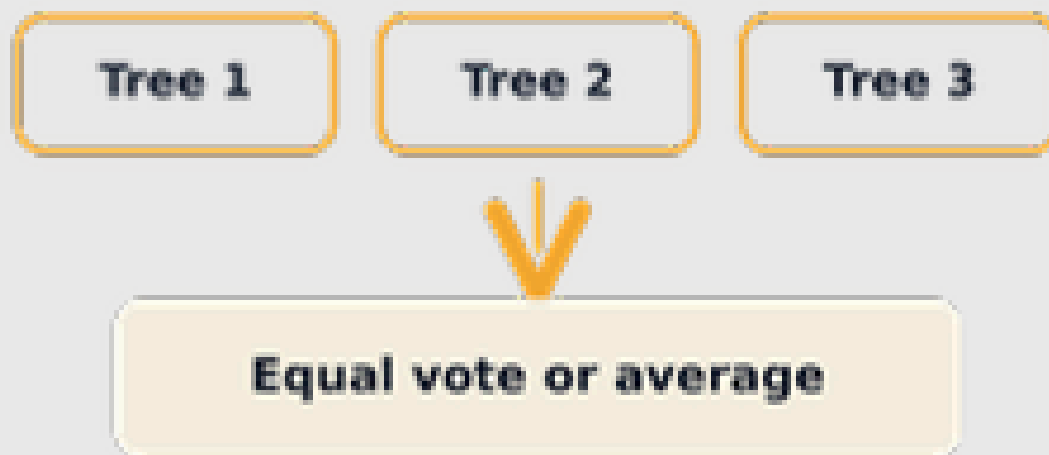
Base model	How it makes prediction
XGBoost	Adds trees that correct earlier errors
Random Forest	Averages predictions from varied trees
KNN	Looks at students with similar features

The stack must still beat the strongest base model on unseen data

WHY STACK MODELS?

BAGGING

Same learner, fitted many times



Classification: majority vote

Regression: mean prediction

Purpose: reduce the variance of one learner

Example: Random Forest

MANUAL WEIGHTED AVERAGE

Different model types



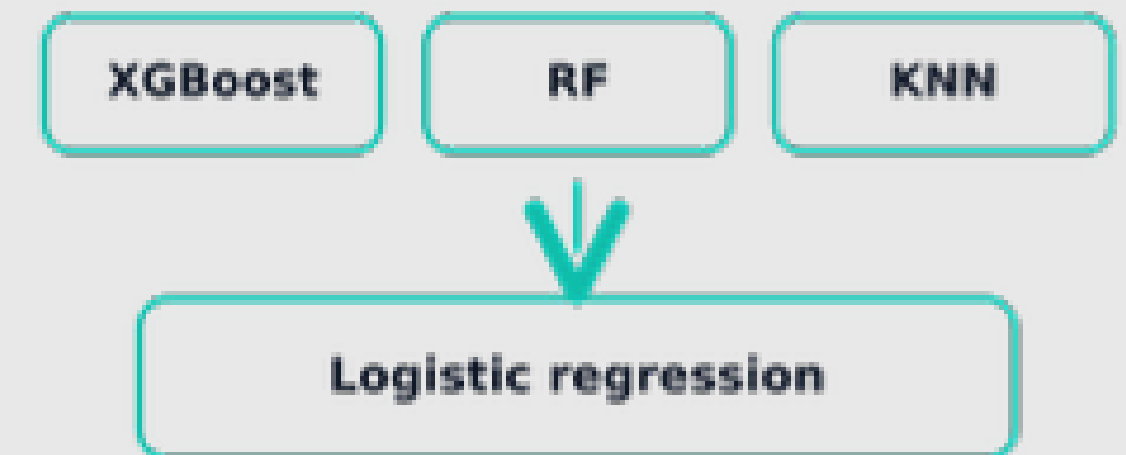
$$p = 0.4p_{\text{XGB}} + 0.4p_{\text{RF}} + 0.2p_{\text{KNN}}$$

Simple and transparent

The chosen weights may ignore overlap
between model predictions

STACKING

Different model types



$$p = \sigma(b + w_1p_{\text{XGB}} + w_2p_{\text{RF}} + w_3p_{\text{KNN}})$$

Learns b and the weights together from
out-of-fold predictions

Accounts for overlapping model signals

HOW STACKING WORKS

Logistic regression learns from the base models' out-of-fold (OOF) probabilities

$$z = b + w_1 P_{XGB} + w_2 P_{RF} + w_3 P_{KNN}$$

$$P(pass) = \frac{1}{1 + \exp(-z)}$$

Training data Collect one pass probability from each base model

Learning Logistic regression changes b and the weights to reduce log loss

Each weight reflects a model's value after considering the other models

OUT-OF-FOLD PREDICTIONS

Give the meta-learner predictions from rows the base model did not train on

FIVE-FOLD OOF

Round	Train on 4 folds		Predict held-out fold
1	B C D E	→	A
2	A C D E	→	B
3	A B D E	→	C
4	A B C E	→	D
5	A B C D	→	E

Each row is predicted by a model that did not train on that row.

Repeat for
XGBoost,
Random Forest &
KNN

↓

META-LEARNER INPUTS

Row	XGB	RF	KNN	Actual
1	0.80	0.70	0.60	Pass
2	0.30	0.20	0.40	Fail
⋮	⋮	⋮	⋮	⋮
N	0.68	0.55	0.72	Pass

3 OOF probabilities = features

Actual pass/fail = target

HOW ERRORS INFLUENCE WEIGHTS

Logistic regression learns all coefficients together

OOF prediction pattern	Possible coefficient response
Useful and adds new information	May receive a larger positive coefficient
Mostly repeats another model	May receive a smaller coefficient
Often confidently wrong	May receive a small or negative coefficient

The coefficients are learned from all rows, not assigned from accuracy alone.

WORKED META-LEARNER

Illustrative coefficients chosen for simple arithmetic

Base model	Pass probability	Learned weight	Contribution
XGBoost	0.80	2.0	1.60
Random Forest	0.70	1.0	0.70
KNN	0.60	0.5	0.30

With intercept $b = -2$

$$z = -2 + 1.60 + 0.70 + 0.30 = 0.60$$

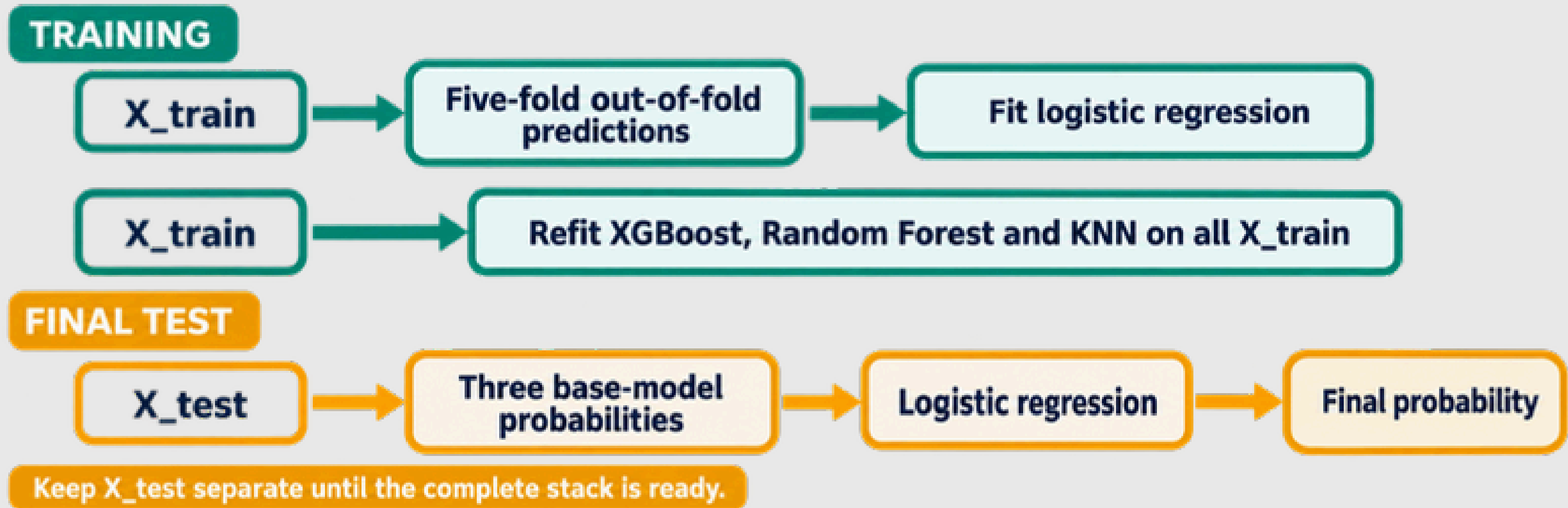
$P(\text{pass}) = \text{approx. } 0.646$, at a 0.5 threshold: pass

$$z = b + w_1 P_{XGB} + w_2 P_{RF} + w_3 P_{KNN}$$

$$P(\text{pass}) = \frac{1}{1 + \exp(-z)}$$

TRAINING AND PREDICTING

OOF predictions train the meta-learner. Full models predict new rows.



UNSUPERVISED LEARNING

FINDING PATTERNS /
STRUCTURE IN DATA

DIFFERENT CATEGORIES

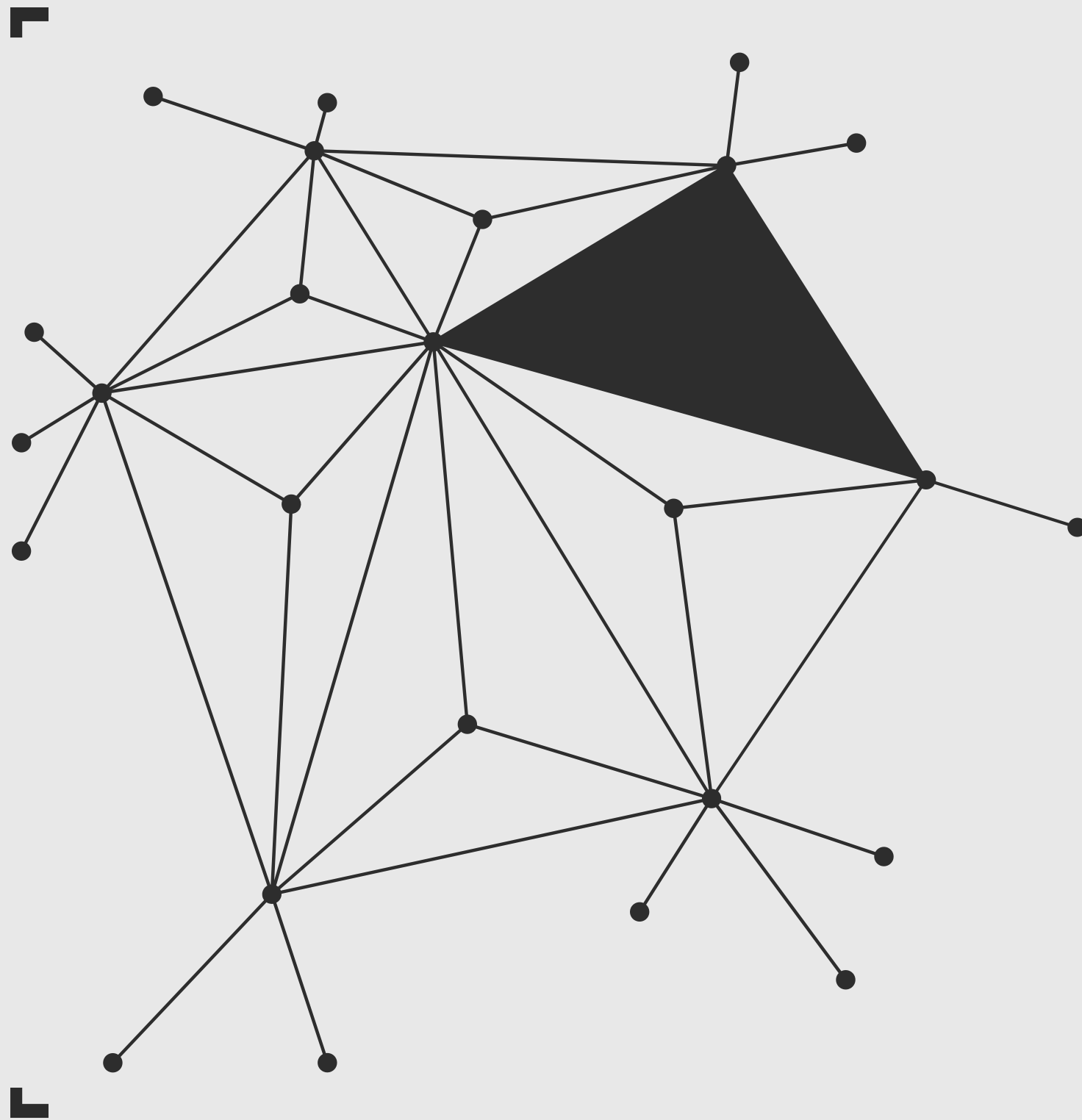
Clustering

Dimensionality Reduction

Anomaly Detection

Association Rule Mining

...



KMEANS

Objective: To partition the data set into k distinct groups based on feature similarity.

Idea: Data points similar to each other would be close to each other in the Euclidean n-space.

THE OPTIMISATION PROBLEM

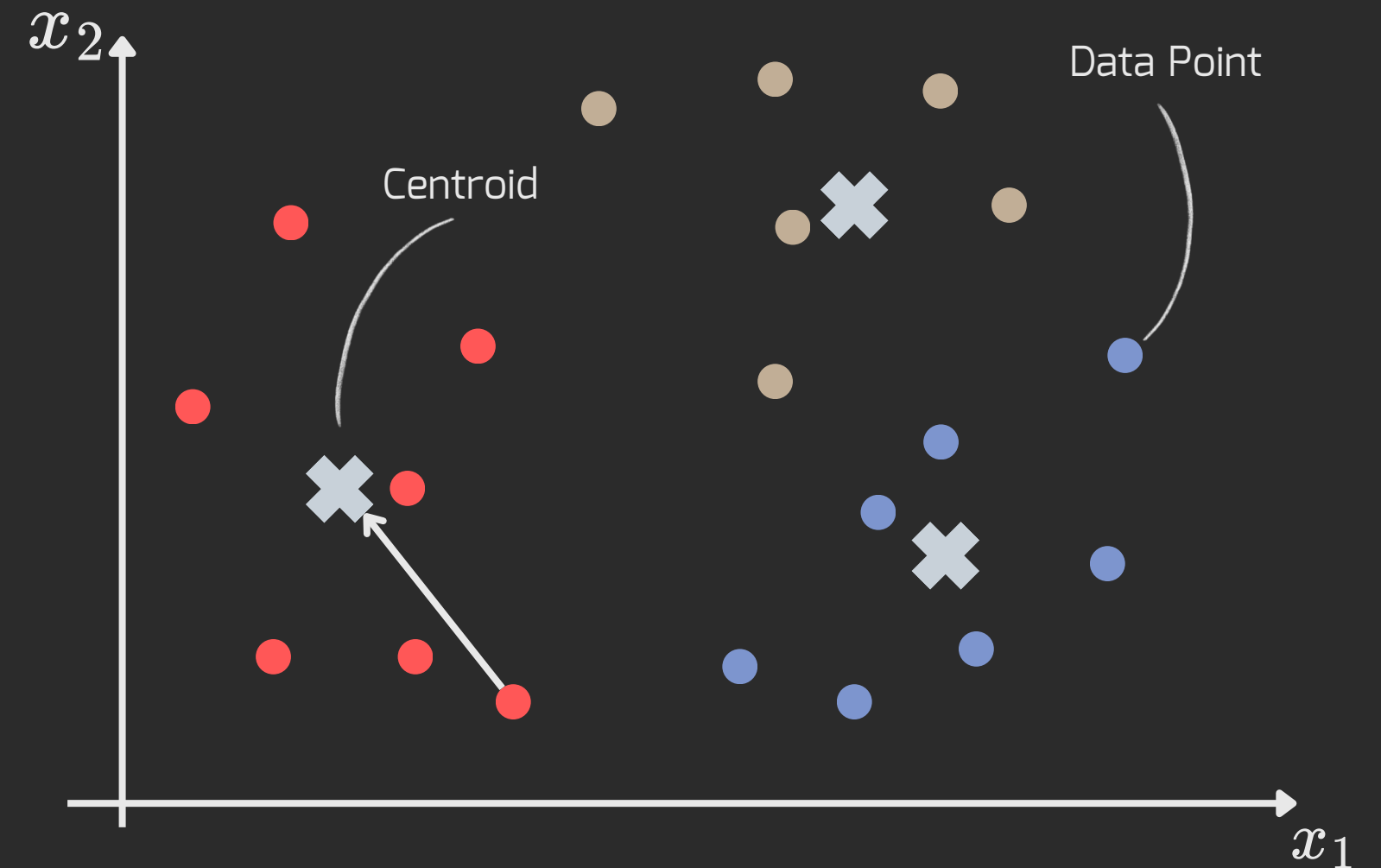
$$C^* = \arg \min_C \sum_{i=1}^n \|\mathbf{x}_i - \mu_{C_i}\|^2$$

C_i is the cluster that the point belongs to

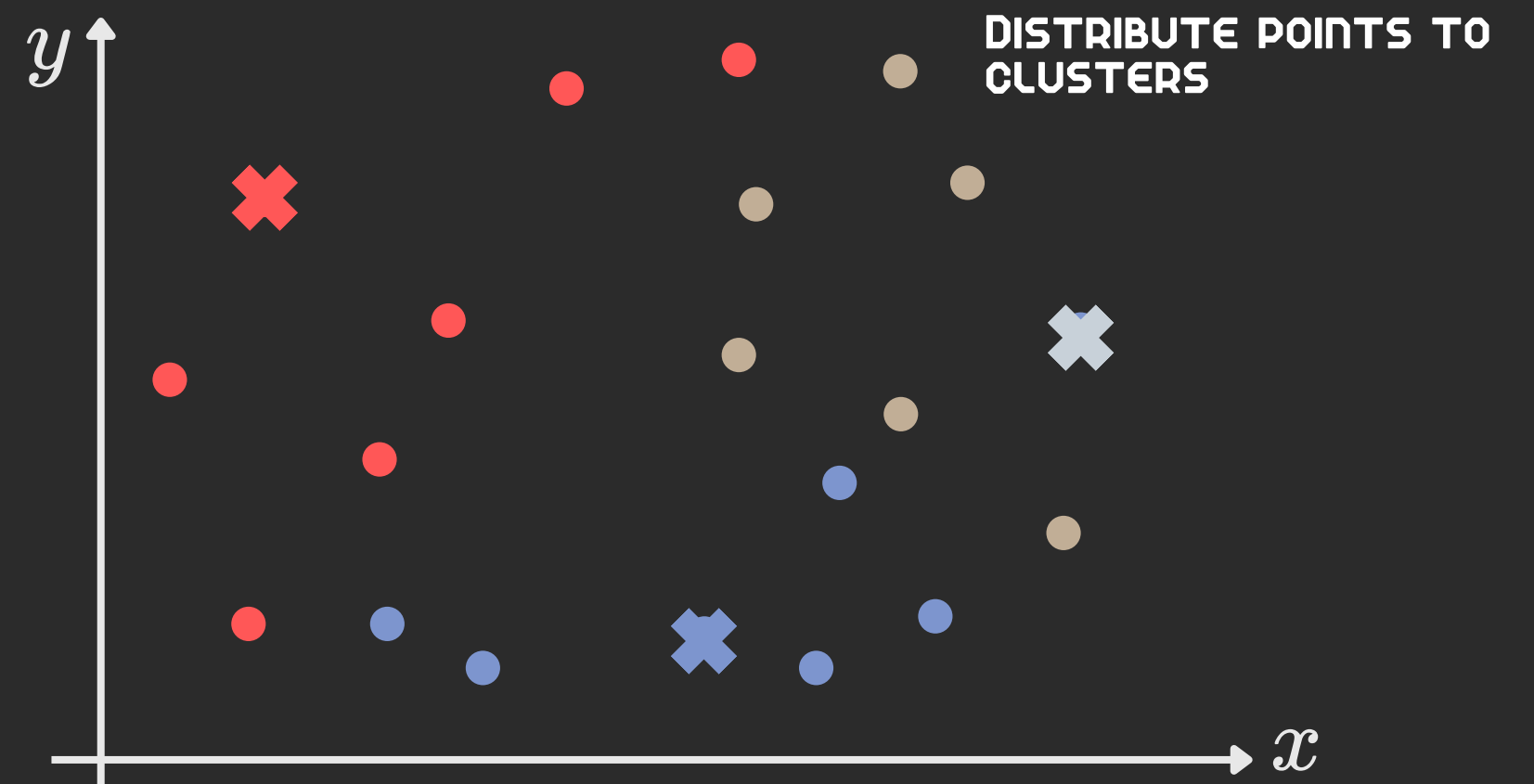
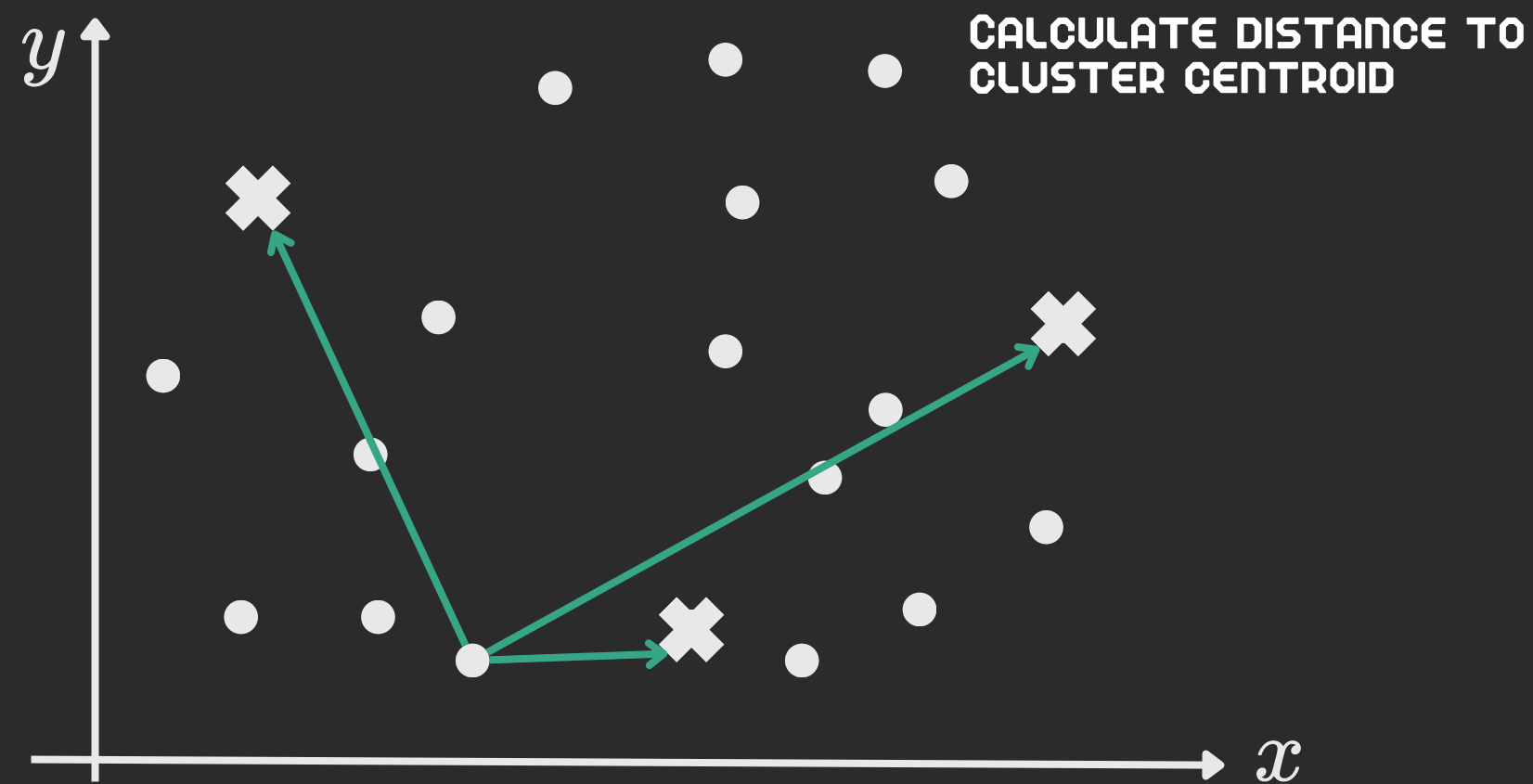
Minimising the Within Cluster Sum of Squares (WCSS)

KMeans presents an optimisation problem where objective is to minimise the average squared Euclidean distance (squared vector norm) between data points and their cluster centroids.

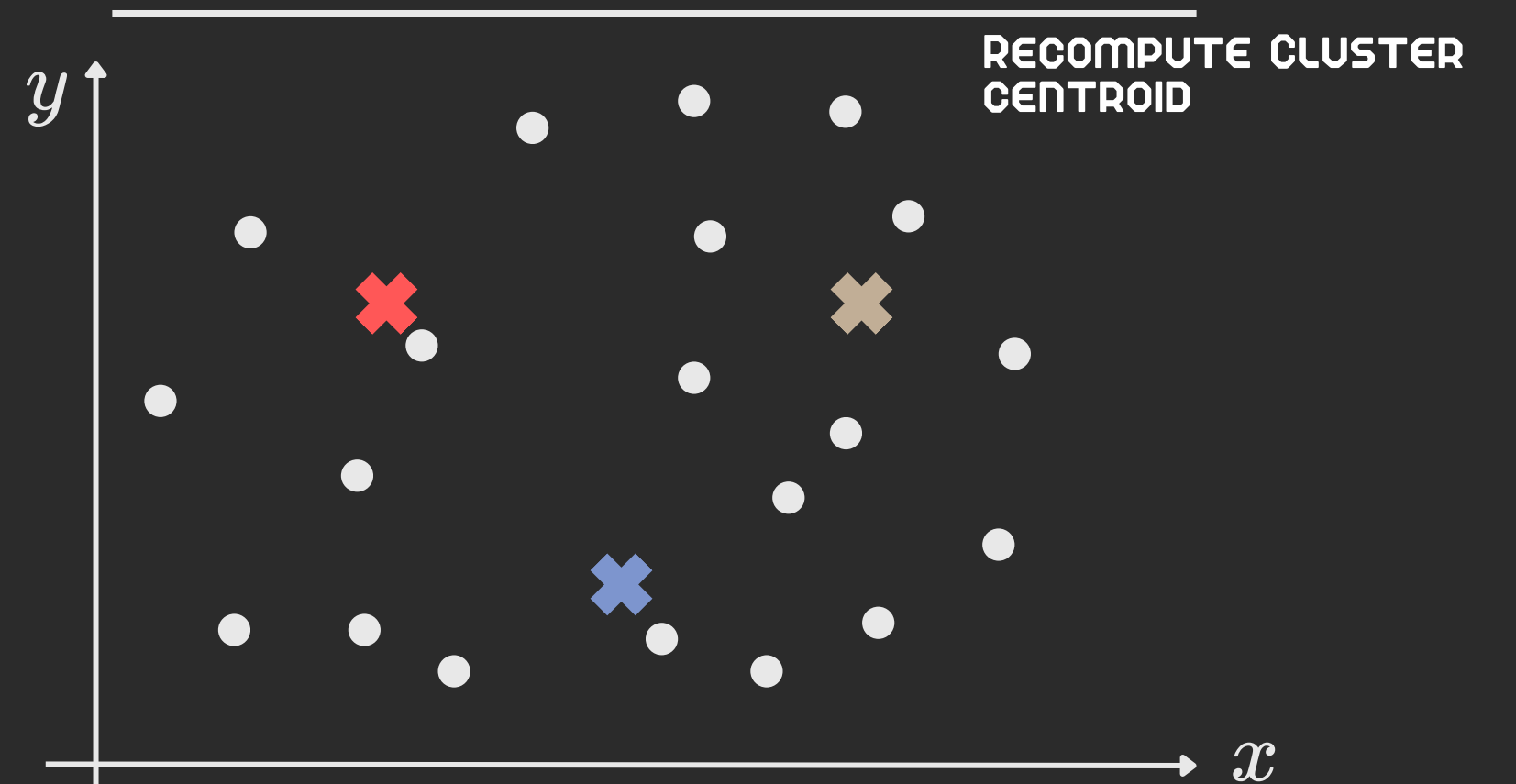
VISUALISATION



$$d = \sqrt{(x_{21} - x_{11})^2 + (x_{22} - x_{12})^2}$$



VISUALISING KMEANS



KMEANS

General Algorithm

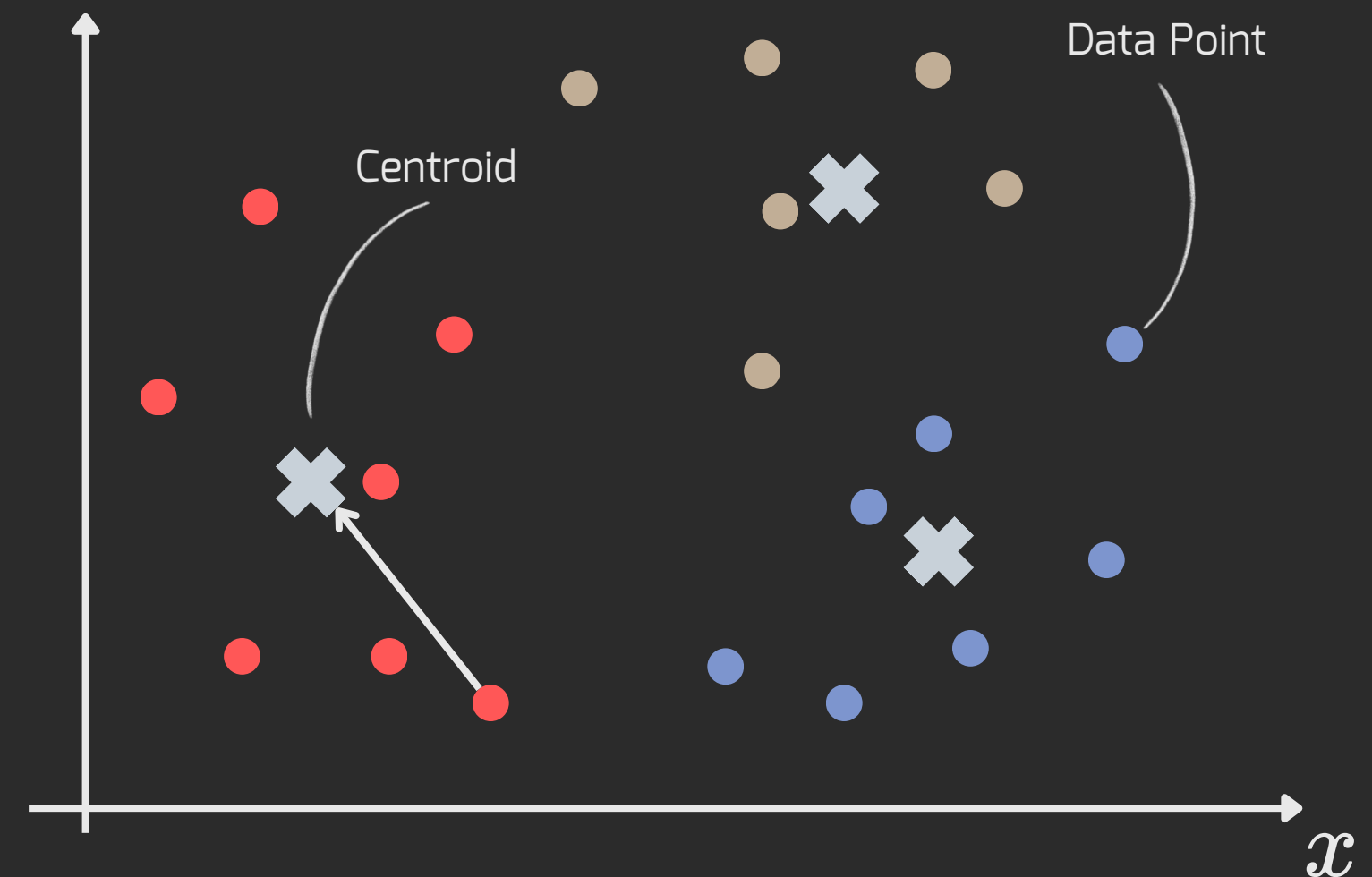
1. Pick K points at random as initial centroids
2. Until the change in WCSS is minimal, do:
 - a. Calculate the distance between each data point and each centroid
 - b. Assign data point to nearest centroid
 - c. Recalculate WCSS

Parameters

To be demonstrated in Python Code

1. K is the number of clusters you desire
2. Distance metric (usually the L2-norm)
3. Convergence Criteria (shift in centroid falls below a user specified tolerance level)

VISUALISATION



KMEANS ++

KMeans Algorithm (Original)

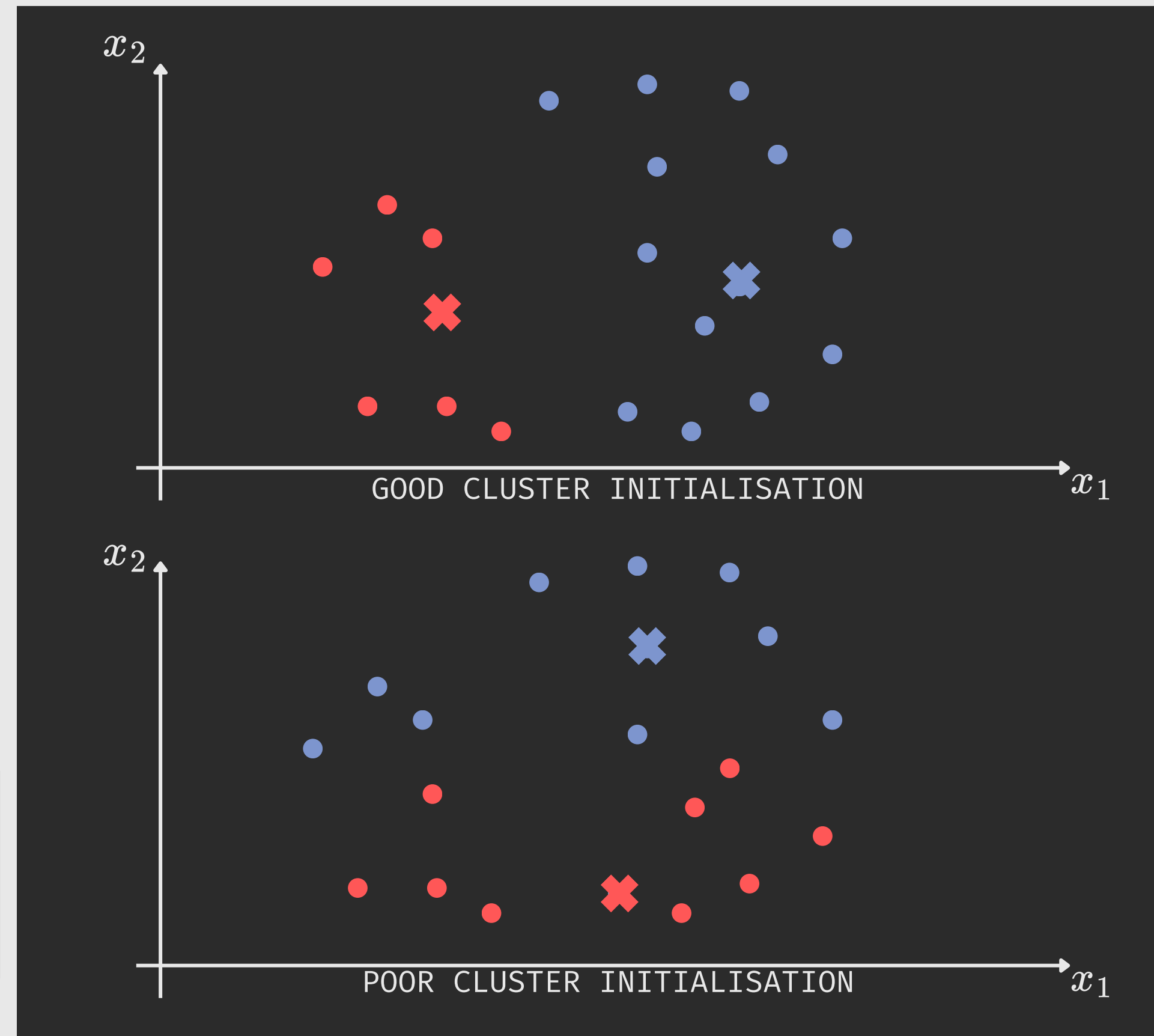
1. Pick K points **at random** as initial centroids
2. Until the change in C is minimal, do:
 - a. Calculate the distance between each data point and each centroid
 - b. Assign data point to nearest centroid
 - c. Recalculate C

KMeans ++ Algorithm

1. Pick K points that are **as far apart** as possible
2. Same as KMeans

Rationale

- Random starting points often bunch together
- This traps the algorithm in a poor optimal solution
- Spacing them far apart ensures every data group is covered from the start
- Helps algorithm reach a global minimum



CHOOSING K

$$C^* = \arg \min_C \sum_{i=1}^n \|\mathbf{x}_i - \mu_{C_i}\|^2$$

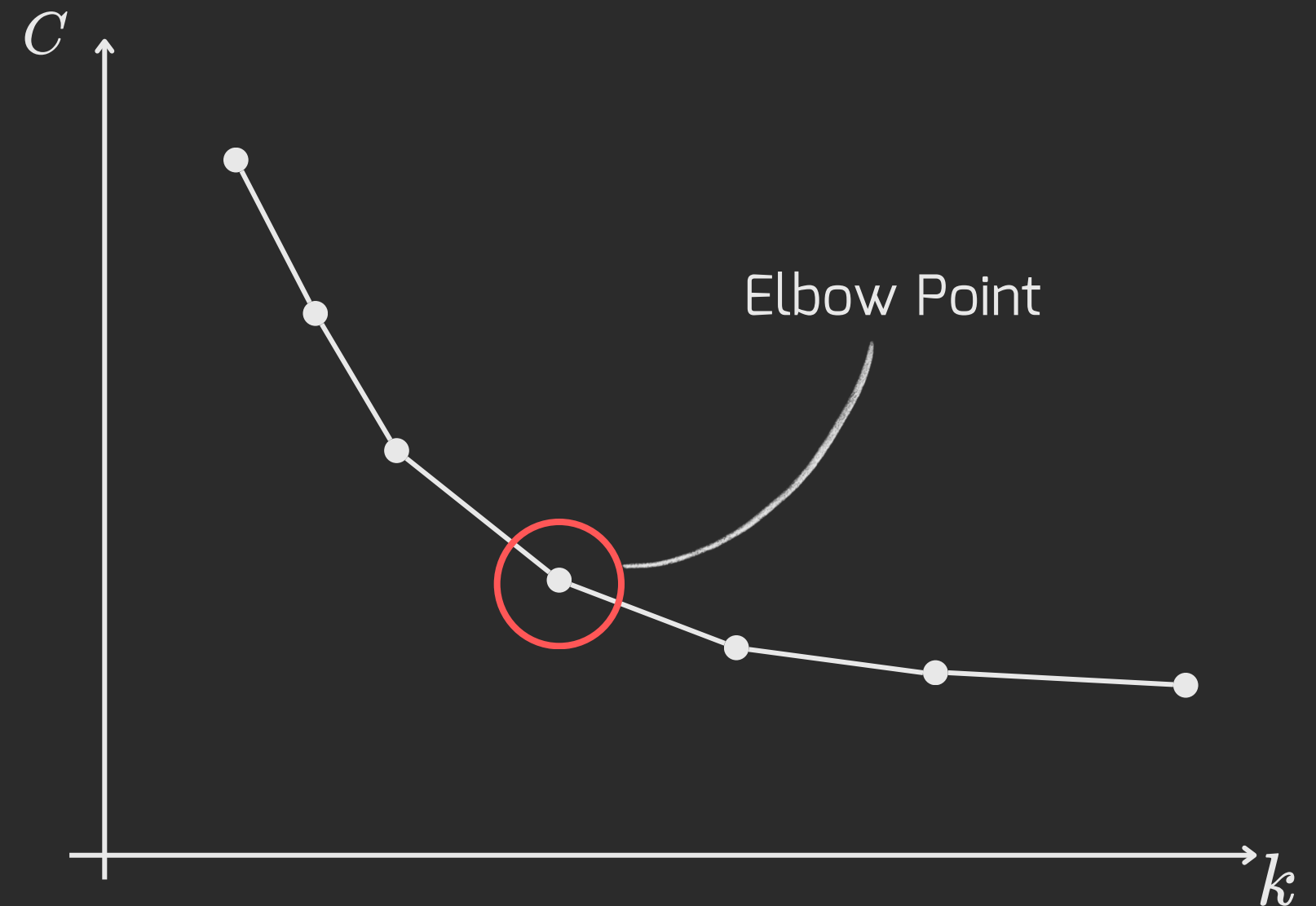
Elbow Point Method

The elbow is where the benefit from adding another cluster begins to fall substantially

Theory / Application

The data set has natural groupings – you want to confirm that data of the same group fall in the same groupings

SCREE PLOT



CHOOSING K

Silhouette Score

The score measures how similar a data point is to its own cluster compared to other clusters.

$$Score_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

a_i (Cohesion): The distance from a point to its own cluster compared to other clusters. Lower is better

b_i (Separation): The distance from a point in the nearest neighbouring cluster. Higher is better

`silhouette_score` from `sklearn` reports the average of silhouette score for all points in the dataset

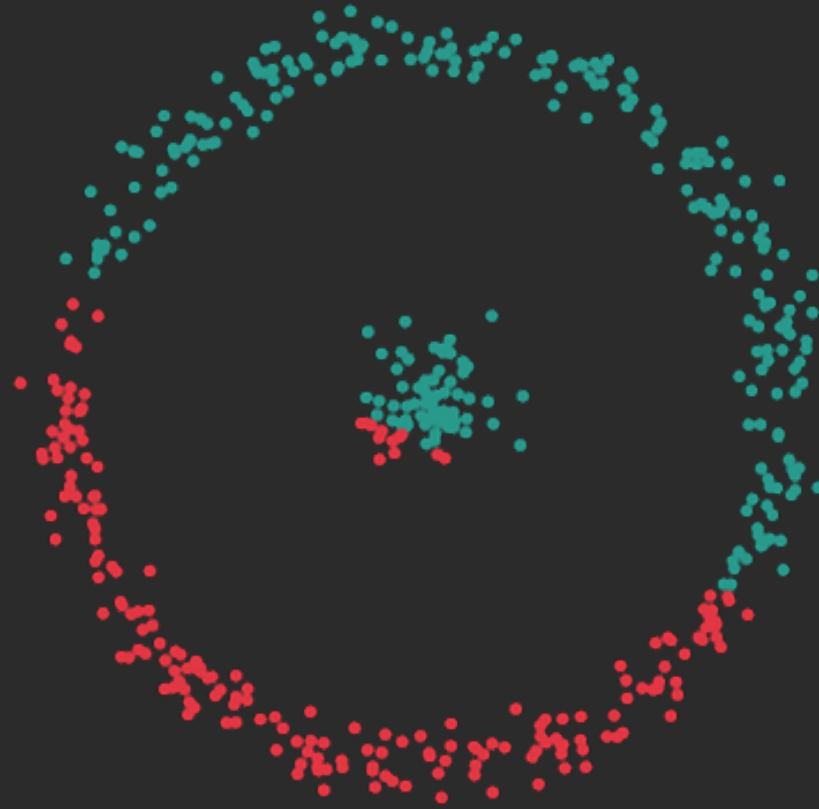
Interpreting Scores: $-1 < \text{Score} < 1$

- If silhouette score is close to 1: cluster is dense and well-separated
- if silhouette score is close to -1: data point may have been assigned to the wrong cluster

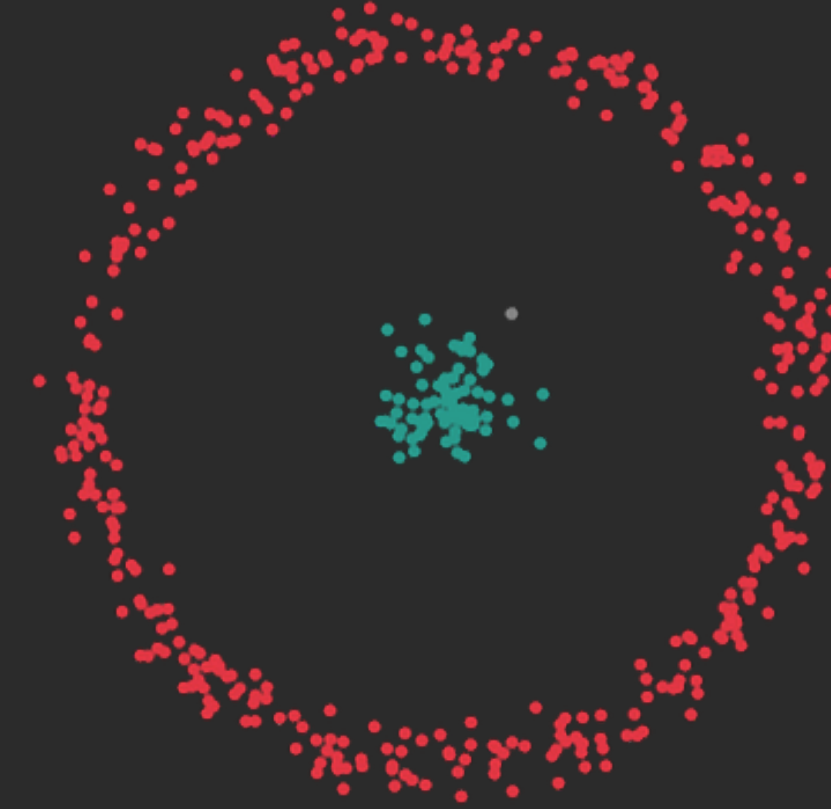
SILHOUETTE ANALYSIS PLOT



KMEANS



DBSCAN



DBSCAN

MOTIVATION

K-MEANS GROUPS OBSERVATIONS BASED ON THEIR DISTANCE TO CLUSTER CENTROIDS, MAKING IT MOST EFFECTIVE FOR **COMPACT, ROUGHLY SPHERICAL CLUSTERS**.

DBSCAN INSTEAD IDENTIFIES CLUSTERS **AS DENSE REGIONS** OF OBSERVATIONS, ALLOWING IT TO DISCOVER ARBITRARILY SHAPED CLUSTERS AND DISTINGUISH ISOLATED POINTS AS NOISE.

THE MECHANICS

Objective: To partition the data set into distinct groups based on feature similarity. (No K this time!)

Idea: Dense regions have many data points per unit space

MATH SPECIFICS

Counting
Neighbours

$$N_{\varepsilon}(x_i) = \{x_j : \|x_j - x_i\| \leq \varepsilon\}$$

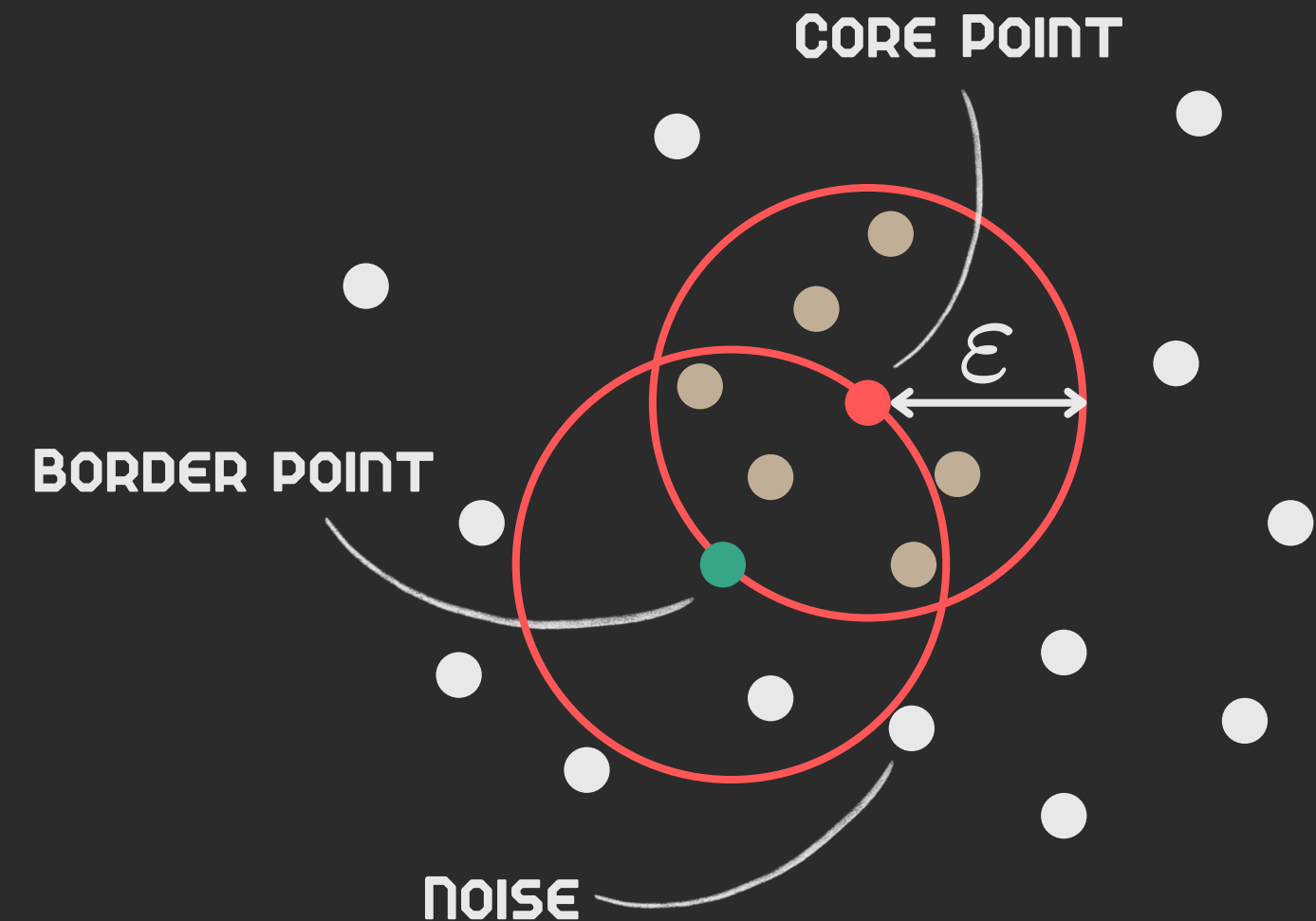
Setting
Core Points

$$|N_{\varepsilon}(x_i)| \geq MinPts$$

BONUS!

APART FROM CLUSTERING, DBSCAN ALSO DOUBLES AS AN ANOMALY DETECTION ALGORITHM

VISUALISATION



ALGORITHM

General Algorithm

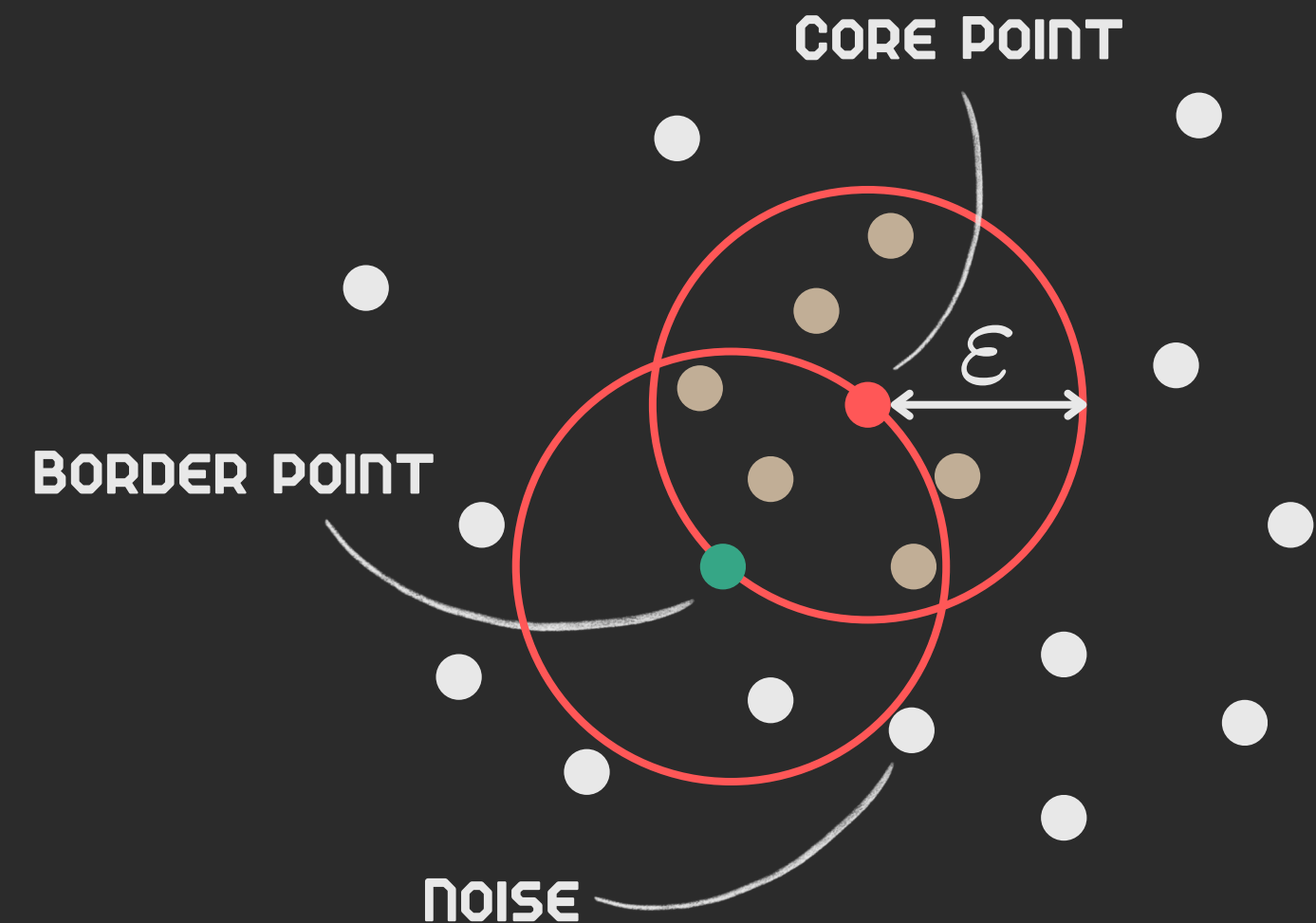
1. **Choose an epsilon (ϵ) and MinPts**
2. **Select a starting point** (automatically done by algo)
3. **Examine the Neighbourhood**
 - a. Algo retrieves points within ϵ distance
 - b. If number of neighbours $\geq$ MinPts. Point is core
 - c. If number of neighbours $<$ MinPts. Point is noise
4. **Expand the Cluster**
 - a. All neighbours of the core point is added to the cluster
 - b. If neighbour is core, all neighbours of this point is added to cluster
 - c. If neighbour is noise, label as border
5. **Repeat**

Parameters

To be demonstrated in Python Code

1. Minimum Points required to consider a core point (MinPts)
2. Epsilon (radius) within which defines the neighbourhood (ϵ)

VISUALISATION



PARAMETER TUNING

Choosing MinPts

- Choosing MinPts too large will result in a lot of data points being classified as noise.
- Choosing MinPts too small will result in too many core points

$$\text{MinPts} \geq d + 1$$

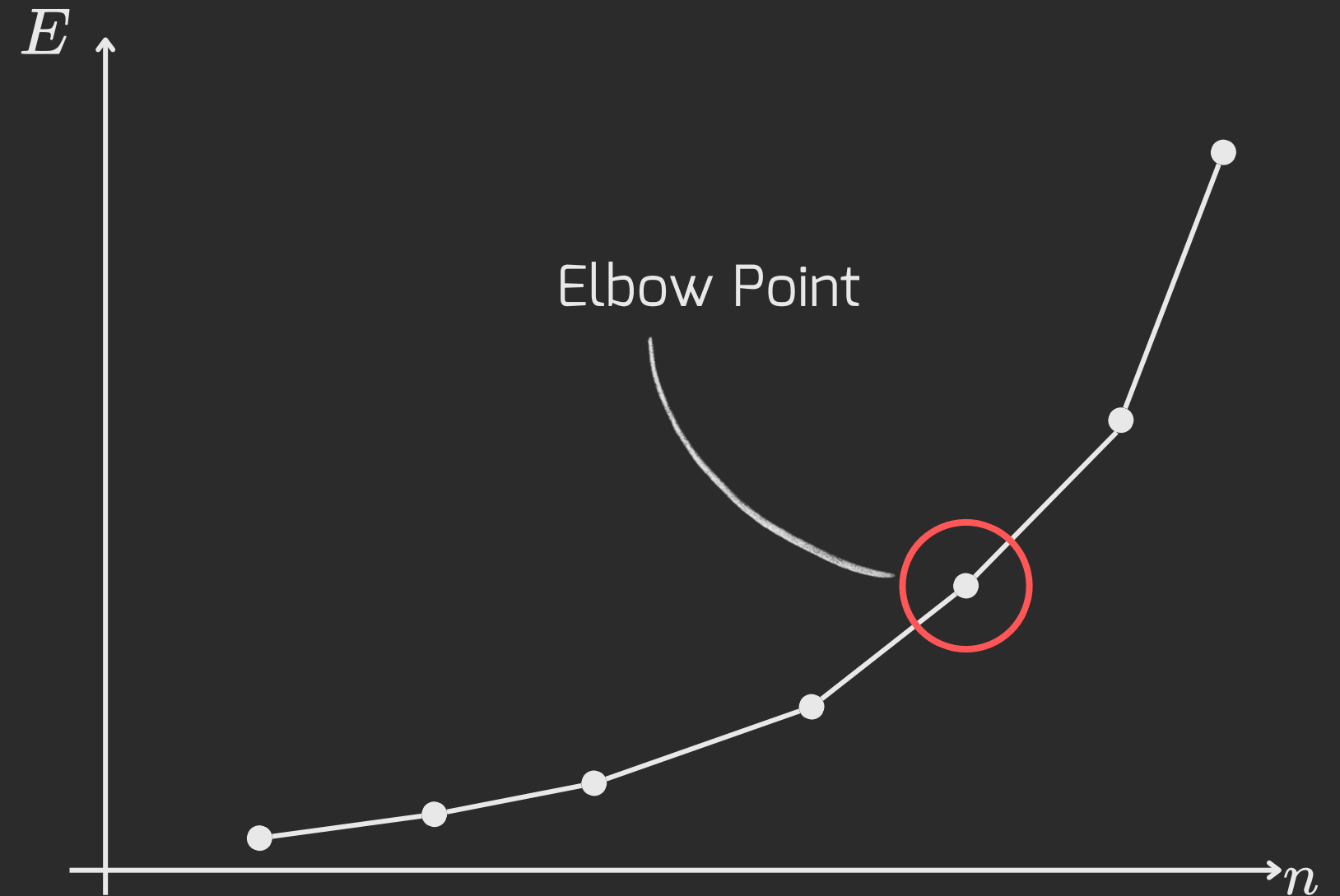
$$\text{MinPts} \approx 2d$$

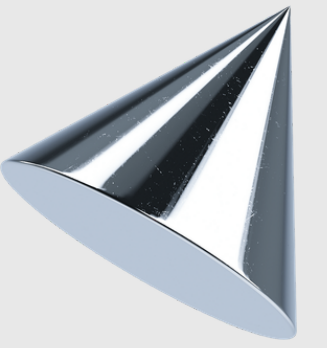
d = Number of dimensions

K-distance plot

1. Find the k-th nearest neighbour to each point (where $k = \text{MinPts}$)
2. Record the distance for each point
3. Sort them from smallest to largest
4. Plot
5. Find the Elbow/ Knee in the curve

K-DISTANCE PLOT



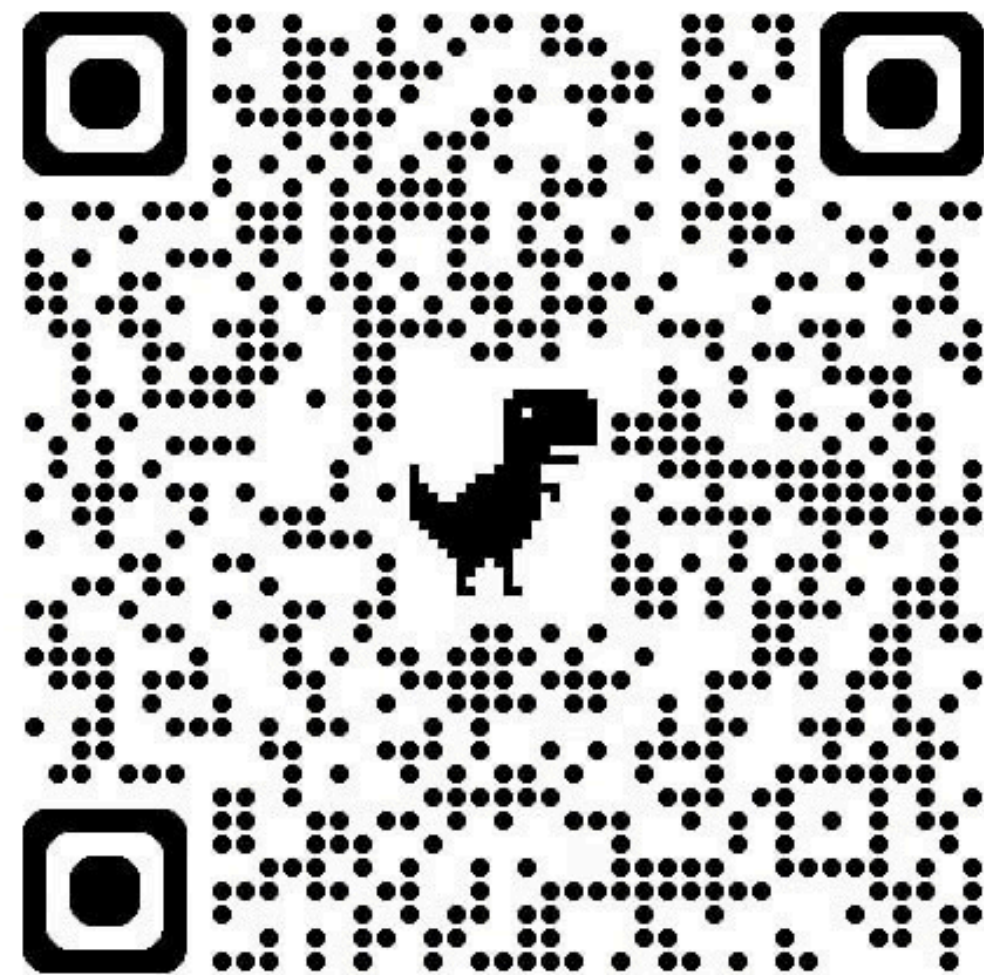
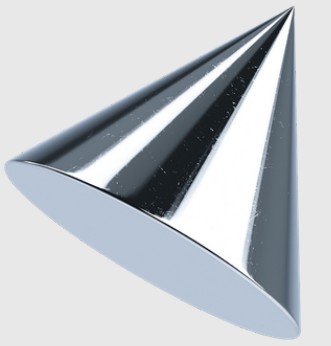


HANDS ON CODING

WE WILL SPLIT YOU GUYS INTO A GROUP OF 4 TO CONDUCT A MINI CODING PROJECT
[HTTPS://WWW.KAGGLE.COM/DATASETS/SAMARTALARWAR/SLEEP-DEBT-AND-SCREEN-TIME-LATE-NIGHT-PHONE-HABITS](https://www.kaggle.com/datasets/samartalarwar/sleep-debt-and-screen-time-late-night-phone-habits)



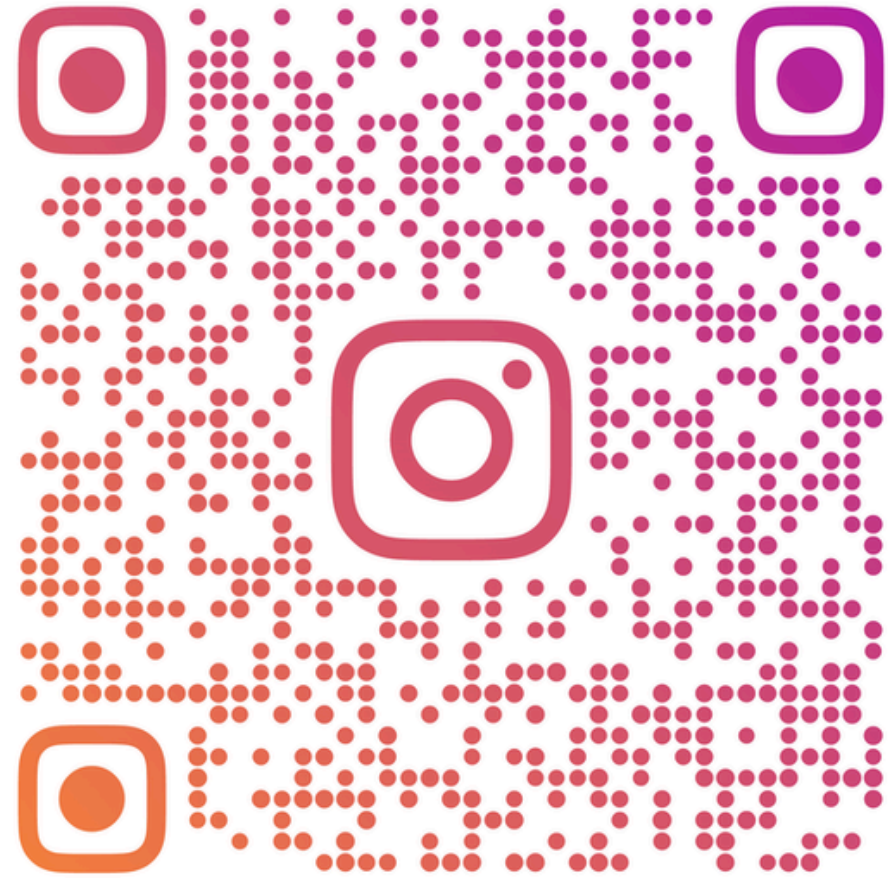
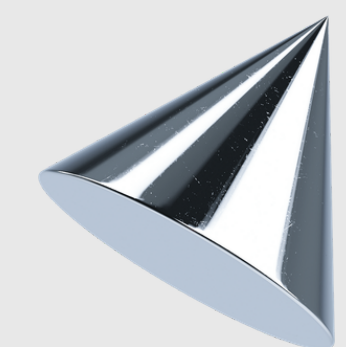
EVALUATION & FEEDBACK



[FORMS.GLE/UND1BBP2MJUVHT0S9](https://forms.gle/UND1BBP2MJUVHT0S9)

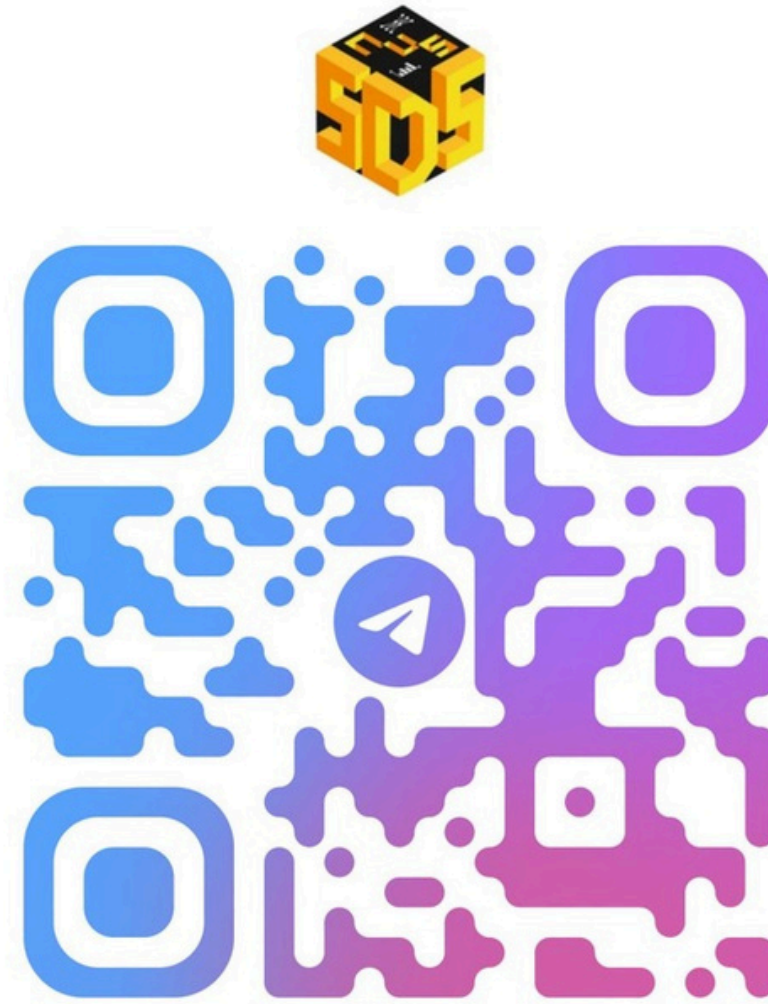


FOLLOW OUR SOCIALS TO STAY IN THE LOOP



NUS.SDS

SDS Instagram

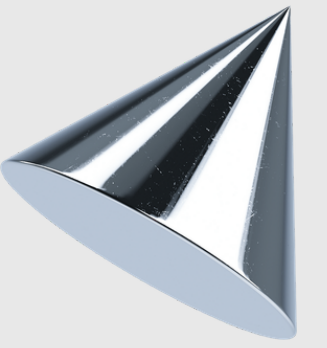


@NUSSDS

SDS Channel

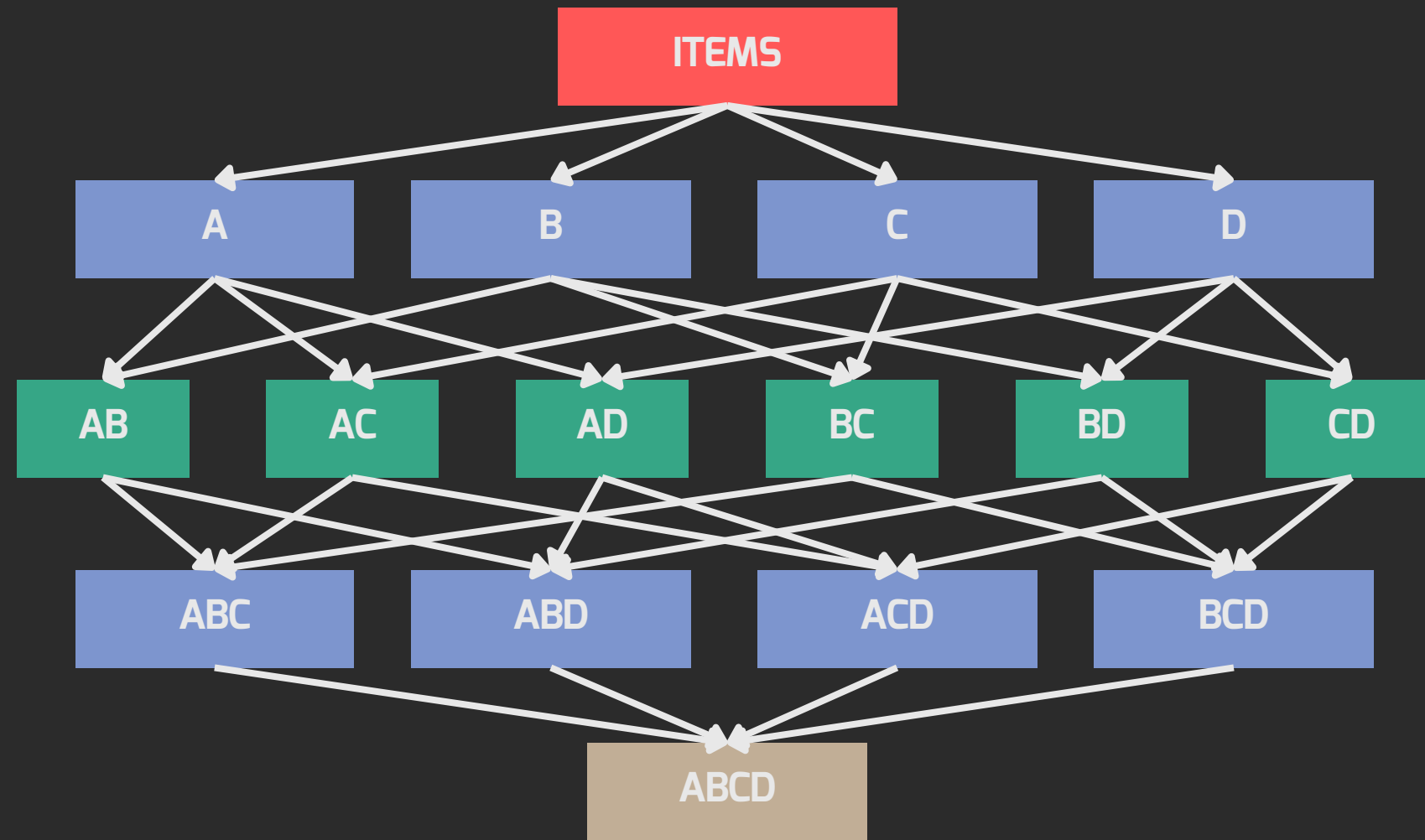


SDS Workshop GitHub



APPENDIX

FOR YOUR INFORMATION/ INTEREST!



ASSOCIATION RULE MINING IS AN UNSUPERVISED MACHINE LEARNING TECHNIQUE USED TO **UNCOVER HIDDEN RELATIONSHIPS**, PATTERNS, AND CO-OCCURRENCES BETWEEN VARIABLES IN LARGE DATASETS.

APRIORI IS DESIGNED TO DISCOVER **FREQUENT ITEMSETS** AND REVEAL HIDDEN RELATIONSHIPS WITHIN A LARGE TRANSACTIONAL DATASET

APRIORI

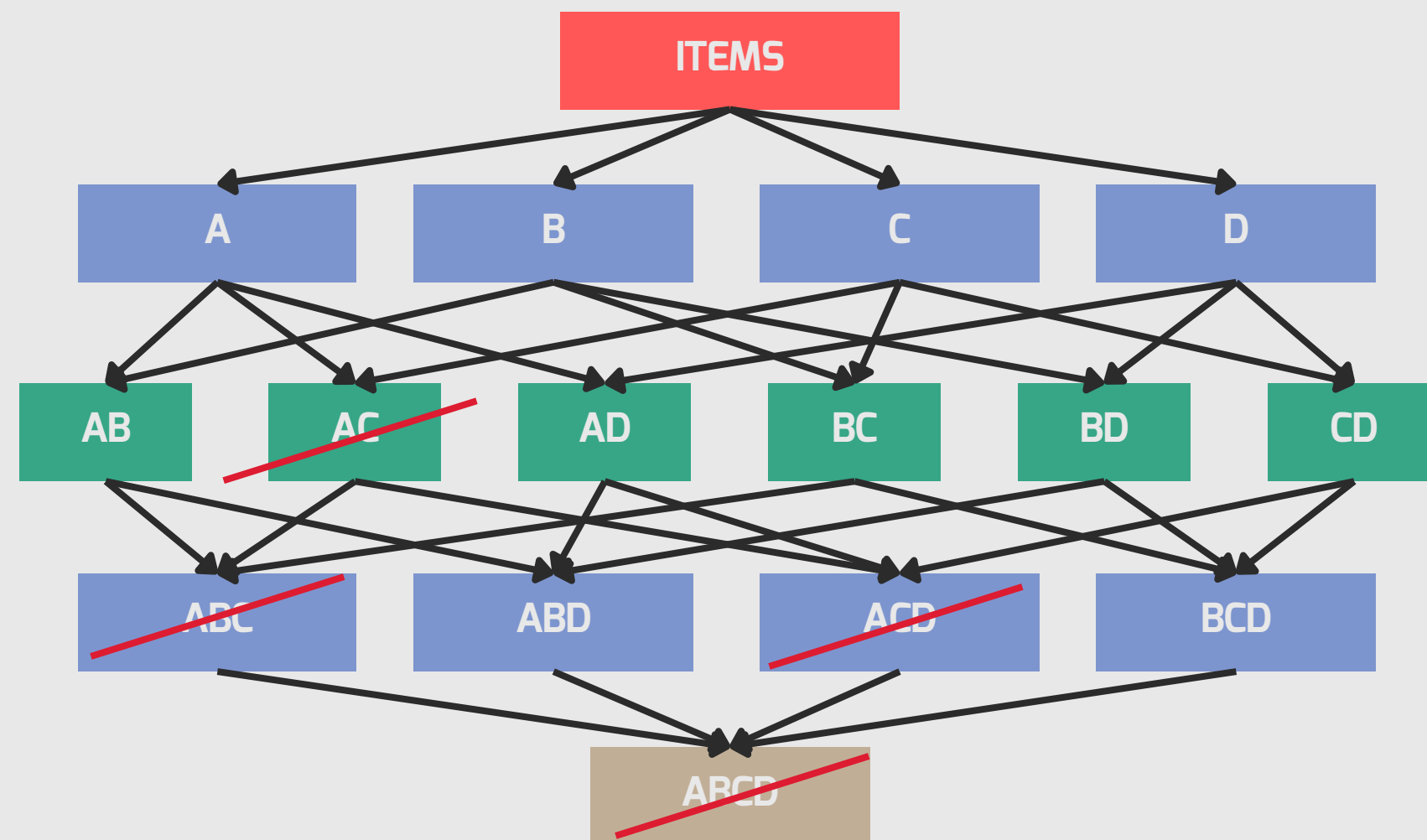
USE CASES:

- MARKET BASKET ANALYSIS
- RECOMMENDATION SYSTEMS
- HEALTHCARE AND MEDICAL ANALYSIS

APRIORI

Objective: Extract frequent itemsets from transactional databases and generate association rules

Idea: If an itemset is frequent, all its subsets must also be frequent



MATH SPECIFICS

Support

$$Support(A) = P(A) = \frac{\text{TRANSACTIONS CONTAINING A}}{\text{TOTAL TRANSACTIONS}}$$

Confidence

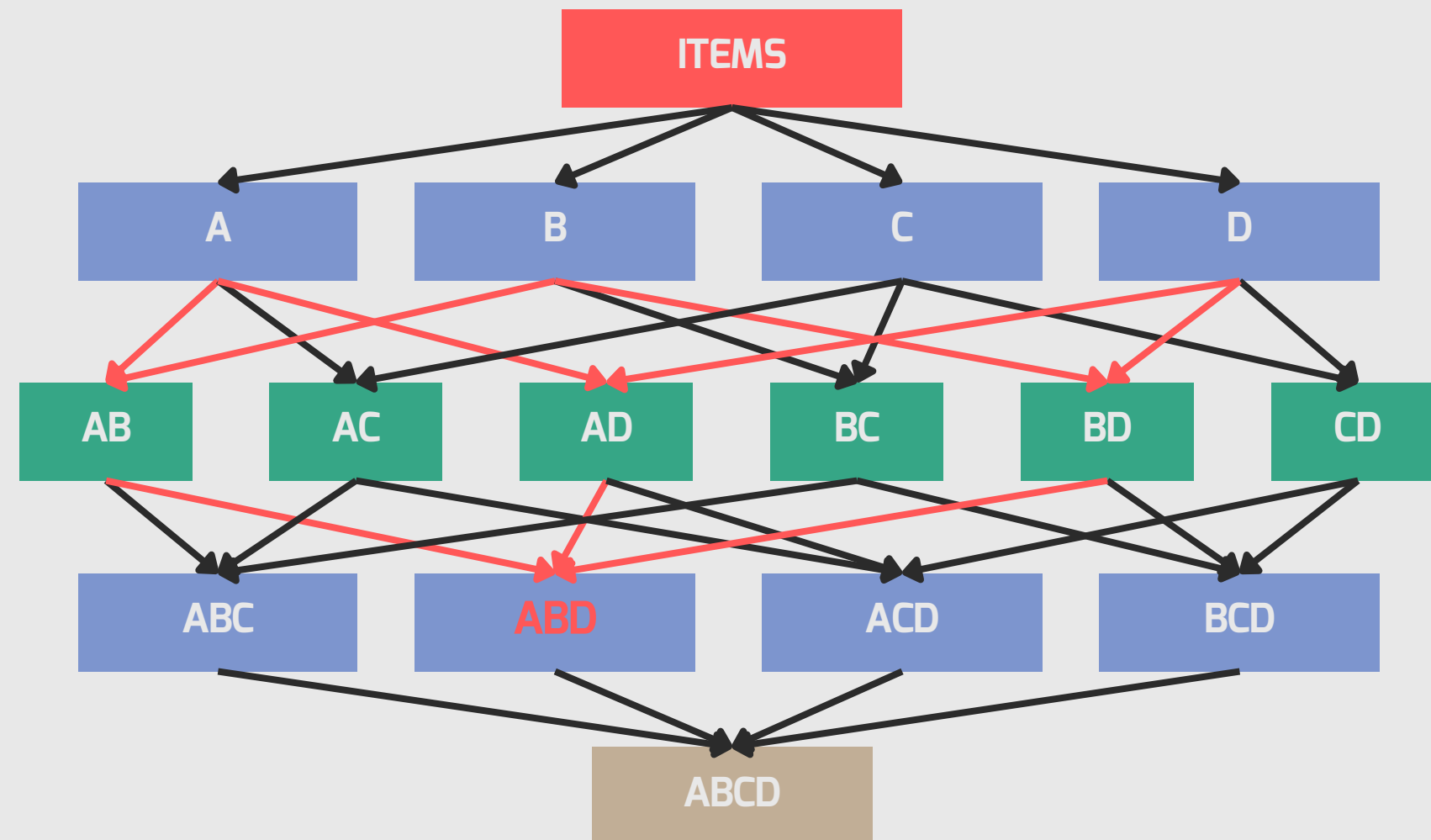
$$\begin{aligned} conf(A \rightarrow B) &= \frac{P(A \cap B)}{P(A)} = P(B|A) \\ &= \frac{\text{TRANSACTIONS CONTAINING A AND B}}{\text{TOTAL TRANSACTIONS CONTAINING A}} \end{aligned}$$

Lift

$$Lift(A \rightarrow B) = \frac{Conf(A \rightarrow B)}{Support(B)} = \frac{P(B|A)}{P(B)} = \frac{P(A \cap B)}{P(A)P(B)}$$

STAGE 1

YOU FOUND THAT **{ABD}** IS A FREQUENT ITEMSET



YOU CAN NOW FORM POSSIBLE RULES SUCH AS:

$A \rightarrow BD$

$B \rightarrow AD$

$AB \rightarrow D$

MATH SPECIFICS

Support

$$\text{Support}(A \cup B \cup D) = P(A \cap B \cap D)$$

$$= \frac{\text{TRANSACTIONS CONTAINING } A, B, D}{\text{TOTAL TRANSACTIONS}}$$

Confidence

$$\text{conf}(A \rightarrow B) = \frac{P(A \cap B)}{P(A)} = P(B|A)$$

Lift

$$\text{Lift}(A \rightarrow B) = \frac{\text{Conf}(A \rightarrow B)}{\text{Support}(B)} = \frac{P(B|A)}{P(B)} = \frac{P(A \cap B)}{P(A)P(B)}$$

STAGE 2

NOW YOU NEED TO EVALUATE ASSOCIATION RULES

LETS EVALUATE THE RULE $AB \rightarrow D$

AMONG THE TRANSACTIONS CONTAINING A AND B, HOW OFTEN DO THEY ALSO CONTAIN D?

$$\begin{aligned} \text{conf}(AB \rightarrow D) &= \frac{P(A \cap B \cap D)}{P(A \cap B)} \\ &= \frac{\text{Support}(ABD)}{\text{Support}(AB)} \\ &= \frac{0.30}{0.40} \quad \begin{array}{l} \text{LET SUPPORT}(ABD) = 0.3 \\ \text{LET SUPPORT}(AB) = 0.4 \end{array} \\ &= 0.75 \end{aligned}$$

75% OF TRANSACTIONS CONTAINING AB ALSO CONTAINS D

HIGH CONFIDENCE INDICATES THAT $AB \rightarrow D$ IS A POTENTIALLY USEFUL ASSOCIATION

MATH SPECIFICS

Support

$$\text{Support}(A) = P(A) = \frac{\text{TRANSACTIONS CONTAINING A}}{\text{TOTAL TRANSACTIONS}}$$

Confidence

$$\text{conf}(AB \rightarrow D) = \frac{P(A \cap B \cap D)}{P(A \cap B)} = P(D | A \cap B)$$

Lift

$$\text{Lift}(A \rightarrow B) = \frac{\text{Conf}(A \rightarrow B)}{\text{Support}(B)} = \frac{P(B|A)}{P(B)} = \frac{P(A \cap B)}{P(A)P(B)}$$

STAGE 3

DOUBLE CHECKING PROCESS

LETS EVALUATE THE RULE $AB \rightarrow D$

WHAT HAPPENS IF D IS EXTREMELY COMMON AND APPEARS IN 75% OF ALL TRANSACTIONS ANYWAYS?

$$\begin{aligned} Lift(AB \rightarrow D) &= \frac{Conf(AB \rightarrow D)}{Support(D)} \\ &= \frac{0.75}{0.75} = 1 \end{aligned}$$

IF THIS HAPPENS, KNOWING SOMEONE HAS A AND B TELLS US NOTHING NEW ABOUT D

SUPPOSE

$$Lift(AB \rightarrow D) = \frac{0.75}{0.3}$$

D IS 2.5 TIMES MORE COMMON AMONG TRANSACTIONS CONTAINING AB AS IT IS OVERALL

MATH SPECIFICS

Support

$$Support(A) = P(A) = \frac{\text{TRANSACTIONS CONTAINING A}}{\text{TOTAL TRANSACTIONS}}$$

Confidence

$$\begin{aligned} conf(A \rightarrow B) &= \frac{P(A \cap B)}{P(A)} = P(B|A) \\ &= \frac{\text{TRANSACTIONS CONTAINING A AND B}}{\text{TOTAL TRANSACTIONS CONTAINING A}} \end{aligned}$$

Lift

$$Lift(AB \rightarrow D) = \frac{Conf(AB \rightarrow D)}{Support(D)} = \frac{P(D|AB)}{P(D)} = \frac{P(A \cap B \cap D)}{P(AB)P(D)}$$

OVERALL ALGORITHM

1. GENERATE CANDIDATE ITEMSETS

Algorithm starts by identifying individual items and counting their occurrences to determine their frequencies

2. PRUNING BASED ON SUPPORT THRESHOLD

Items with frequencies (probabilities) less than the minimum support thresholds are pruned (removed)

3. GENERATING FREQUENT ITEMSETS

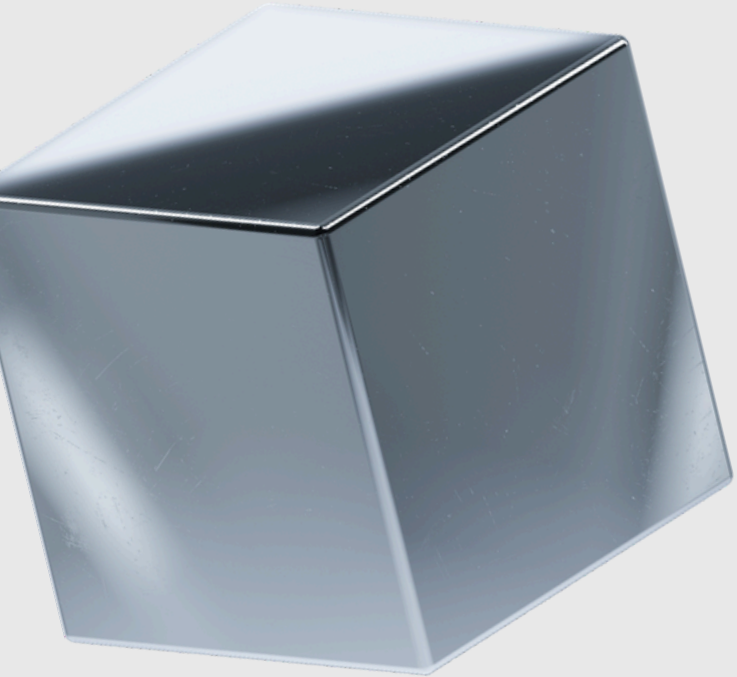
Algorithm combines smaller itemsets and counting their frequencies. It iterates this process until no more itemsets can be formed

4. DERIVING ASSOCIATION RULES (Stages 1-3)



ARMA MODEL

...

- Until now, we have introduced basic forecasting methods (like SES) that extrapolate past patterns forward as a constant flat line.
 - But what if we want to capture short-term dynamics so that instead of a flat line, we project immediate patterns forward?
 - For this kind of task, we need a more flexible model—such as an ARMA model.
- 

AUTOREGRESSIVE MODEL

- In an autoregression model, we forecast the variable of interest using a linear combination of past values of the variable.
- This is called AR(P) process where P is the number of past values to be used.

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t,$$

- For example, AR(2) process calculates the immediate future value by substituting the values for the two most recent observations.

MOVING AVERAGE MODEL

- Rather than using past values of the forecast variable in a regression, a moving average model uses past **forecast errors** in a regression-like model.
- MA(q) model means that the current value can be estimated through modeling the ripple effect of q previous forecast errors.

$$y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \dots + \theta_q \varepsilon_{t-q},$$

MA models assume the underlying series is stationary, meaning it fluctuates around a constant mean with constant variance. We are simply modeling the short-term fluctuations around that mean.

ARMA MODEL

$$y_t = \underbrace{c}_{\text{Baseline}} + \underbrace{\phi_1 y_{t-1} + \dots + \phi_p y_{t-p}}_{\text{AR Component (Past Values)}} + \underbrace{\theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}}_{\text{MA Component (Past Shocks)}} + \underbrace{\varepsilon_t}_{\text{New Shock}}$$

- Combining AR and MA models, we get an ARMA(p,q) model (AutoRegressive Moving Average model)
- p and q stand for the order of AR and MA terms respectively.
- Since it's an extension of MA models, it still assumes stationarity.
- ARMA models are a subset of ARIMA models where I stands for 'integrated' and is applicable for non-stationary series as well.

MODEL EVALUATION – AICC

$$\text{AIC} = -2 \log(L) + 2k, \quad \text{AIC}_c = \text{AIC} + \frac{2k(k+1)}{T-k-1}$$

- **Log(L)** is the log-likelihood of the model's errors following a normal/gaussian distribution. This measures model fit.
- **K** is the number of estimated parameters in the model (such as AR coefficients, variance, etc)
- **T** is the sample size
- Lower AIC or AICc indicates a better model.

Standard AIC assumes a large sample size T . When sample sizes are small (or K is large relative to T), standard AIC tends to under-penalize complexity, leading to overfitted models with too many parameters.

→ We add a correction term to AIC to increase penalty on parameters, especially when T is small

MODEL EVALUATION – MASE

MASE asks how good the forecasts were for the test years.

1. Each test error is divided by the average error the naive method made one step ahead on the training years
2. Then, we take the mean of these values to find the MASE for the forecast.

$$q_j = \frac{e_j}{\frac{1}{T-1} \sum_{t=2}^T |y_t - y_{t-1}|}, \quad \text{MASE} = \text{mean}(|q_j|)$$

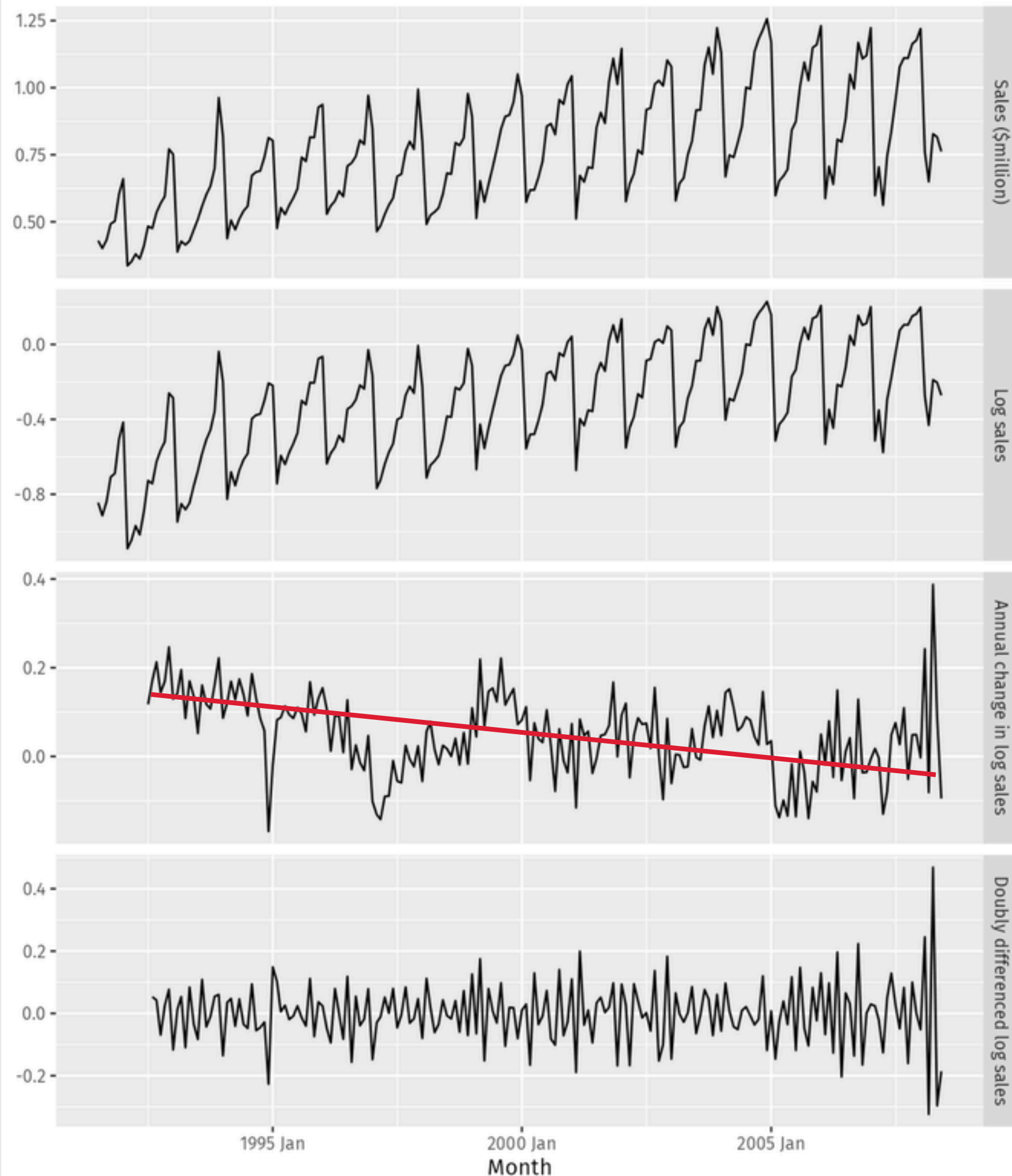
Numerator: model's absolute error on the test years

Denominator: average error the naive method would have made one step ahead there

Below 1 means that forecast performed better than naive errors from training data.

Above 1 means it did worse.

Corticosteroid drug sales



ASSUMPTIONS

Stationarity

- A stationary time series has statistical properties (mean, variance) that remain constant over time.
- **Why it matters:** Ensures past statistical patterns hold true for future predictions.

Trends & Seasonality

- **Trends:** Non-stationary series with clear upward/downward trends are made stationary through differencing (e.g., analyzing period-to-period changes instead of absolute levels).
- **Seasonality:** Periodic repeating patterns are handled by extending models (e.g., SARMA / SARIMA).